diffmrc Manual

Reference, methods, and verification records

Version 0.3

The reference document for the diffmrc package: installation and configuration, the API surface, protocol details, the design of the differentiable chain and its second derivatives, the verification records, benchmark results, and the development architecture. The companion *Tutorial* covers first use.

Contents

I User reference **7**

1 Overview and package layout **8**
1.1 Design principles . . . 8
1.2 Package layout . . . 8
1.3 How the documents are organized . . . 9

2 Installation and configuration **10**
2.1 Supported environment . . . 10
2.2 Runtime configuration . . . 10
2.3 Verifying the installation . . . 10
2.4 Optional: running the figures . . . 11
2.5 Known environment friction . . . 11

3 API reference **12**
3.1 core: the numerical kernel . . . 12
3.2 analysis: NumPy only . . . 13
3.3 policy: selection rules . . . 13
3.4 driver: the differentiable pipelines . . . 14
3.4.1 Effective core potentials . . . 14
3.5 cli . . . 15

4 Protocols and options **16**
4.1 Orbital-point convergence . . . 16
4.2 State averaging . . . 17
4.3 Active-space windows . . . 17
4.4 The e2_scale switch . . . 18
4.5 Differentiation settings . . . 18
4.6 MC-PDFT on-top functionals (tPBE, tPBE0) . . . 18

II Methods and records **20**

5 The differentiable chain **21**
5.1 Purpose and scope . . . 21
5.2 Theory and derivation . . . 21
5.2.1 Exponential parameterization . . . 21
5.2.2 Energy derivative . . . 22
5.2.3 The envelope theorem . . . 22
5.2.4 Non-variational corrections and the Z-vector equation . . . 23
5.3 Input and operation . . . 24
5.4 Worked example . . . 25
5.4.1 H_2/STO-3G: the zero-correlation anchor . . . 25

5.4.2 H_2O/6-31G: AD against the finite-difference golden standard . . . 25
5.4.3 N_2/6-31G: the parameter set is part of the answer . . . 26
5.5 Pitfalls . . . 26
5.6 Validation and references . . . 26

6 Second derivatives **28**
6.1 Purpose and scope . . . 28
6.2 Theory and derivation . . . 28
6.2.1 Implicit differentiation of the stationarity condition . . . 28
6.2.2 The pseudo-inverse and null-space exclusion . . . 29
6.2.3 The unrolled route and its consistency . . . 29
6.2.4 Hessian-vector products . . . 30
6.3 Input and operation . . . 30
6.4 Worked example . . . 30
6.4.1 H_2/STO-3G: orbital-space HVP against a finite-difference ladder . . . 30
6.4.2 H_2O/6-31G: relaxed Hessian against the golden finite-difference standard . 31
6.4.3 CO/6-31G: the frequency anchor . . . 31
6.5 Pitfalls . . . 31
6.6 Validation and references . . . 32

7 Verification records **33**
7.1 Energy chain . . . 33
7.2 First derivatives . . . 33
7.3 Second derivatives . . . 34
7.4 External-data cross-checks . . . 34
7.5 The layer contracts . . . 34

8 Benchmarks **35**
8.1 Ground-state line (M5-A) . . . 35
8.2 Excited-state line (QUEST subset) . . . 35
8.3 Selection-protocol cross-check (DVS-APC) . . . 36
8.4 What is not benchmarked . . . 36

9 Known limitations **37**
9.1 Method-family limits . . . 37
9.2 Implementation limits . . . 37
9.2.1 Degenerate pairs at a stationary point . . . 38
9.3 Protocol limits . . . 38

III Development **39**

10 Architecture and contracts **40**
10.1 The six layers . . . 40
10.2 Contracts are tests . . . 40
10.3 How the test suite is organized . . . 40
10.4 Development workflow . . . 41

11 Reproducing the paper **42**
11.1 Environment . . . 42
11.2 The paper's figures and tables . . . 42
11.3 The later result lines . . . 43
11.4 Building this documentation . . . 43

IV Acceleration and release extensions **44**

12 Resolution of identity (RI/DF) **45**
12.1 Purpose and scope . . . 45
12.2 Theory and derivation . . . 45
12.2.1 Density fitting as a least-squares problem . . . 45
12.2.2 The $BS^{-1/2}$ factorization . . . 46
12.2.3 Derivative of the symmetric inverse square root . . . 46
12.2.4 Error sources . . . 46
12.2.5 Memory scaling . . . 47
12.3 Input and operation . . . 47
12.3.1 Axis and sign conventions . . . 48
12.4 Worked example . . . 49
12.4.1 H_2O/6-31G: assembly, CASCI, and one derivative check . . . 49
12.4.2 N_2/cc-pVDZ and CO/cc-pVDZ: the physical error . . . 49
12.4.3 Memory at the two measured sizes . . . 49
12.4.4 A standalone gradient script . . . 50
12.5 Pitfalls . . . 50
12.6 Validation and references . . . 51

13 Performance and numerical hygiene **52**
13.1 MO integral transforms: four sequential contractions . . . 52
13.1.1 Why the one-shot form is not used . . . 52
13.1.2 Accuracy on the release tree . . . 53
13.2 The S_z-blocked CI solver . . . 53
13.2.1 Memory and time at two determinant counts . . . 54
13.3 Automatic solver selection . . . 54
13.3.1 What the switch leaves invariant . . . 55
13.3.2 The small-system penalty the default avoids . . . 55
13.4 Reference arrays as jit arguments . . . 55
13.5 Repeated core folding in the CI matrix–vector product . . . 56
13.6 Memory-bounded integral derivatives . . . 56
13.6.1 Measured peak memory . . . 57
13.7 Choosing among the options . . . 58

V New in version 0.3 **59**

14 New modules in version 0.3 **60**
14.1 Relaxed NEVPT2 nuclear gradients . . . 60
14.2 Coupled CASPT2 . . . 62
14.3 Valence-level SOMF for effective-core potentials . . . 63
14.3.1 Element range and validation . . . 65
14.4 RIJONX SCF . . . 65
14.5 Scalar DF-X2C CASSCF . . . 67
14.6 Validation scope and boundaries . . . 69

15 CASPT2 class structure and contraction paths **70**
15.1 Purpose and scope . . . 70
15.2 Theory and derivation . . . 70
15.2.1 Reference, excitation operators, and the first-order wavefunction . . . 70
15.2.2 The projected amplitude equation . . . 71
15.2.3 The diagonal approximation . . . 71

15.2.4 The VP/VM pairing . . . 72
15.2.5 The eight classes . . . 72
15.2.6 The three-active-index classes . . . 72
15.2.7 The coupled full-Fock system . . . 73
15.2.8 The three-center contraction . . . 73
15.3 Input and operation . . . 73
15.4 Worked example . . . 74
15.4.1 N_2/6-31G CAS(6,6): three paths . . . 74
15.4.2 H_2O/6-31G and CO/6-31G: the CAS(2,2) class table . . . 75
15.4.3 Running the three contraction routes . . . 75
15.5 Pitfalls . . . 76
15.6 Validation and references . . . 77

16 Relativistic one-electron layers: scalar and two-component X2C 79
16.1 Purpose and scope . . . 79
16.2 Theory and derivation . . . 79
16.2.1 The four-component problem in block form . . . 79
16.2.2 Block decoupling and the Foldy–Wouthuysen Hamiltonian . . . 80
16.2.3 Why the scalar and two-component energies differ . . . 80
16.2.4 The two-electron part and picture change . . . 80
16.2.5 The SOMF operator . . . 81
16.2.6 The ECP spin–orbit term . . . 81
16.3 Input and operation . . . 82
16.3.1 Common conventions . . . 82
16.3.2 Scalar DF-X2C CASSCF . . . 82
16.3.3 Two-component DF-X2C Hartree–Fock . . . 84
16.4 Worked example . . . 85
16.4.1 OH and OF: one-electron versus SOMF spin–orbit splitting . . . 85
16.4.2 GeF with an ECP: the ECP spin–orbit operator . . . 85
16.4.3 H_2O/6-31G: two-component X2C acceptance bounds . . . 86
16.5 Pitfalls . . . 86
16.5.1 Scalar layer . . . 86
16.5.2 Two-component layer . . . 86
16.5.3 SOMF and ECP . . . 87
16.6 Validation and references . . . 87

17 MPS/DMRG support in the release 88
17.1 Purpose and scope . . . 88
17.2 Theory and derivation . . . 88
17.2.1 Degenerate subspaces and gauge freedom . . . 88
17.2.2 The pinning map . . . 89
17.2.3 Conditions and failure modes . . . 89
17.3 Input and operation . . . 89
17.4 Worked example . . . 90
17.4.1 N_2/6-31G: the recorded start window {6,7} . . . 90
17.4.2 N_2/6-31G: the exhaustive-window best . . . 90
17.5 Pitfalls . . . 90
17.6 Validation and references . . . 91

18 Manifold-merged entanglement entropy **92**
18.1 Purpose and scope . . . 92
18.2 Why one state is not enough . . . 92
18.2.1 The four occupation weights . . . 92
18.2.2 The $M_S = +S$ component of a high-spin determinant . . . 92
18.2.3 Merging the manifold . . . 93
18.3 Which roots form the manifold . . . 93
18.4 API . . . 93
18.5 The robust-level gate . . . 94
18.6 Worked example: the nitrogen quartet . . . 94
18.7 Limitations . . . 95

Part I

User reference

Chapter 1

Overview and package layout

1.1 Design principles

Three commitments shape the package, and most of its structure follows from them.

The first is that derivatives are never written by hand. Every nuclear gradient and second derivative is obtained by differentiating the energy code path with JAX. This is what makes the CASSCF, state-averaged CASSCF, and NEVPT2 layers share one implementation of the physics: a correction to the energy expression propagates to all of its derivatives without a separate derivation.

The second is that the package is layered, and the layer boundaries are tested. The kernel cannot import the layers above it; the analysis layer cannot import the kernel at all. The contract is enforced in the test suite (see Chapter 10), so a violation is a failing test rather than a matter of review discipline.

The third is that convergence is reported, not assumed. Optimizers return the gradient norm at the point they stopped at, and the package distinguishes "the optimizer stopped improving" from "the point is stationary." Callers are expected to carry that distinction into whatever they claim about a result.

1.2 Package layout

```
src/diffmrc/
  core/       numerical kernel: rotations, CASCI, CASSCF, NEVPT2,
              stationary points, the frozen source of the chain
  analysis/   post-processing that reads converged objects: entropies,
              mutual information, spin and state identification,
              orbital identity and frontier windows; NumPy only
  policy/     selection rules over analysis output: composition,
              entropy-based and ranked orbital selection, verification
  driver/     composition layer: assembles the kernel into the
              differentiable SCF -> CASSCF -> NEVPT2 pipelines,
              orbital-point convergence, frozen-geometry factories
  report/     result objects and rendering
  cli/        the command-line entry point
```

The dependency direction runs downward only: `core` at the bottom, `cli` at the top. A user script may import any layer; the layers themselves may not import upward. This is what allows the analysis layer to be tested and used with nothing but NumPy installed.

1.3 How the documents are organized

This manual is the reference. Part I covers installation, the API surface, and the protocol details that the code reports but does not enforce. Part II carries the methods: the design of the differentiable chain, the second-derivative construction, the verification records against independent implementations, the benchmark results, and the consolidated limitation list. Part III is for development: layer contracts, the test architecture, and the reproduction steps for the paper. Part IV collects the acceleration layer and the release extensions: the RI/DF path, the performance choices, the new modules, the CASPT2 class structure and its three contraction paths, the scalar and two-component X2C layers, and the MPS/DMRG support that is present in the release tree.

The companion *Tutorial* is the on-ramp: installation, a first CASSCF gradient, state-averaged workflows, active-space selection, and a full benchmark walkthrough, each with runnable code.

Chapter 2

Installation and configuration

2.1 Supported environment

The package is developed and verified on Linux and Windows with CPython 3.11, on CPU. The Tutorial (Chapter 2) gives the minimal recipe; this chapter records the constraints behind it.

Dependency	Constraint	Why it is pinned
Python	≥ 3.10	`int.bit_count` in the determinant tools
numpy	≥ 1.24	analysis layer, reference arithmetic
scipy	≥ 1.10	L-BFGS-B and trust-region stages
pyscfad	$\geq 0.3.3$	provides the traceable SCF and integral path
pyscf	≥ 2.5	reference energies and comparisons
jax	≥ 0.4	array layer; the exact pin is pyscfad's
matplotlib	any recent	figure sources only

`pyscfad` pins a specific JAX version, which is why the package should live in its own environment. Mixing it into an environment with a different JAX pin produces failures that look like numerical bugs and are not.

2.2 Runtime configuration

Three settings are load-bearing.

`jax_enable_x64 = True`. The package is validated in double precision. With the default single precision the errors are not loud: gradients lose several digits and some convergence gates become unreachable, while energies look plausible.

`JAX_PLATFORMS = cpu`. The CPU platform is the verified configuration. On a machine with a CUDA device, leaving the platform unset selects the GPU, where the solver path used by the orbital canonicalization raises an error unrelated to the chemistry. Set the variable before importing JAX; setting it afterwards has no effect.

Thread environment variables. The driver assumes it owns the thread budget it was given. Setting the five BLAS thread variables keeps multiple concurrent calculations from oversubscribing a node; their absence does not corrupt results, it makes timing meaningless and slows everything.

2.3 Verifying the installation

```
pytest tests/ -v
lint-imports        # or: pytest tests/test_layer_contracts.py
```

The suite is the primary artifact to look at: it contains the first-order and second-order derivative checks against finite differences, the cross-validation against PySCF, and the layer contracts. A passing suite on your machine is the intended proof that the environment is sound.

2.4 Optional: running the figures

The figure sources under `figures/` import matplotlib and read JSON results produced by the scripts; they require no additional configuration. They are not part of the package tests.

2.5 Known environment friction

- The first evaluation of a freshly built node of the chain triggers XLA compilation that can take minutes. This is expected; a stopwatch on a single evaluation will mislead.
- pyscfad's `Mole` objects cannot cross a `jax.jit` boundary. The driver layer offers frozen-geometry factories for jitted contexts; the factory docstrings state which is which.
- On shared nodes, leave headroom for the owner of the other processes. The memory profile of this package is dominated by compilation- and evaluation-time buffers of the differentiated chain, not by the integrals, and it grows steeply with the number of rotation parameters.

Chapter 3

API reference

This chapter lists the entry points a user is expected to reach for, layer by layer. It is a map, not a transcription: the authoritative signature of every function is its docstring in the source, and the module-level `__all__` lists are the canonical inventories. Entries marked *stable* have been exercised by the verification records in Chapter 7; the rest follow the same conventions but have lighter coverage.

3.1 core: the numerical kernel

Entry point	Purpose
`rotation.block_rotation(t, nmo, pairs)`	rotation matrix from the parameter vector
`orblib.orbital_pairs(ncore, nact, nmo)`	the non-redundant rotation pairs
`casci.cs_casci(...)`	CASCI energies; state-averaged branch with `nroots`, `return_ci`
`optimizer.optimize_theta_scipy(...)`	L-BFGS-B stage
`optimizer.optimize_theta_hand(...)`	the fallback when scipy raises
`grad.make_energy_fn / make_grad_fn`	energy and nuclear-gradient callables
`grad.solve_theta_star(...)`	the two-stage orbital optimization
`grad.nuclear_gradient(mol, grad_fn)`	the differentiated gradient
`stationary.refine_stationary_point(...)`	damped-Newton refinement (Newton–CG step)
`stationary.curvature_report(H)`	the saddle-point check
`stationary.StopReason / StationaryPoint`	convergence verdict types
`nevpt2.impl.cas_dets_for2(...)`	determinant bitmaps of the active space
`nevpt2.impl.canonicalize_jax(...)`	orbital canonicalization on the differentiated path
`nevpt2.impl.nevpt2_e2_full2(...)`	the frozen NEVPT2 contraction
`mcpdft.w_sigma / pi_active`	on-top densities: spin 1-RDM blocks and the factorized Π
`mcpdft.eot_energy / mcpdft_eot`	the on-top energy, kernel injected (tPBE)
`mcpdft.pbe_exc / pbe_eval_xc`	the closed-form PBE kernel (jit-safe)
`mcpdft.exchange_energy(...)`	the hybridś HF-like exchange term (diagnostic)

The kernel is frozen: later layers build on it and corrections to it go through the verification records, not through ad-hoc edits.

3.2 analysis: NumPy only

Entry point	Purpose
`states.state_groups / select_groups`	roots to physical states; state selection
`states.spin_square / det_abs_sz`	exact $\langle S^2 \rangle$ and $\lvert M \rvert$ of determinants
`orbitals.frontier_report / classify_orbital`	frontier identities; window suggestion
`orbitals.heavy_atoms / p_axis_profile`	atom keys; per-axis p profiles
`ci.occupations_from_ci / entropy_from_ci`	occupations and entropies from a CI vector
`ci.manifold_entropy_from_ci`	the same entropy over a merged spin manifold
`ci.pair_density / mutual_information`	two-orbital reductions and their correlations
`rdm.entropy_from_rdms`	the density-matrix route to the same quantities
`diagnostics.is_single_reference / zs1`	multireference diagnostics
`plateau.detect_plateaus`	entropic plateaus of an orbital set

This layer imports nothing from the kernel; it operates on the determinant-bitmap convention and on arrays, and its tests run with NumPy alone.

3.3 policy: selection rules

The policy layer turns analysis output into selections: composition-based rules, entropy-based ordering, the ranked orbital selection with its singly-occupied and virtual-removal options, and the verification helpers. The module names are the inventory (`policy.composition`, `policy.entropy`, `policy.ranking`, `policy.rules`, and their siblings); the docstrings state the conventions each rule follows.

3.4 driver: the differentiable pipelines

Entry point	Purpose
`nevpt2.make_nevpt2_joint_fn(...)`	(mol, θ) SCF→CASSCF→NEVPT2, geometry-differentiable
`nevpt2.make_nevpt2_theta_fn(...)`	frozen geometry, θ only; jit-safe
`nevpt2.make_nevpt2_sa_fn(...)`	state-averaged joint factory
`nevpt2.make_nevpt2_sa_theta_fn(...)`	state-averaged, frozen geometry, jit-safe
`nevpt2.sa_cas_solution(...)`	state-averaged CAS solution and CI vectors
`nevpt2.sa_nevpt2_state_table(out, dets)`	grouped physical-state table
`nevpt2.sa_vertical_excitations(states)`	per-state vertical excitation energies
`kernel.mcpdft_energy_function(...)`	the differentiable one-step tPBE chain (`hyb` for tPBE0)
`kernel.mcpdft_terms_function(...)`	the same chain with its four energy components
`nevpt2.sa_mcpdft_table(...)`	SA-CASSCF + per-state one-step on-top column, explicit window
`nevpt2.sa_mcpdft_table_from_mol(...)`	the same with the frontier window rules
`dvs.dvs_scan(mol, ...)`	the (lambda, N) DVS candidate scan and selection
`kernel.frozen_grid(mol)`	the frozen quadrature (AO values, gradients, weights)
`orbitals.converge_orbital_point(E_theta, npar)`	two-stage convergence with refinement and polish
`kernel.scf_reference(..., ecp=None)`	the reference SCF; `ecp` reaches `Mole.ecp`
`kernel.compute_ground_state(...)`	the self-checked end-to-end run; `params['ecp']` is forwarded
`orbitals.hessp_of(E_theta)`	Hessian-vector product for the trust-region stage

3.4.1 Effective core potentials

`scf_reference(atoms, charge, spin, basis, ecp=None)` and the end-to-end path (`compute_ground_state`, and the command line through `params['ecp']`) accept a pseudopotential and assign it to `Mole.ecp` before the molecule is built. The argument is either a name, which applies to every element, or a mapping keyed by element. The mapping is the safe form: the `def2` sets carry an ECP above Kr only, so an iodine–hydrogen molecule needs `{'I': 'def2-svp'}`; a bare `'def2-svp'` also addresses hydrogen, which has no ECP under that name. Without the parameter a heavy element is built as the all-electron atom against a valence basis, and the reference cannot be constructed because $N_{\text{occ}} > N_{\text{mo}}$.

The core Hamiltonian carries the ECP scalar term on every path. The core-Hamiltonian helper in `kernel` adds `ECPscalar` when the molecule reports an ECP, and the same construction in the on-top terms of `driver.nevpt2` is corrected. Measured on ThO, dropping the term moved the trace of the one-electron matrix by 150.7 Ha and the energy by 71 Ha; a molecule without an ECP is bit-identical to the plain kinetic-plus-nuclear sum.

The two factory families exist because of a pyscfad constraint: the `Mole`-based factories cannot be traced inside `jax.jit`, and the theta-only factories are built for exactly that context. When in

doubt about which to call, ask whether the geometry is fixed; if it is, use the theta-only factory.

3.5 cli

`python -m diffmrc.cli` runs an end-to-end calculation from a geometry file; `--help` lists the options. The command-line entry is the least settled surface of the package.

Chapter 4

Protocols and options

The package reports convergence and protocol evidence; it does not enforce it. This chapter collects the settings that decide whether a number is trustworthy, with the values the benchmark records were produced under. Where a default can be changed by a caller, the change belongs next to the number it produced, because several of the gates below are reachable at more than one tolerance and the results differ at the margins.

4.1 Orbital-point convergence

The two-stage protocol separates two different failure modes: the optimizer that can no longer lower the energy, and a point that is not stationary. The stages and their defaults:

Setting	Value	Meaning
`optimizer_grad_tol`	10^{-4}	L-BFGS-B target; loose on purpose
`refine_target`	10^{-11}	the Newton target, and the verdict
`max_steps`	2000	L-BFGS-B iteration cap
`refine_max_iter`	80	damped-Newton iteration cap
`_CG_tol` / `_CG_maxiter`	10^{-6} / 40	Newton–CG budget for the step solve

The first-stage target was raised from 10^{-6} to 10^{-4} because the first stage cannot reach 10^{-6}: its measured floor is 1.5×10^{-5}, so a tighter target reports a false non-convergence and keeps L-BFGS-B iterating in a flat valley. The Newton refinement continues from wherever the first stage stops and carries the verdict. Measured on HI/def2-SVP CAS(2,2), the 10^{-6} target ran past 55 minutes, while stopping the first stage at 10^{-4} and letting the refinement finish reached 4.3×10^{-8} in 128 s. The driver parameter is `optimizer_grad_tol`; a direct call to `converge_orbital_point` that omits it still sees the module constant of 10^{-6}.

The verdict is the gradient norm at the point the chain stopped at, classified by the stop reason:

`StopReason`	Reading
`CONVERGED`	gradient below the refine target
`STALLED_GRADIENT`	a step no longer reduced the gradient
`GUARD_STOPPED`	no damped step lowered the energy
`ITERATION_CAP`	budget exhausted; re-check before claiming convergence

Two details of the Newton stage are load-bearing and easy to lose when reimplementing. The Hessian enters only through Hessian-vector products — the explicit `jax.hessian` broadcasts every intermediate over n_{par}^2 tangent pairs and was killed by a 402 GB allocation at $n_{\mathrm{par}} \approx 500$

— and the downhill acceptance test is not optional: an unguarded refinement moved an energy from −76.07 to +1.04 Ha in this project's history and printed no error.

One more case needs its own route. At an exactly symmetric stationary point the blocked eigensolver's second derivative is undefined: every entry of the analytic orbital Hessian is not finite, while the energy and the analytic gradient are smooth. The polish probes one Hessian-vector product, and a non-finite entry hands the point to a damped Newton step whose Hessian is rebuilt from symmetric differences of the analytic gradient (step 10^{-5} in rotation-parameter units, dense matrix, 300-parameter cap). Measured on N_2/6-31G CAS(4,4): the L-BFGS-B stage stops at 7.3×10^{-5}, two refinement steps reach 8.8×10^{-14}, and the robust verdict passes. On a system whose analytic Hessian is finite the probe costs one Hessian-vector product and the path is unchanged.

4.2 State averaging

The state-averaged factories take `nroots` and `state_weights`; the weights are normalized and must have one entry per root. The measured effect of between-group equal weights (the second pass) on the reported excitation energies stayed within a few tenths of an eV for the benchmark systems, while the second pass usually tightens the final orbital gradient by orders of magnitude. The benchmark records quote which pass each number came from.

Two root-table facts belong with every state-averaged number. The eigensolver returns roots, so a triplet travels as three degenerate roots; and a per-component NEVPT2 correction can be undefined (`nan`) for a pure $|M| = 1$ component, in which case the group's representative is the lowest component with a finite value. `sa_nevpt2_state_table` performs both bookkeeping and returns the evidence.

4.3 Active-space windows

The frontier-window rules are stated in the Tutorial (its chapter on choosing an active space). The part that belongs to a protocol table is the bookkeeping: the electron count follows the window,

$$n_{\mathrm{ele}} = N_{\mathrm{ele}} - 2\, n_{\mathrm{core}}, \tag{4.1}$$

and a window that widens to three orbitals for the linear two-pi and amide patterns therefore carries four active electrons, not two. A hard-coded two here produces a wrong ground state that the energy alone will not flag.

A second bookkeeping rule belongs to the automatic fallback (the occupation criterion used when no element rule covers the composition). The fallback completes degenerate partners, and the completed space can be one the orbital optimizer cannot converge. When the converged gradient of an automatically selected space with more than two orbitals exceeds 10^{-3}, the driver retries the pure frontier pair — highest occupied and lowest virtual — and keeps that point when its gradient is lower. Measured on HI/def2-SVP CAS(2,2), the completed three-orbital space stalled at 10^{-2} after hundreds of steps, while the frontier pair reached 4×10^{-8} in 128 s. On N_2/6-31G the completed frontier manifold stalls in the same way and the frontier pair is kept. The retry is confined to the automatic fallback: a manually given active space is never overridden.

The rotation set itself can dominate the cost. The pairs include the core–virtual block, and for a large core that block outnumbers the core–active and active–virtual pairs together; the first stage then spends most of its iterations on directions that barely change the energy. When the core–virtual pair count exceeds twice the sum of the other two blocks, the driver first optimizes the subset without core–virtual rotations and then refines the point over the full set: `embed_theta` maps the subset vector onto the full pair list by pair identity, and

`converge_orbital_point_two_stage` runs the second stage from the embedded point. Measured on HI/def2-SVP CAS(2,2) (262 full and 58 subset parameters), the two-stage route reaches a final gradient of 1.4×10^{-9} in 378.5 s, against 420.5 s and 7.5×10^{-10} for the single-stage full-space route, with the two converged points 1.1×10^{-13} Ha apart. The split is gated rather than unconditional because the subset optimum is not always a minimum: on H_2O/6-31G it is a saddle with two negative curvature directions, and only the full-space refinement reaches the minimum. H_2O and N_2 therefore keep the single-stage route and are bit-identical. The gate is an engineering rule supported by three systems, not a theorem.

4.4 The e2_scale switch

Both NEVPT2 factories accept `e2_scale`. With `e2_scale = 0` the perturbation correction is switched off and the call reduces to the CASSCF energy of the same chain — the identity used by the test suite to pin the joint and state-averaged factories against the validated single-root path. With the default value of 1.0 the correction is included. There is no other intermediate value with a physical meaning; the switch exists for verification.

4.5 Differentiation settings

The theta-only factories are safe inside `jax.jit`; the `Mole`-based ones are not, and the failure mode is a tracing error at compile time rather than a wrong number. The second-order paths (the refinement, the trust-region polish) differentiate the gradient with forward-mode `jvp` calls; a caller wrapping the whole convergence in `jit` should expect the compile of those nodes to dominate the first evaluation.

4.6 MC-PDFT on-top functionals (tPBE, tPBE0)

The reported energy of `--method mcpdft` is

$$E_{\text{MC-PDFT}} \;=\; E_{\text{mcwfn}} \;+\; E_{ot}, \qquad E_{\text{mcwfn}} \;=\; V_{nn} + \langle h|D\rangle + \tfrac{1}{2}\,\langle J[D]\,|\,D\rangle, \tag{4.2}$$

with D the *full* (core plus active) one-body density and E_{ot} the on-top functional of the translated spin densities, evaluated on a frozen quadrature grid. The conventions that the validation had to pin down, and that a reimplementation must reproduce, live in `core.mcpdft`:

- the translation uses gradient rows $\rho'_\sigma = (\nabla\rho/2)(1 \pm \zeta)$ with **no** $\nabla\zeta$ term (Li Manni et al. 2014; the engine's `get_rho_translated`);
- the on-top pair density is the convention-free factorized form $\Pi = \frac{1}{2}\sum \phi^4(A - B_{ss}) + (\rho_f\rho_f - \rho_a\rho_a)$, with A the M-block product identical to the stack's spin-summed 2-RDM mechanism;
- the xc kernel is the package's own closed-form PBE (`core.mcpdft.pbe_exc`), because the pyscfad/libxc wrapper cannot be traced under `jit`; it matches libxc to 2×10^{-15} in the physical domain and deviates only below $\rho \sim 10^{-13}$ (libxc's internal guards), where the grid weights put the integral contribution below 10^{-25}.

The frozen grid. The quadrature is built once and shared by every evaluation, so geometry derivatives omit the grid-velocity term. The calibration on CO/6-31G bounds that term at 7.3×10^{-5} Eh/bohr at grid level 3, falling to 2.5×10^{-5} at level 4 – three orders of magnitude below the wavefunction response it rides on.

One-step protocol and the relaxed variant. The orbitals are the CASSCF-driven ones and the on-top energy is evaluated once; this is the DVS paper's protocol. The orbital point is *not* stationary for the on-top functional (measured $|\nabla E| \approx 3 \times 10^{-3}$ there). The theta chain can also minimize the on-top energy itself, which converges to a fully relaxed stationary point below the one-step value by 3.0×10^{-4} Eh on H_2/6-31G; the two numbers answer different questions and the reports say which one they carry.

The tPBE0 hybrid. `--method mcpdft0` is the 0.25 hybrid. Its engine decomposition is $E = E_{\text{classical}} + \lambda(E_x + E_c) + (1-\lambda)E_{ot}$ with $\lambda = 0.25$, E_x the HF-like exchange of the full density and E_c the active-space cumulant contraction. The standard RDM split gives $E_x + E_c = E_{\text{CASSCF}} - E_{\text{classical}}$, so at the one-step orbitals the hybrid is the convex combination $E_{\text{tPBE0}} = 0.75\, E_{\text{tPBE}} + 0.25\, E_{\text{CASSCF}}$; the chain implements exactly this, with no 2-RDM convention in the path.

Verification anchors. Against `pyscf.mcpdft`: H_2/STO-3G reproduces $e_{tot}/e_{ot}/e_{mcwfn}$ to 10^{-16} (golden -1.156628479816; the tPBE0 value -1.151792318484 to 9×10^{-16}), H_2O/STO-3G to 1.6×10^{-9}, and H_2/6-31G to 1.3×10^{-8} – the engine's own FCI convergence floor, measured as its Davidson vector differing from a full block decomposition by $\sim 10^{-7}$. The rotation-chart AD gradient agrees with finite differences to 10^{-10}, and the fully relaxed optimization reaches $|\nabla| = 1.4 \times 10^{-14}$.

The (lambda, N) DVS scan. `driver.dvs.dvs_scan` builds the candidate orbital sets of the DVS construction ($G(\lambda) = F - \lambda K$ diagonalization, the exact t369 reproduction), picks windows with the APC chooser (pyscf's `mcscf.apc`, the authors' implementation), and carries each candidate through the one-step columns; the selection is the lowest mean on-top energy with the runner-up gap reported. Windows whose determinant count exceeds `max_ndet` (default 2000) are refused with the count recorded: the in-stack CI is a full determinant decomposition, so the paper's ZnH/CuCl $(12,12)/(14,14)$ windows stay engine-side by design.

Part II

Methods and records

Chapter 5

The differentiable chain

5.1 Purpose and scope

This chapter describes how the package makes a multireference energy differentiable and records the design facts that shaped the implementation. The chain covers CASCI, CASSCF, state-averaged CASSCF, and the NEVPT2 correction. It applies to molecular systems with a fixed active-space partition and a full-electron or effective-core Hamiltonian whose one- and two-electron integrals are differentiable in the nuclear coordinates.

The scope excludes three cases. A non-variational correction whose orbitals are optimized on a different surface needs the response term of Section 5.2.4; the frozen derivative alone is not the physical derivative. A system whose orbital Hessian has a near-null direction needs the pseudo-inverse construction of Chapter 6; a plain solve amplifies the null component. A wavefunction whose CI space is enlarged by a non-differentiable selection rule is outside the contract: every quantity differentiated here is a smooth function of a fixed parameter vector.

5.2 Theory and derivation

5.2.1 Exponential parameterization

Let C_0 be a reference coefficient matrix of shape $n_{\mathrm{ao}} \times n_{\mathrm{mo}}$, and let $\theta \in \mathbb{R}^{n_{\mathrm{par}}}$ be a vector of rotation parameters. The parameterized orbitals are

$$C(\theta) = C_0\, R(\theta), \qquad R(\theta) = \prod_{k=1}^{n_{\mathrm{par}}} \exp(\theta_k A_k)\,, \tag{5.1}$$

where the product is ordered and

$$A_k = e_{i_k} e_{j_k}^T - e_{j_k} e_{i_k}^T, \qquad A_k^T = -A_k, \qquad A_k^2 = -\left(e_{i_k} e_{i_k}^T + e_{j_k} e_{j_k}^T\right) \tag{5.2}$$

is the elementary antisymmetric generator of the pair (i_k, j_k). Each factor is a plane rotation,

$$\exp(\theta_k A_k) = \cos\theta_k \left(e_{i_k} e_{i_k}^T + e_{j_k} e_{j_k}^T\right) + \sin\theta_k \left(e_{j_k} e_{i_k}^T - e_{i_k} e_{j_k}^T\right) + \sum_{m \notin \{i_k, j_k\}} e_m e_m^T, \tag{5.3}$$

so $R(\theta)$ is orthogonal, $R^T R = I$, and the transformed coefficients remain orthonormal when $C_0^T S C_0 = I$ in the overlap metric.

The factors do not commute for different pairs, so the ordered product in Eq. (5.1) is not the single exponential $\exp(\sum_k \theta_k A_k)$ in general. The implementation evaluates the ordered product directly as a sequence of Givens updates (`rotation.block_rotation`); the distinction matters when a closed-form derivative is written down, because the derivative of the product is not the product of the derivatives in the same order.

The allowed pair set is

$$(i,a), \quad (i,v), \quad (a,v), \qquad i \in \text{core},\ a \in \text{active},\ v \in \text{virtual}. \tag{5.4}$$

Intra-block rotations (core–core, active–active, virtual–virtual) leave the CASSCF energy invariant: the core block is fully occupied, the virtual block is empty, and the active block is complete inside the CI step. They are therefore omitted. The core–virtual family is not redundant. On N_2/6-31G CAS(2,2) the measured energies are -108.903578776806 Eh with 32 parameters (core–active and active–virtual only), -108.903650545967 Eh with 92 parameters (adding core–virtual), and the same value with 107 parameters (adding intra-core). The 0.045 kcal/mol defect of the pruned set is systematic and does not show up as a convergence failure.

5.2.2 Energy derivative

Let the energy depend on the orbitals through a differentiable functional $E(C)$, and let $G = \partial E/\partial C$ denote the $n_{\text{ao}} \times n_{\text{mo}}$ matrix of partial derivatives. The chain rule gives

$$\frac{\partial E}{\partial \theta_k} = \operatorname{tr}\left[G^T C_0 \frac{\partial R}{\partial \theta_k}\right], \qquad \frac{\partial R}{\partial \theta_k} = \left(\prod_{m<k} \exp(\theta_m A_m)\right) A_k \exp(\theta_k A_k) \left(\prod_{m>k} \exp(\theta_m A_m)\right). \tag{5.5}$$

The implementation never forms Eq. (5.5) by hand; JAX differentiates the ordered Givens product and the energy composition. The equivalence with the MO-basis form is useful for diagnostics. For a Givens rotation of the pair (i,j) the orbital gradient is

$$\frac{\partial E}{\partial \theta_{ij}} = 2\,(F_{ji} - F_{ij})\,, \tag{5.6}$$

where F is the generalized Fock matrix in the MO basis,

$$F_{pq} = \sum_r \gamma_{pr} h_{qr} + \sum_{rst} \Gamma_{prst} (qr|st), \tag{5.7}$$

with γ and Γ the state 1- and 2-RDMs. The factor 2 and the antisymmetric index order follow from $A_k^T = -A_k$; a symmetric F gives zero, which is the statement that a rotation that does not change the state has no gradient.

5.2.3 The envelope theorem

Write the relaxed energy as a function of the geometry alone,

$$E^*(R) = E\big(\theta^*(R), R\big), \qquad \theta^*(R) = \arg\min_\theta E(\theta, R), \tag{5.8}$$

and assume that $\theta^*(R)$ is an interior, isolated stationary point and that E is twice continuously differentiable in a neighborhood. Differentiating Eq. (5.8) gives

$$\frac{dE^*}{dR} = \left.\frac{\partial E}{\partial R}\right|_\theta + \left(\frac{\partial E}{\partial \theta}\right) \frac{d\theta^*}{dR}. \tag{5.9}$$

The stationarity condition

$$\frac{\partial E}{\partial \theta}\big(\theta^*(R), R\big) = 0 \tag{5.10}$$

annihilates the second term, so

$$\frac{dE^*}{dR} = \left.\frac{\partial E}{\partial R}\right|_{\theta^*}. \tag{5.11}$$

Equation (5.11) is the envelope theorem used by the production gradient: the orbital parameters are frozen at their stationary values and the reference orbitals are recomputed at the displaced

geometry. Both ingredients are required. Holding C_0 fixed at the reference geometry is a different physical orbital set and produces a spurious gradient of order 4×10^{-1} Eh/bohr; taking C_0 from a converged CASSCF calculation at the reference geometry is also not the displaced reference and produces an error of order 4×10^{-3} Eh/bohr.

Three conditions are implicit in Eq. (5.11):

- the stationary point is interior, so no active constraint enters the derivative;
- the branch is fixed, so $\theta^*(R)$ is a differentiable function rather than a set-valued map at a bifurcation;
- the reference orbitals are a differentiable function of the geometry, including any gauge choice used to align degenerate blocks.

5.2.4 Non-variational corrections and the Z-vector equation

NEVPT2 is not variational in the orbitals. Let

$$E_{\mathrm{tot}}(\theta, R) = E_{\mathrm{CASSCF}}(\theta, R) + e_2(\theta, R), \tag{5.12}$$

and let θ^* minimize E_{CASSCF}, not E_{tot}. Differentiating along the physical path,

$$\begin{aligned} \frac{dE_{\mathrm{tot}}}{dR} &= \left.\frac{\partial E_{\mathrm{CASSCF}}}{\partial R}\right|_{\theta} + \left.\frac{\partial e_2}{\partial R}\right|_{\theta} + \left(\frac{\partial e_2}{\partial \theta}\right)\frac{d\theta^*}{dR}, \\ &= g_{\mathrm{frozen}} + s^T \frac{d\theta^*}{dR}, \end{aligned} \tag{5.13}$$

where the second line defines the frozen gradient and the correlation source. because the CASSCF stationarity condition removes $(\partial E_{\mathrm{CASSCF}}/\partial\theta)$ but not $(\partial e_2/\partial\theta)$. The orbital response follows from differentiating the stationarity condition:

$$H_{\theta\theta}\frac{d\theta^*}{dR} + M = 0, \qquad H_{\theta\theta} = \frac{\partial^2 E_{\mathrm{CASSCF}}}{\partial\theta^2}, \qquad M = \frac{\partial^2 E_{\mathrm{CASSCF}}}{\partial\theta\,\partial R}. \tag{5.14}$$

Hence

$$\frac{d\theta^*}{dR} = -H_{\theta\theta}^{-1} M, \tag{5.15}$$

and the relaxed gradient is

$$g_{\mathrm{relaxed}} = g_{\mathrm{frozen}} - M^T H_{\theta\theta}^{-1} s, \qquad s = \frac{\partial e_2}{\partial\theta}, \tag{5.16}$$

with the frozen gradient $g_{\mathrm{frozen}} = \partial_R(E_{\mathrm{CASSCF}} + e_2)|_{\theta^*}$. The vector

$$z = H_{\theta\theta}^{-1} s \tag{5.17}$$

is the orbital Z-vector. One linear solve serves every Cartesian direction, which is the adjoint form of the response; the direct form would solve one linear system per displacement.

The CI coordinates need no separate multiplier in this reduced space. Both energy functions solve the CI eigenvalue problem internally, so their θ and R derivatives are taken along the CI-relaxed manifold. The CI block of the full Hessian is replaced by its Schur complement, which is the $H_{\theta\theta}$ above. Working in the full (θ, c) space would add a CI multiplier, but that multiplier is annihilated by the variational CI condition for E_{CASSCF} and only reshuffles the same reduced Hessian.

The Hessian in Eq. (5.16) is singular along directions that leave the CASSCF energy invariant. The implementation uses a truncated pseudo-inverse with a relative cutoff, not a direct solve; the construction and the null-space argument are in Chapter 6. On H_2O/6-31G the CASSCF Hessian carries one soft mode of eigenvalue 3.8×10^{-6} and condition number 2.3×10^7 whose response is not part of the correlation source; the relaxed-gradient record uses `rcond`$= 10^{-7}$ to cut it.

5.3 Input and operation

The chain has two factory families. The *joint* factories (`driver.nevpt2.make_nevpt2_joint_fn` and its state-averaged sibling) take the molecule and θ and rebuild the reference SCF inside the call; they are the geometry-differentiable path. The *theta-only* factories (`make_nevpt2_theta_fn`, `make_nevpt2_sa_theta_fn`) compute the SCF and the integrals once at construction and return a callable of θ alone. The theta-only form exists because this pyscfad build cannot trace its SCF inside `jax.jit`, and the orbital optimizer wraps its objective in `jit`.

A minimal CASSCF gradient skeleton is

```
import jax
jax.config.update('jax_enable_x64', True)
from jax import numpy as jnp
from pyscfad import gto as pgto, scf as pscf

from diffmrc.core.rotation import block_rotation
from diffmrc.core.orblib import orbital_pairs
from diffmrc.core.optimizer import optimize_theta
from diffmrc.core.grad import (make_energy_fn, make_grad_fn,
                               solve_theta_star, nuclear_gradient)

mol = pgto.Mole()
mol.atom = 'O 0 0 0; H 0.7557 0 0.5880; H -0.7557 0 0.5880'
mol.basis = '6-31g'
mol.build()
nmo, ncore, nact, nele = mol.nao_nr(), 4, 2, 2
pairs = orbital_pairs(ncore, nact, nmo, include_cv=True)
npar = len(pairs)

mf = pscf.RHF(mol)
mf.kernel()
C_rhf = jnp.asarray(mf.mo_coeff)
h1e = mol.intor('int1e_kin') + mol.intor('int1e_nuc')
eri = mol.intor('int2e', aosym='s1')
E_cas = make_energy_fn(ncore, nact, nele, use_blocked=True)

E_theta = lambda t: E_cas(C_rhf @ block_rotation(t, nmo, pairs),
                          h1e, eri, mol.energy_nuc())
theta, gnorm = solve_theta_star(mol, C_rhf, E_theta, npar,
                                optimize_theta, steps=2000,
                                g_conv=1e-12)
g = nuclear_gradient(mol, make_grad_fn(nmo, pairs, theta, E_cas, pscf))
```

The parameter list `pairs` is the contract: it must include core–active, core–virtual, and active–virtual pairs. `ncore`, `nact`, and `nele` are spatial-orbital and electron counts. `theta` is in radians and is defined relative to the reference coefficient matrix; `gnorm` is the Euclidean norm of $\partial E/\partial\theta$ at the returned point. A value above 10^{-6} means the point is not usable as a stationary reference. The gradient is returned in Eh/bohr with shape `(natm, 3)`.

For a non-variational correction the caller builds the two energy functions and calls the Z-vector wrapper, as in Chapter 14:

```
from diffmrc.driver.nevpt2_zvector import (
    make_nevpt2_joint_fn_si_exact, nevpt2_relaxed_gradient_zvector)

def scf_builder(m):
    mf = pscf.RHF(m)
    mf.conv_tol = 1e-13
    mf.conv_tol_grad = 1e-10
```

```
    return mf

E0 = make_nevpt2_joint_fn_si_exact(ncore, nact, nele, nmo, pairs,
                                   scf_builder, e2_scale=0.0,
                                   align_ref=C_rhf,
                                   align_bounds=[(0, ncore),
                                                 (ncore, ncore + nact),
                                                 (ncore + nact, nmo)])
E2 = make_nevpt2_joint_fn_si_exact(ncore, nact, nele, nmo, pairs,
                                   scf_builder, e2_scale=1.0,
                                   align_ref=C_rhf,
                                   align_bounds=[(0, ncore),
                                                 (ncore, ncore + nact),
                                                 (ncore + nact, nmo)])
g_relaxed, info = nevpt2_relaxed_gradient_zvector(
    mol, theta, E0, E2, rcond=1e-7)
```

`theta` must be the stationary point of the same aligned CASSCF surface that `E0` evaluates; a parameter vector optimized against a different reference gauge is not the response point, even if the energy is close.

5.4 Worked example

5.4.1 H_2/STO-3G: the zero-correlation anchor

The first example isolates the variational part of the chain. On H_2/STO-3G at $R = 0.74$ Å the CAS(2,2) energy is -1.137283834489 Eh (`tests/core/test_iterative_ci.py`). The joint NEVPT2 factory at `e2_scale=0.0` returns the same CASSCF energy: the test bound is 10^{-5} Eh (`tests/driver/test_nevpt2.py`). With the correlation scaled off, the joint gradient and the independent CASSCF gradient agree to 10^{-10} Eh/bohr, and the gradient is translationally invariant to 10^{-8} Eh/bohr on this system. The system has two orbitals only, so the correlation term is identically zero and the example tests the variational chain, not the perturbation layer.

The numerical output to record is therefore a set of agreement bounds, not a gradient component: the energy anchor -1.137283834489 Eh, the factory difference below 10^{-5} Eh, and the gradient difference below 10^{-10} Eh/bohr. A finite-difference pair on this system is not recorded in the release tree; the finite-difference comparison is the next example.

5.4.2 H_2O/6-31G: AD against the finite-difference golden standard

The production regression compares the AD gradient with a central difference of an independent analytic CASSCF energy at $h = 10^{-3}$ bohr (`experiments/gradient_regression.py`). On H_2O/6-31G CAS(2,2) the recorded largest deviation over converged points is 3.5×10^{-8} Eh/bohr (the baseline in that file is 3.5×10^{-8}; the analytic-reference run in `experiments/gradient_gold_validation.py` reports $3.496256317792046 \times 10^{-8}$). The archived analytic reference gradient is, in Eh/bohr,

```
[ 1.06581410e-10,  7.78044296e-09, -2.32218066e-02,
 -4.76657289e-03,  2.55795385e-10,  1.16109088e-02,
  4.76657296e-03, -1.42108547e-10,  1.16109094e-02]
```

The AD gradient reproduces this vector within the stated deviation; the two near-zero x components are constrained by the molecular plane and are the first place a broken invariance shows up.

5.4.3 N_2/6-31G: the parameter set is part of the answer

The third example is the parameter-set record of `core.orblib`. On N_2/6-31G CAS(2,2), against an independent CASSCF implementation:

Parameter set	Count	E_{CAS} (Eh)	Deviation (kcal/mol)
core–active and active–virtual	32	−108.903578776806	0.045036
adding core–virtual	92	−108.903650545967	8×10^{-10}
adding intra-core	107	−108.903650545967	8×10^{-10}

The last row confirms that intra-core rotations are redundant. The middle row shows that the core–virtual family is not: dropping it leaves the orbital optimization at a non-stationary point whose energy is too high by 0.045 kcal/mol.

5.5 Pitfalls

- *Pruning the pair list.* Core–virtual rotations are not redundant. The failure is a smooth, systematic energy defect, not a convergence failure; it is invisible without an independent reference.
- *Using the raw CASSCF orbitals as the reference.* The reference C_0 must be the geometry-dependent SCF solution, not the converged CASSCF coefficients from the reference geometry. The measured error is 4×10^{-3} Eh/bohr for the mildest case and 4×10^{-1} Eh/bohr when C_0 is held fixed.
- *Calling a joint factory inside `jit`.* This pyscfad build cannot trace the SCF inside a jit context. Use the theta-only factory for the optimizer and the joint factory for the implicit Hessian.
- *Forgetting the gauge alignment.* The SCF solution is defined up to orthogonal mixing inside degenerate blocks. An unaligned reference makes the same θ mean different orbitals at neighbouring geometries; the measured fixed-theta energy slope on H_2O/6-31G changes from −5.8 to −0.0528 Eh/bohr under a phase flip. Pass the base-geometry coefficients as `align_ref` and the block bounds as `align_bounds`.
- *Applying the envelope theorem to a non-variational correction.* The frozen derivative of NEVPT2 is not the relaxed derivative. The response term of Eq. (5.16) is required; on H_2O it is not small.
- *Finite differences near a zero derivative.* A central difference of a near-zero mixed derivative is unreliable: this project measured errors of over one hundredfold, growing as the step shrank. Use an invariance or an analytic cross-check instead.
- *Unblocked eigensolvers on degenerate spectra.* The plain eigensolver's second derivative is NaN at a cross-S_z degeneracy; the blocked solver is required for second derivatives, and linear molecules with bit-exact degenerate pairs need the Lanczos kernel.

5.6 Validation and references

The records behind this chapter are `tests/core/test_stationary.py`, `tests/core/test_hessian.py`, `tests/driver/test_nevpt2.py`, `tests/driver/test_nevpt2_zvector.py`, `experiments/gradient_regression.py`, `experiments/gradient_gold_validation.py`, and `experiments/orbital_response_implicit.py`. The measured facts quoted above are: the H_2/STO-3G energy anchor and gradient-equivalence bounds; the H_2O/6-31G AD-versus-FD

deviation 3.5×10^{-8} Eh/bohr; the N_2/6-31G parameter-set table; and the H_2O/6-31G soft-mode eigenvalue and condition number used for the Z-vector cutoff.

The derivation is the standard analytic-derivative construction for MCSCF and internally contracted perturbation theory. The relevant references are P. Pulay, *Mol. Phys.* **17**, 197 (1969), for the orbital-response (force) theorem; T. Helgaker, P. Jørgensen and J. Olsen, *Molecular Electronic-Structure Theory* (Wiley, 2000), Chapters 10–12, for the exponential parameterization, the generalized Fock matrix, and the coupled-perturbed equations; N. C. Handy and H. F. Schaefer, *J. Chem. Phys.* **81**, 5031 (1984), for the Z-vector method; C. Angeli, R. Cimiraglia, S. Evangelisti, T. Leininger and J.-P. Malrieu, *J. Chem. Phys.* **114**, 10252 (2001) and **117**, 9138 (2002), for NEVPT2; and the Park (2019) NEVPT2-gradient derivation cited in `tests/driver/test_nevpt2.py`.

Chapter 6

Second derivatives

6.1 Purpose and scope

The package differentiates its energy chain twice, so that harmonic frequencies, curvature checks, and trust-region steps come out of the same code path that produced the gradients. The target quantity is the nuclear Hessian of the relaxed energy $E^*(R)$ of Chapter 5, not the Hessian at frozen orbitals. The frozen object is available as an intermediate, but it is not the physical second derivative: the orbital response contributes at the same order and, on the systems measured here, moves every frequency in the same direction.

The chapter applies to any energy whose orbitals are determined by a stationarity condition and whose Hessian is available through automatic differentiation. It does not cover analytic second derivatives of the single-reference SCF surface, and it does not cover a matrix-free solver that removes the iteration cap of the least-squares response; both are listed as open problems in the README.

6.2 Theory and derivation

6.2.1 Implicit differentiation of the stationarity condition

Let $g(\theta, R) = \partial E/\partial\theta$ and let $\theta^*(R)$ satisfy

$$g\big(\theta^*(R), R\big) = 0. \tag{6.1}$$

Differentiating Eq. (6.1) with respect to R,

$$H_{\theta\theta}\frac{d\theta^*}{dR} + \frac{\partial g}{\partial R} = 0, \qquad H_{\theta\theta} = \frac{\partial^2 E}{\partial\theta^2}, \qquad \frac{\partial g}{\partial R} = \frac{\partial^2 E}{\partial\theta\,\partial R} \equiv M, \tag{6.2}$$

so

$$\frac{d\theta^*}{dR} = -H_{\theta\theta}^{-1} M. \tag{6.3}$$

The relaxed energy is $E^*(R) = E(\theta^*(R), R)$. Its first derivative is the envelope result

$$\frac{dE^*}{dR} = \left.\frac{\partial E}{\partial R}\right|_{\theta}, \tag{6.4}$$

and its second derivative follows from differentiating Eq. (6.4):

$$\frac{d^2E^*}{dR^2} = \left.\frac{\partial^2 E}{\partial R^2}\right|_{\theta} + M^T\frac{d\theta^*}{dR} = H_{RR} - M^T H_{\theta\theta}^{-1} M. \tag{6.5}$$

Here $H_{RR} = \partial^2 E/\partial R^2|_\theta$ is the frozen Hessian, and

$$H_{\text{response}} = -M^T H_{\theta\theta}^{-1} M \tag{6.6}$$

is the orbital-response contribution. Equation (6.5) is the construction implemented by `core.hessian.implicit_hessian`; the result object exposes H_{RR}, H_{response}, and the total separately.

6.2.2 The pseudo-inverse and null-space exclusion

The orbital Hessian is not positive definite and is singular along directions that leave the energy invariant. Rotations inside a block are the physical example: the core block is fully occupied, the virtual block is empty, and the active block is complete inside the CI step, so a rotation that only mixes orbitals inside one block changes no occupied–virtual or occupied–active matrix element. The optimized θ is also free to move along any direction that the optimizer never constrained.

Write the symmetric eigendecomposition

$$H_{\theta\theta} = \sum_i \lambda_i \, v_i v_i^T, \qquad v_i^T v_j = \delta_{ij}. \tag{6.7}$$

The truncated pseudo-inverse with relative cutoff τ is

$$H_{\theta\theta}^{+} = \sum_{|\lambda_i| > \tau \lambda_{\max}} \lambda_i^{-1} \, v_i v_i^T, \tag{6.8}$$

and the response becomes

$$\frac{d\theta^*}{dR} = - \sum_{|\lambda_i| > \tau \lambda_{\max}} \lambda_i^{-1} \, v_i \left(v_i^T M \right). \tag{6.9}$$

A dropped direction contributes to $d\theta^*/dR$ only through $v_i^T M$. If M has no component along the null space, dropping the direction changes nothing. If it does, the response along that direction is not determined by the second-order expansion, and the pseudo-inverse silently omits it. The result object exposes $H_{\theta\theta}$ and M, so the projection $v_i^T M$ can be formed for any discarded direction; the curvature report records the spectrum and the near-null count, not the projection itself.

A relative cutoff is not interchangeable with an absolute one. An earlier script used an absolute singular-value threshold of 10^{-3}; on a matrix with condition number 10^{12} the absolute and relative rules retained five and two singular values and produced solutions differing by 99.9%. The default in `core.hessian` is relative, `rcond`$= 10^{-9}$.

6.2.3 The unrolled route and its consistency

The alternative construction differentiates the optimizer itself. Write one Newton-type step as

$$\theta_{k+1}(R) = \theta_k(R) - H_k^{-1} g\big(\theta_k(R), R\big), \tag{6.10}$$

and evaluate N steps inside the computational graph. At a fixed point $\theta_N = \theta^*$ the derivative of the map satisfies

$$\frac{d\theta_{k+1}}{dR} = \big(I - H_k^{-1} H_k\big) \, \frac{d\theta_k}{dR} - H_k^{-1} M_k = -H_k^{-1} M_k. \tag{6.11}$$

With an exact Hessian the fixed-point derivative is reached in one step; with the quasi-Newton or damped Hessian used in practice the iteration converges linearly with the contraction factor of the optimizer. The unrolled and implicit routes therefore agree in the limit of a converged iteration and an exact response solve. The measured agreement on the validation systems is 1.6×10^{-11} (Chapter 7).

The unrolled route has two costs that the implicit route avoids. It inherits every numerical accident of the optimizer, and its memory and time scale with the number of unrolled steps. It is retained as the independent check, not as the production route.

6.2.4 Hessian-vector products

The refinement and trust-region stages need the Hessian only through its action on a vector. The product is the directional derivative of the gradient,

$$Hv = \frac{d}{d\epsilon} g(\theta + \epsilon v)\Big|_{\epsilon=0} = \mathrm{jvp}_v(g)(\theta), \tag{6.12}$$

and one forward-mode tangent per reverse pass is enough. Forming the full Hessian with `jax.hessian` is forward-over-forward: every intermediate is broadcast over the square of the parameter count. At about five hundred parameters that construction requested a single 402 GB allocation and died; the Hessian-vector form is what keeps the memory at one tangent vector.

6.3 Input and operation

The production entry point is

```
from diffmrc.core.hessian import (implicit_hessian,
                                  make_joint_energy_fn,
                                  make_theta_energy_fn,
                                  implicit_hessian_columnwise,
                                  implicit_hessian_hvp)
```

`implicit_hessian(mol, theta, E_joint, rcond=1e-9)` returns an `ImplicitHessianResult` with the fields `H_tt`, `H_tt_inverse`, `curvature`, `mixed`, `H_RR`, `H_response`, `H_total`, `dtheta_dR_max`, and `rcond`. The total is in Eh/bohr^2 with shape `(3 natm, 3 natm)`. `theta` is in radians; `rcond` is a relative cutoff on the singular values of `H_tt`.

`E_joint` is built by `make_joint_energy_fn` and has the signature `(mol, theta) -> energy`. The SCF is rebuilt inside every call, which is what makes the geometry derivative correct and what makes each evaluation expensive. `E_tt`, when available from `make_theta_energy_fn`, removes the SCF from the theta-only part of the graph; the mixed and frozen blocks still use the joint function because they differentiate the reference orbitals with respect to geometry.

For large parameter counts, `implicit_hessian_columnwise` takes one tangent per column, and `implicit_hessian_hvp` assembles the total without forming `H_tt`, `M`, or `H_RR`. The matrix-free route solves `H_tt x = M v` by LSMR and records `lsmr_iterations`, `lsmr_istop`, and `lsmr_residual` per column. A converged solve is part of the result; the iteration cap is $8n_{\mathrm{par}}$.

Harmonic frequencies are computed from the total Hessian with the `wavenumbers` routine in `analysis.vibrations`, which performs the mass-weighting and the normal-mode analysis. The Hessian must be handed over in the (atom, coordinate) block layout the thermodynamics module expects. A plain reshape compiles and returns a plausible number; for CO/6-31G it produced 1501.8 cm^{-1} against the correct 2293.5 cm^{-1}.

6.4 Worked example

6.4.1 H_2/STO-3G: orbital-space HVP against a finite-difference ladder

The HVP test in `tests/driver/test_nevpt2.py` uses the NEVPT2 joint energy on H_2/STO-3G, a symmetric bond-stretch probe vector, and a ladder of central differences of the AD gradient. The recorded errors of the HVP against the ladder are

Step δ (bohr)	$\max \lvert \mathrm{HVP} - \mathrm{FD} \rvert$
1×10^{-3}	6.6×10^{-6}
3×10^{-4}	6.0×10^{-7}

The ratio is approximately $(10/3)^2$, which is the expected quadratic truncation term of the central difference. The test requires monotone improvement and the bounds 2×10^{-5} at $\delta = 10^{-3}$ and 2×10^{-6} at $\delta = 3 \times 10^{-4}$. This is an HVP in the orbital-parameter space, not in the nuclear-coordinate space; the probe vector is the bond stretch because a generic displacement of a diatomic would compare zero against zero.

6.4.2 H_2O/6-31G: relaxed Hessian against the golden finite-difference standard

The recorded H_2O/6-31G CAS(2,2) Hessian diagonal for the three oxygen displacements is

Route	H_{xx}	H_{yy}	H_{zz}
golden standard (finite differences)	0.7192711493	0.0217924736	0.4702696863
unrolled iteration, $N = 3$	0.7192706205	0.0217921326	0.4702703066
frozen orbitals (no response)	0.7261828915	0.0217921326	0.4711140468

The values are in Eh/bohr^2 and come from the `REF['H2O']` record in `experiments/orbital_response_implicit.py`; the golden standard is the Newton-refined reference recorded in the same file and in `experiments/orbital_response_generalize.py`. The unrolled route differs from the golden standard by 5.3×10^{-7}, 3.4×10^{-7}, and 6.2×10^{-7} Eh/bohr^2 (relative 7.4×10^{-7}, 1.6×10^{-5}, and 1.3×10^{-6}). The frozen route misses the first and third components by 6.9×10^{-3} and 8.4×10^{-4} Eh/bohr^2: the response term is not a small correction. The implicit route reproduces the unrolled construction to 1.6×10^{-11}.

6.4.3 CO/6-31G: the frequency anchor

The frequency conversion is anchored against an independent value: CO at 6-31G gives 2294.2 and 2293.5 cm^{-1} from the two assembly routes, against an external anchor at 2293.5 cm^{-1}. The near-agreement of the two routes is the internal check; the external anchor is the physical check. A validation target near zero should not be checked by finite differences: this project measured finite-difference errors of over one hundredfold on a near-zero mixed derivative, growing as the step shrank.

6.5 Pitfalls

- *A plain solve on a singular Hessian.* The orbital Hessian reaches a condition number of 2.6×10^{15} on N_2. A direct solve inverts the near-null direction and amplifies whatever the right-hand side has there. Use the pseudo-inverse, and report the cutoff.
- *An absolute instead of a relative cutoff.* The two rules retain different subspaces on an ill-conditioned matrix; the measured difference was 99.9% in the response vector. `rcond` is relative.
- *Forgetting `stop_gradient` on θ in the frozen Hessian.* The frozen term then silently acquires the orbital response, and the two routes stop being distinguishable.
- *Unsymmetrized AD Hessians.* AD returns a matrix symmetric only to round-off; `eigvalsh` and `pinv` assume exact symmetry. Symmetrize as $0.5(H + H^T)$.
- *Finite differences near a zero derivative.* The measured failure is over one hundredfold, with the error growing as the step shrinks. Use an invariance or an analytic cross-check.
- *The frequency layout trap.* A plain reshape of the Hessian gives a plausible but wrong spectrum (1501.8 against 2293.5 cm^{-1} for CO/6-31G). Use the (atom, coordinate) block layout.

- *Unblocked eigensolvers and degenerate pairs.* A plain eigensolver's second derivative is NaN at a cross-S_z degeneracy; linear molecules need the blocked solver and, for bit-exact degenerate pairs, the Lanczos kernel.
- *Unrolled iteration cost.* The unrolled route scales with the number of steps and inherits optimizer noise; it is the independent check, not the production path.

6.6 Validation and references

The records behind this chapter are `tests/core/test_hessian.py` (implicit assembly on analytic surrogates, pseudo-inverse behaviour), `tests/core/test_stationary.py` (Newton–CG and saddle checks), `tests/driver/test_orbitals.py` (Hessian-vector products against the explicit Hessian), and `tests/driver/test_nevpt2.py` (the H_2/STO-3G HVP ladder). The experimental records are `experiments/orbital_response_implicit.py`, `experiments/orbital_response_unrolled.py`, `experiments/orbital_response_generalize.py`, `experiments/hessian_to_frequencies.py`, `experiments/verify_nevpt2_an.py`, and `experiments/verify_rcond_choice.py`. Chapter 7 collects the measured agreement values: 1.6×10^{-11} between the implicit and unrolled routes, 10^{-10} for the HVP against the explicit Hessian on a non-quadratic fixture, and the CO frequency anchor.

The derivation is the implicit-function construction of analytic second derivatives for MCSCF. The relevant references are P. Pulay, *Mol. Phys.* **17**, 197 (1969); N. C. Handy and H. F. Schaefer, *J. Chem. Phys.* **81**, 5031 (1984); T. Helgaker, P. Jørgensen and J. Olsen, *Molecular Electronic-Structure Theory* (Wiley, 2000), Chapters 10–12; and J. A. Pople, R. Krishnan, H. B. Schlegel and J. S. Binkley, *Int. J. Quantum Chem. Symp.* **13**, 225 (1979), for the coupled-perturbed second-derivative construction.

Chapter 7

Verification records

Every claim this package makes about itself rests on a recorded comparison. This chapter lists the records by the question they answer, with the measured deviation and the test or script that holds it. Where a check is an equality that must hold by construction, the deviation is quoted at machine precision; where it is a comparison against an independent implementation or an external reference, the deviation is quoted with its protocol.

7.1 Energy chain

Question	Measured	Where
CASCI/CASSCF against PySCF, same geometry, basis and window	10^{-12} to 10^{-10} Ha (LiH, HF, NH_3)	`t305`; driver kernels
NEVPT2 correction against an independent implementation	4×10^{-7} on the large-core test after the block-loop fix	the frozen-contraction tests
State-averaged solution at one root against the ground-state path	bit-for-bit (10^{-12})	`tests/driver/test_sa_pipeline.py`
Joint and theta-only factories at fixed geometry	10^{-10} Ha	the same suite

7.2 First derivatives

The nuclear gradient is validated by invariance rather than by agreement with a single reference, and then by a regression against an independent derivative-free route:

- the CASSCF gradient is invariant under coordinate transformations at the 10^{-14} level, and reproduces the finite-difference reference to its own noise on smooth systems;
- the regression against the analytic reference reproduces the reference to the level at which both are converged; departures appear at points where the reference geometry was not stationary, which is a property of the comparison, not of the gradient;
- a near-zero mixed derivative is a known trap for finite differences: measured errors of over one hundredfold on a derivative whose value sits near zero, growing as the difference step shrinks. Such checks use invariances instead.

7.3 Second derivatives

Question	Measured	Where
Implicit route against the unrolled construction	1.6×10^{-11}	`orbital_response_implicit`; the implicit-Hessian tests
Harmonic frequency against an external anchor (CO, 6-31G)	2294.2 and 2293.5 cm^{-1} from the two routes, against 2293.5 cm^{-1}	frequency tests
Hessian-vector product against the explicit Hessian	10^{-10} on a non-quadratic fixture	`tests/driver/test_orbitals.py`
Damped-Newton step solver (CG) against a dense solve	10^{-8} on SPD fixtures	`tests/core/test_stationary.py`

7.4 External-data cross-checks

Two independent data lines were used to check layers that have no internal ground truth:

- **A published selection-protocol data set** (DVS-APC, Zenodo record, CC-BY-4.0): the energy layer reproduces three reference state energies to 10^{-11} Ha; a port of the published entropy-based selection formula, evaluated on all 116 orbitals of every record, agrees with the published values to exactly zero difference; the singly-occupied rule of the protocol is confirmed on the full set rather than on a sample.
- **An external benchmark database** (QUEST): the state-averaged excitation pipeline is checked over fourteen states of seven molecules; the records and the family split are in Chapter 8 and the Tutorial.

7.5 The layer contracts

The six-layer architecture is itself under test: `tests/test_layer_contracts.py` (or the `lint-imports` entry point) parses the import graph and fails on any upward import. The check is structural, not conventional: an import performed lazily inside a function body is still reported, because the contract is parsed from the syntax tree.

Chapter 8

Benchmarks

Two benchmark lines exercise the package end to end. The first is a ground-state line against direct PySCF runs at matched settings; the second is the excited-state line against the QUEST reference set, whose protocol and reading are walked through in the Tutorial (Chapter on the benchmark walkthrough). This chapter records both with their conventions, so that a quoted number can be traced back to its run.

8.1 Ground-state line (M5-A)

Three closed-shell molecules at matched geometry, basis, and active window, chain against PySCF:

System	Deviation (Ha)	Note
LiH	-3.5×10^{-12}	active window [4,5], 6-31G
HF	-3.6×10^{-13}	active window [1,2], 6-31G
NH_3	-1.9×10^{-10}	active window [1,2], 6-31G

The agreement is at the level where the two codes' convergence tolerances meet. The chain's end-to-end entry reports the energy and the stationarity diagnostics; nuclear gradients on this line come from the lower-level interface and are covered by the first-derivative records of Chapter 7.

8.2 Excited-state line (QUEST subset)

Fourteen vertical excitation energies over seven molecules, 6-31G, the frontier-window protocol with its corrections, against the QUEST database's best estimates and its own SC-NEVPT2 column. The full table sits in the Tutorial; the summary by family:

Family	MAD vs best estimate	MAD vs SC-NEVPT2
$n \to \pi^*$ (8 states)	0.31 eV	0.36 eV
$\pi \to \pi^*$ (6 states)	1.14 eV	1.08 eV

Two protocol facts belong to these numbers. The basis is 6-31G while the reference set's values are near the complete-basis limit, so part of every deviation is basis, not method; and the window corrections of the Tutorial's active-space chapter move individual entries by up to a factor of two in deviation before the family statistics are computed. The residual $\pi\pi^*$ deviation — uniform in sign across five chemically different systems — is recorded as a capability boundary of the minimal-space protocol at this basis, not as a molecule-specific error.

8.3 Selection-protocol cross-check (DVS-APC)

The entropy-based active-space selection layer was checked against a published automated-selection data set on its own terms: a port of the published formula evaluated on every orbital of every record agrees with the published entropy values to exactly zero difference, the energy layer reproduces reference state energies to 10^{-11} Ha, and the protocol's singly-occupied rule is confirmed on the full data set. The cross-check also fixed a defect this package had carried: the published protocol includes a virtual-removal correction of order N that the port initially lacked; with the correction in place, the agreement statistic reaches a mean of 0.63 over the full selection set on the primary data variant and 0.59 on the alternative, roughly six times a random baseline. The per-molecule statistics, including the cases where the two variants disagree, are in the stored report.

8.4 What is not benchmarked

Large-active-space behaviour, GPU execution, spin–orbit coupled targets, and relativistic Hamiltonians have no benchmark here because they are outside the present scope (Chapter 9). A number produced by this package on those targets has no verification record behind it.

Chapter 9

Known limitations

This is the consolidated list. Each entry states the limitation, the measurement behind it where one exists, and the check a user can run. The Tutorial has the same material in shorter form; the entries here carry the numbers.

9.1 Method-family limits

Contracted NEVPT2 accuracy. The perturbation layer inherits the standard contracted-NEVPT2 approximations; the reference set's own MAE for the method is the accuracy band claimed here, not a target this package improves on.

Valence–Rydberg mixing. $\pi\pi^*$ states near Rydberg states move by up to an electron volt when diffuse functions enter the basis. Measured on ethylene at a fixed geometry: the singlet $\pi\pi^*$ vertical excitation moves from 9.27 to 8.09 eV between 6-31G and 6-31+G* in the same three-orbital window, while the triplet moves the opposite way (4.83 to 5.68 eV); the singlet–triplet gap, which the reference fixes near 3.4 eV, collapses from 4.44 to 2.41 eV. A gap collapse in the wrong direction is the signature of a mixed-in state, not a converged one. Check: re-run with and without diffuse functions.

Minimal-window limits for multiconfigurational $\pi\pi^*$ states. The benchmark record shows a uniform positive deviation of the $\pi\pi^*$ family (+0.6 to +1.5 eV) that survives every window correction tried. Check: widen the window by one occupied orbital; a deviation that shrinks is window-limited.

No spin–orbit coupling; scalar-relativistic with ECPs only. Fine structure is out of scope. **Translated functionals beyond tPBE/tPBE0.** The on-top kernel is a closed-form PBE, written out because the libxc wrapper cannot be traced; tPBE and its 0.25 hybrid tPBE0 are the covered pair. Another translated functional needs its own closed-form kernel — a one-line change in principle, a new validation in practice. Below ρ 1e-13 the kernel differs from libxc internal guards by up to 1e-4 absolute, where the grid weights put the integral contribution below 1e-25.

9.2 Implementation limits

Dense diagonalization. The CI step materializes and diagonalizes the full determinant space; the practical ceiling is a few thousand determinants per root calculation. A differentiable iterative eigensolver is future work. The (lambda, N) DVS scan inherits this ceiling: candidate windows are refused with their determinant count recorded once they exceed `max_ndet` (default 2000), which places the published (12,12)/(14,14) windows beyond the in-stack CI by design — those remain engine-side.

Frozen on-top quadrature. Geometry derivatives of the on-top energy omit the grid-velocity term by construction; the calibration bounds it at 7.3e-5 Eh/bohr on CO/6-31G at grid

level 3, falling to 2.5e-5 at level 4.

Second-derivative cost. The implicit route requires the orbital Hessian in memory and does not exploit sparsity; the unrolled route scales with the iteration count. Neither is competitive with analytic second derivatives of single-reference methods.

Memory and compile profiles of the differentiated chain. Intermediates are broadcast over the rotation-parameter count. The measured record that shaped the current design: an explicit Hessian at about five hundred parameters requested a 402 GB allocation in a single request; the Hessian-vector-product stages that replaced it carry one tangent per pass, and their first evaluation still spends minutes in XLA compilation at that size. Check: watch resident memory during the first optimization step, and leave headroom on shared machines.

9.2.1 Degenerate pairs at a stationary point

The eigensolver's cluster tolerance must separate two scales: the floating-point noise that splits symmetry-degenerate pairs at a rotated stationary point (measured between 10^{-8} and 1.2×10^{-5} on the systems in this work) and the physical gaps of the same spectrum (1.9×10^{-2} and above). A tolerance below the noise exposes the first and second derivatives to a bare inverse gap; one above the physical gaps merges distinct states. The window measured here spans 10^{-5} to 10^{-2}, and there is no admissible value when a physical gap falls inside the noise band — the fallback for that case is the decomposition-free Lanczos kernel. Check: if a linear or otherwise degenerate system returns unstable derivatives, this tolerance is the first knob to inspect.

9.3 Protocol limits

Multiple stationary points. The orbital energy is non-convex; different starting points reach different stationary points, and the selection is not stable even at fixed settings — water at 6-31G has two minima 13 kcal/mol apart, and the reference-orbital start reaches a saddle. Report the stationarity certificate and the stationary-point spread with every multireference number.

NEVPT2 convergence sensitivity. The correction depends on the orbitals more sensitively than the energy; energy-based gates are insufficient when the target is the correction. Measured: orbital stalls that move the correction by several kcal/mol across runs whose energies agree to the microhartree. Use the stability of the correction across independent runs together with a tightened orbital-gradient gate.

Window rules and bookkeeping. The frontier audit covers the patterns met so far; the electron count follows the window ($n_{\mathrm{ele}} = N_{\mathrm{ele}} - 2n_{\mathrm{core}}$) and a hard-coded count produces a wrong ground state that the energy alone does not flag. **One-step versus relaxed MC-PDFT.** The one-step orbitals are not stationary for the on-top functional (measured |grad| 3e-3 at the CASSCF point); a relaxed minimization lies 3.0e-4 Eh below on H2/6-31G. The two numbers answer different questions — papers using the one-step protocol must not be compared with relaxed values without saying which is which.

On-top columns amplify orbital-point differences. The per-state on-top energy reacts to the state-averaged orbital point at first order, where the CASSCF column reacts at second order (slope 3e-3 per radian at a CASSCF point). Two runs whose state-averaged energies agree to 1e-7 Eh can differ by 1e-5 eV in the on-top column; treat that as the column's reproducibility floor, not as noise.

Part III

Development

Chapter 10

Architecture and contracts

10.1 The six layers

Layer	Responsibility
`core`	the numerical kernel: rotations, CASCI, CASSCF, NEVPT2, stationary points. Frozen: corrections go through the verification records.
`analysis`	post-processing of converged objects; NumPy only; no kernel import.
`policy`	selection rules over analysis output.
`driver`	composition: the differentiable SCF–CASSCF–NEVPT2 pipelines, orbital-point convergence, frozen-geometry factories.
`report`	result objects and rendering.
`cli`	the command-line entry point.

Dependencies point downward only. A user script may import any layer; the layers themselves may not import upward. The most consequential of these rules is the analysis layer's isolation: it must run with nothing but NumPy installed, which is what keeps its diagnostics usable inside any environment.

10.2 Contracts are tests

The import direction is enforced mechanically, not by review: `tests/test_layer_contracts.py` parses the syntax tree of every module and fails on any upward import. The check has a deliberate property — an import hidden inside a function body is still reported, because the contract is structural. The `lint-imports` configuration in the package metadata encodes the same rules for environments where the import-linter tool is available.

10.3 How the test suite is organized

The suite mirrors the layers: `tests/core/`, `tests/analysis/`, `tests/policy/`, `tests/driver/`, `tests/report/`, plus cross-layer integration tests. Two conventions are worth knowing before adding tests:

- The analysis tests run on synthetic fixtures with hand-built numbers and no chemistry package import; this is what keeps the layer's tests runnable anywhere. The conventions under test (determinant bitmaps, AO labels) are stated in the fixtures themselves.
- The driver tests bind the layer against the kernel: the frozen factories are pinned against the joint ones, the state-averaged solution against the single-root path, and the Hessian-vector product against the explicit Hessian. When a change touches the chain, one of these bindings is the test that should notice.

10.4 Development workflow

Non-trivial additions start as a written plan, are implemented test-first, and land with the verification record that justifies them. Bug fixes follow the same rule: the record is the fix's justification, and the test is the record made executable. The differentiable-chain and second-derivative chapters were developed in that order, and several of their numerical facts are traceable to the record that found them.

Chapter 11

Reproducing the paper

11.1 Environment

Every reproduction script sets its own runtime environment at the top: the five BLAS thread variables and `JAX_PLATFORMS=cpu` before the first JAX import, and `jax_enable_x64` immediately after. A script run outside this convention may produce different timings and, in the single-precision case, wrong numbers; the settings are quoted in Chapter 2.

The scripts write JSON results next to themselves (or into a `results/` directory); the figure sources read those JSON files. Run them from a directory with the package importable, or adjust the path insert at the top of the script.

11.2 The paper's figures and tables

Script	Result
`t157_grad_gold_fix.py`	nuclear-gradient validation; invariance conditions
`t155_grad_regression.py`	gradient regression against an analytic reference
`t150_gold_refine.py`	golden-standard Hessian by re-optimizing at each displaced geometry
`t149_mix_AD_vs_FD.py`	why finite differences fail for the mixed derivative
`t151_unrolled_theta.py`	response term by unrolled iteration
`t152_generalize.py`	unrolled route on CO and N_2
`orbital_response_implicit_` `implicit_hessian.py`	response term by implicit differentiation
`t153_hessian_to_freq.py`	harmonic frequencies from the mass-weighted Hessian
`t159_basis_active_space.py`	frequency dependence on basis set and active space
`t160_collect_freq.py`	frequency summary
`t142_layerA_v2.py`, `europium_scf_convergence`, `europium_dissociation_rerun`	lanthanide dissociation energies

11.3 The later result lines

The state-averaged and benchmark lines that postdate the paper's first draft live under the project's `code/` directory:

Script	Result
`t315 / t317 / t321`	the QUEST-subset excitation runs (with `t320` the window audit)
`t316`	the assembled benchmark table (reads the recorded JSON)
`t318`	the large-molecule single-stage protocol
`t306`	the first state-averaged targets, with the two-pass weights
`t300`–`t302`	the external selection-protocol cross-check

Each of these carries its conventions in its docstring: basis, geometry source, window rule, state-weighting pass. A number quoted from one of them without those four facts is a number that will not survive being compared with the others.

11.4 Building this documentation

The Tutorial and the Manual are xelatex documents under `manual/`; compile each twice:

```
cd manual/tutorial && xelatex tutorial.tex && xelatex tutorial.tex
cd ../manual    && xelatex manual.tex    && xelatex manual.tex
```

Part IV

Acceleration and release extensions

Chapter 12

Resolution of identity (RI/DF)

12.1 Purpose and scope

The four-center two-electron integrals of the multireference chain are the memory bottleneck of its nuclear-derivative path. Under `pyscfad`, the derivative of `int2e` allocates a dense `nctr` $\times$ `nao`^{4} array before any chemistry happens: 55.0 GB at `nao` = 96 and 2033 GB at `nao` = 175 for the systems measured here. The module `diffmrc.core.ri` replaces the four-center tensor in the correlation stage by two three-center factors and a metric, and supplies a nuclear-derivative path whose working set is `nao`^{2} $\times$ `naux` per atom.

The layer applies to any energy assembled from two-electron integrals in a Gaussian basis, provided the caller supplies a matching auxiliary basis and accepts the density-fitting approximation. It does not make the reference SCF layer RI; the SCF layer remains dense unless the separate RIJONX driver of Chapter 14 is used. It does not provide analytic second derivatives, and it does not provide an actinide fitting basis beyond the elements covered by the def2 and cc-pVDZ-RI families. Those boundaries are stated again in Section 12.5.

12.2 Theory and derivation

12.2.1 Density fitting as a least-squares problem

For each orbital product $\rho_{pq}(\mathbf{r}) = \phi_p(\mathbf{r})\phi_q(\mathbf{r})$, introduce an auxiliary expansion

$$\tilde{\rho}_{pq}(\mathbf{r}) = \sum_P c_P^{pq}\, \chi_P(\mathbf{r}), \tag{12.1}$$

where $\{\chi_P\}$ is the auxiliary basis. The coefficients are chosen by minimizing the Coulomb self-repulsion of the residual,

$$J[\epsilon] = \frac{1}{2}\iint \epsilon(\mathbf{r})\,\frac{1}{|\mathbf{r}-\mathbf{r}'|}\,\epsilon(\mathbf{r}')\,d\mathbf{r}\,d\mathbf{r}', \qquad \epsilon = \tilde{\rho}_{pq} - \rho_{pq}. \tag{12.2}$$

Stationarity with respect to c_P^{pq} gives the linear system

$$\sum_Q S_{PQ}\, c_Q^{pq} = b_{pq,P},$$

$$S_{PQ} = (P|Q), \qquad b_{pq,P} = (pq|P), \tag{12.3}$$

so

$$c^{pq} = S^{-1} b^{pq}. \tag{12.4}$$

Approximating one product in the integral by its fit and keeping the other exact gives

$$(pq|rs) \approx \left(\tilde{\rho}_{pq}\middle|\rho_{rs}\right) = \left(c^{pq}\right)^T S\, c^{rs} = b_{pq}^T S^{-1} b_{rs} = \sum_{PQ} (pq|P)\, S_{PQ}^{-1}\, (Q|rs), \tag{12.5}$$

which is the RI approximation used by the code. The same metric is used for both products; this symmetrizes the approximation and is the standard density-fitting form. The residual metric Eq. (12.2) is the origin of the physical error: the fit is optimal in the Coulomb norm, not in the wavefunction energy.

12.2.2 The $BS^{-1/2}$ factorization

Because S is symmetric and positive definite on its retained subspace, define the symmetric inverse square root

$$S = U\,\Lambda\,U^T, \qquad J \equiv S^{-1/2} = U\,\Lambda^{-1/2}\,U^T, \qquad J^T = J. \tag{12.6}$$

With $W = BJ$, the four-center tensor is

$$\left(WW^T\right)_{pq,rs} = \sum_P W_{pqP} W_{rsP} = \left(BS^{-1}B^T\right)_{pq,rs} = (pq|rs)_{\mathrm{RI}}. \tag{12.7}$$

The implementation builds W as a matrix product and then one GEMM over the pair index. The two-step form is the implementation, not an optimization: an algebraically equivalent single four-operand einsum showed absolute errors of order 10^{-6} on a 96^4 array on this stack, and two such chains differed by eight percent in the rotation gradient.

Directions with eigenvalue below `rcond` $\times$ the largest eigenvalue are truncated to zero in J; the default `rcond` is 10^{-12}, and the number of dropped directions is reported by the factor container. Truncation is a numerical regularization of the metric inverse, not part of the physical RI approximation.

12.2.3 Derivative of the symmetric inverse square root

Let $\dot{S}$ be the derivative of the metric along a nuclear displacement, and let $\dot{J}$ be the corresponding derivative of $J = S^{-1/2}$. From the eigendecomposition and the Daleckii–Krein divided difference,

$$\dot{J} = U\left[f^{[1]}(\lambda_i, \lambda_j) \circ \left(U^T \dot{S}\, U\right)\right] U^T, \qquad f(\lambda) = \lambda^{-1/2}, \tag{12.8}$$

where $\circ$ is the Hadamard product and

$$f^{[1]}(x,y) = \begin{cases} \dfrac{f(x) - f(y)}{x - y} = -\dfrac{1}{\sqrt{x}\sqrt{y}\,(\sqrt{x} + \sqrt{y})}, & x \neq y, \\ f'(x) = -\dfrac{1}{2}x^{-3/2}, & x = y. \end{cases} \tag{12.9}$$

The diagonal limit is finite at an exact degeneracy, which is why the differentiated path does not eigendecompose inside JAX and does not return NaN when two metric eigenvalues coincide. The same rule is exposed as `inv_sqrt_metric_jvp`. The derivative path uses the factor-level form of Eq. (12.7): the host code supplies B, J, and their nuclear derivatives, and a pure-JAX function with an explicit `custom_jvp` maps (B, J) to the four-center tensor. No four-center derivative integral is ever formed.

12.2.4 Error sources

Three error classes have different origins and different remedies.

- *Implementation error.* The two-step assembly against an independent DF reconstruction is 5.8×10^{-13} on H_2O/6-31G and 1.3×10^{-12} on N_2/cc-pVDZ. This is the numerical floor of the contraction, not a physical error.

- *Fitting error.* The residual of Eq. (12.2) propagates into the energy. At fixed orbitals and CI vector the measured NEVPT2 correction difference is $+1.1 \times 10^{-5}$ Ha on N_2/cc-pVDZ and -7.2×10^{-6} Ha on CO/cc-pVDZ; on ThO/6-31G with the `autoaux` set it is -1.8×10^{-6} Ha. The sign is not fixed, because the fit is not variational in the wavefunction.
- *Derivative-chain error.* The RI-CASCI gradient against a finite difference of the same RI energy is 9.6×10^{-8} on H_2O/6-31G and 3.1×10^{-7} on N_2/cc-pVDZ, absolute, in Eh/bohr.

The physical error is controlled by the fitting basis, not by the code. The values sit at the lower edge of the usual RI band (10^{-5}–10^{-3} Ha for correlation energies) and must be quoted with the fitting basis and the system attached.

12.2.5 Memory scaling

Let n_{ctr} be the number of contracted shell coefficients whose derivative is materialized by the dense path. The dense integral-derivative array carries three Cartesian components and has

$$N_{\mathrm{dense}} = 3\, n_{\mathrm{ctr}}\, n_{\mathrm{ao}}^4 \tag{12.10}$$

elements, that is $8N_{\mathrm{dense}}$ bytes in double precision. The RI three-center factor derivative for one atom, over the same three Cartesian directions, has

$$N_{\mathrm{RI}}^{(1)} = 3\, n_{\mathrm{ao}}^2\, n_{\mathrm{aux}} \tag{12.11}$$

elements, plus the two metric-derivative matrices of size $3n_{\mathrm{aux}}^2$. The ratio of the leading terms is therefore

$$\frac{N_{\mathrm{dense}}}{N_{\mathrm{RI}}^{(1)}} = \frac{3\, n_{\mathrm{ctr}}\, n_{\mathrm{ao}}^4}{3\, n_{\mathrm{ao}}^2\, n_{\mathrm{aux}}} = \frac{n_{\mathrm{ctr}}\, n_{\mathrm{ao}}^2}{n_{\mathrm{aux}}}. \tag{12.12}$$

The ratio grows with n_{ao}^2 and falls with n_{aux}; the auxiliary basis therefore has to grow only linearly with system size to keep the derivative working set below the dense array. The table in Section 12.4 compares the dense array with the peak of the whole RI chain, not with the single per-atom factor: the chain peak includes several factor arrays and intermediates, so its ratio is smaller than Eq. (12.12). The value path still materializes the dense $\mathtt{nao}^4$ tensor, because that is the object the CI layer consumes; only the derivative path is factor-level.

12.3 Input and operation

The functions below are exported by `core.ri`. Shapes follow the conventions of Section 12.3.1. All auxiliary indices are last; all integrals are real double precision.

Entry point	Returns and purpose
`make_auxmol(mol, auxbasis=None)`	`(auxmol, label)`. Tries the names in `AUX_BASIS_PRIORITY` and falls back to the orbital basis; a basis that produces zero functions counts as a failure.
`ri_three_center(mol, auxmol)`	B, shape `(nao, nao, naux)`; uses `df.incore.aux_e2`, not `mol.intor('int3c2e_sph')`.
`ri_metric(auxmol)`	S, shape `(naux, naux)`, symmetrized.
`inv_sqrt_metric(S, rcond, eig=None)`	Truncated symmetric $S^{-1/2}$, shape `(naux, naux)`.
`ri_factors(mol, auxmol, auxbasis=None, rcond)`	`RIFactors` carrying B, S, J, the metric eigendecomposition, the basis label and `rcond`.
`ri_eri_ao(mol, auxmol, factors=None, rcond)`	RI four-center tensor, shape `(nao, nao, nao, nao)`, chemist notation.
`ri_eri_mo(C, b, jhalf=None, full=True)`	`(b_mo, eri_mo)` with shapes `(norb, norb, naux)` and `(norb, norb, norb, norb)`; `eri_mo` is `None` when `full=False`.
`ri_three_center_deriv(mol, auxmol, atom)`	dB/dR_{atom}, shape `(3, nao, nao, naux)`.
`ri_metric_deriv(auxmol, atom)`	dS/dR_{atom}, shape `(3, naux, naux)`.
`ri_factors_deriv(mol, auxmol, atom, factors)`	`RIFactorDerivs` with `db`, `ds` and `djhalf`.
`ri_eri_deriv_contract(gamma, factors, derivs, per_axis=False)`	Contracts $\gamma = \partial E/\partial(\mathrm{eri_ao})$ with one atom's derivative; scalar, or a length-3 vector when `per_axis=True`.
`ri_eri_from_factors(b, jhalf)`	Pure-JAX RI four-center tensor with the analytic `custom_jvp`.
`ri_eri_jvp(b, jhalf, db, djhalf)`	Convenience wrapper returning the value and the directional derivative.

`rcond` is relative to the largest metric eigenvalue and defaults to 10^{-12}. `auxbasis=None` means the priority list; when the caller needs a specific family (for example cc-pVDZ-RI with a cc-pVDZ orbital basis), pass it explicitly. `ri_eri_deriv` materializes the full `(3, nao, nao, nao, nao)` derivative and refuses to allocate above a caller-supplied byte limit; production code uses the factor derivatives with `ri_eri_deriv_contract` or the JAX rule instead.

The intended nuclear-gradient recipe is

```
from diffmrc.core import ri

factors = ri.ri_factors(mol, auxmol)              # host, numpy
E = energy_from_factors(factors.b, factors.jhalf) # pure JAX chain
dE_db, dE_djhalf = jax.grad(E, argnums=(0, 1))(
    factors.b, factors.jhalf)
for atom in range(mol.natm):
    d = ri.ri_factors_deriv(mol, auxmol, atom, factors)
    dE_dR[atom] = (np.sum(dE_db * d.db)
                   + np.sum(dE_djhalf * d.djhalf))
```

The only derivative integrals touched are the three-center `ip1`/`ip2` arrays and the metric `ip1`/`ip2` arrays.

12.3.1 Axis and sign conventions

The conventions are fixed in the module and verified by isolated finite differences; they are repeated here because the `libcint` names are misleading.

- $B[\mu,\nu,P] = (\mu\nu|P)$: both orbital indices first, the auxiliary index last. $S[P,Q] = (P|Q)$.
- `eri_ao`$[\mu,\nu,\lambda,\sigma] = (\mu\nu|\lambda\sigma)$ in chemist notation.

- `int3c2e_ip1` differentiates the *first* orbital center and `int3c2e_ip2` differentiates the *auxiliary* center, not the second orbital center. `int3c2e_ip3` is not exported by `libcint` on this stack.
- The `ip` series carries the electronic-coordinate sign: $\partial(\mu\nu|P)/\partial R_{c(\mu)} = -$`ip1`. The second orbital center is $-$`ip1` with the orbital-pair axes exchanged, and the auxiliary center is $-$`ip2`.
- For the metric, $dS/dR_A = -(\texttt{ip1}[P \in A] + \texttt{ip2}[Q \in A])$.

The isolated checks displace the molecule with the auxiliary basis fixed and the auxiliary basis with the molecule fixed: the two groups of terms reproduce their finite differences to 7.9×10^{-11}, and the sum reproduces the full displacement to 4.8×10^{-11}.

12.4 Worked example

12.4.1 H_2O/6-31G: assembly, CASCI, and one derivative check

`examples/ri_df.py` runs the complete H_2O/6-31G CAS(2,2) case. The measured output of that run, as recorded in `manual/tutorial/chapters/10-ri-df.tex`, is

Quantity	Value
assembly identity residual	1.068×10^{-12}
dense-versus-RI RMS	8.317×10^{-6}
dense-versus-RI maximum	7.967×10^{-4}
RI-CASCI energy shift	$+1.05 \times 10^{-5}$ Eh
worst three-center derivative deviation	3.6×10^{-9}
worst ERI derivative deviation	3.7×10^{-8}
script wall time	6.4 s

The assembly identity is the implementation check: it compares the two-step GEMM with an explicit metric solve and should be at the 10^{-12} level. The dense-versus-RI columns are the approximation itself and are four to eight orders of magnitude larger. The derivative deviations are at the finite-difference floor.

12.4.2 N_2/cc-pVDZ and CO/cc-pVDZ: the physical error

At fixed orbitals and CI vector, the recorded NEVPT2 correction differences are

System	Implementation residual	Δe_2 (Ha)
N_2/cc-pVDZ	1.3×10^{-12}	$+1.1 \times 10^{-5}$
CO/cc-pVDZ	—	-7.2×10^{-6}
H_2O/6-31G	5.8×10^{-13}	—

The RI-CASCI gradient against a finite difference of the same RI energy is 9.6×10^{-8} Eh/bohr on H_2O/6-31G and 3.1×10^{-7} Eh/bohr on N_2/cc-pVDZ. These are the recorded numbers for the two requested systems, with the basis noted: the N_2 record is cc-pVDZ, not def2. A timing comparison for N_2/def2 and CO/cc-pVDZ is not part of the release record; the released timing is the H_2O/6-31G run above and the memory table below.

12.4.3 Memory at the two measured sizes

`nao`	`naux`	RI chain peak RSS	dense derivative array	array ratio
96	646	2.11 GB	55.0 GB	26×
175	746	16.05 GB	2033 GB	127×

The peaks cover the integral-and-derivative chain only, without SCF or CI. The `nao` = 96 row uses the orbital-basis fallback and the `nao` = 175 row the `autoaux` route; the memory scaling depends on `naux`, not on which fitting route produced it. The per-atom derivative factors occupy 152.9 MB at `nao` = 96. Against the measured dense whole-chain peak of 210–213 GB at `nao` = 96, the end-to-end reduction is about 100×.

12.4.4 A standalone gradient script

The core of `examples/ri_df.py` is

```
import numpy as np
from pyscf import gto, scf
from diffmrc.core import ri
from diffmrc.core.casci import cs_casci

mol = gto.M(atom='O 0 0 0; H 0.7557 0 0.5880; H -0.7557 0 0.5880',
            basis='6-31g', unit='Angstrom', verbose=0)
mf = scf.RHF(mol).run(verbose=0)
auxmol, label = ri.make_auxmol(mol)
factors = ri.ri_factors(mol, auxmol)

eri_ri = ri.ri_eri_ao(mol, auxmol, factors=factors)
eri_dense = mol.intor('int2e', aosym='s1')

c = mf.mo_coeff
h1_mo, eri_mo_dense = mo_integrals(c, mf.get_hcore(), eri_dense)
b_mo, eri_mo_ri = ri.ri_eri_mo(c, factors.b, factors.jhalf)
e_ri, ci_ri = cs_casci(h1_mo[:6, :6], eri_mo_ri[:6, :6, :6, :6],
                       ncore=4, nact=2, nele_act=2, return_ci=True)

for atom in range(mol.natm):
    d3 = ri.ri_three_center_deriv(mol, auxmol, atom)
    # compare d3 against a central difference of ri_three_center
```

The companion `examples/ri_nevpt2.py` reports the RI-NEVPT2 correction, $+1.7 \times 10^{-5}$ Ha from the dense value on the same system.

12.5 Pitfalls

- *The SCF layer remains dense.* RI removes the memory wall of the correlation stage, not of the reference determinant. A full-chain nuclear gradient for an actinide still requires an RI-JK SCF or an explicit frozen-reference partial-derivative semantics.
- *Second derivatives are not implemented.* The factor-level `custom_jvp` is first order. Differentiating the reverse pass a second time is possible in principle but is outside the validated surface; the analytic $S^{-1/2}$ rule is available first-order only.
- *Actinide fitting sets are missing.* The def2 and cc-pVDZ-RI fitting families stop at Rn. For Th the module falls back to the orbital basis (`naux` = 646, metric condition number 1.5×10^9), which shifts a CASCI energy by −0.594 mHa. The even-tempered construction `aug_etb_for_dfbasis` is the best available substitute (+0.021 mHa, `naux` = 625); a dedicated actinide JKfit basis remains an open item.
- *Basis-family matching is the caller's responsibility.* `make_auxmol` implements a priority list, not a family-aware selector. A cc-pVDZ orbital basis should be paired with cc-pVDZ-RI explicitly.

- *The truncated metric has a floor.* `rcond` defaults to 10^{-12} and no direction was dropped in the systems measured here, but the Th fallback metric is already at the 10^9 condition number where the truncation choice starts to matter.
- *The two-step assembly is not cosmetic.* A one-shot four-operand einsum is algebraically equivalent but numerically weaker on this stack; the measured rotation-gradient difference was eight percent on a degenerate system.
- *Geometry caches.* Build a fresh auxiliary molecule and factor set per geometry. The factor cache is keyed on the built molecule; an in-place geometry edit is not reliably refreshed.

12.6 Validation and references

The records behind this chapter are `tests/core/test_ri.py` (assembly, transforms, derivative contractions, JAX rule), `tests/core/test_caspt2_three_center.py` (the three-center CASPT2 consumer), `examples/ri_df.py` and `examples/ri_nevpt2.py` (the runnable H_2O/6-31G examples), `experiments/measure_ri_scf_driver.py` (the RIJONX comparison), and the tutorial chapter `manual/tutorial/chapters/10-ri-df.tex`. The error-budget table and the memory table above are the recorded values.

The density-fitting approximation and its error behaviour are documented in J. L. Whitten, *J. Chem. Phys.* **58**, 4496 (1973); B. I. Dunlap, D. W. Connolly and J. R. Sabin, *J. Chem. Phys.* **71**, 3396 (1979); O. Vahtras, J. Almlöf and M. W. Feyereisen, Chem. Phys. Lett. **213**, 514 (1993); K. Eichkorn, O. Treutler, H. Öhm, M. Häser and R. Ahlrichs, Chem. Phys. Lett. **240**, 283 (1995); and F. Weigend, *Phys. Chem. Chem. Phys.* **8**, 1057 (2006). The eigendecomposition derivative uses the Daleckii–Krein divided difference; see N. J. Higham, *Functions of Matrices: Theory and Computation* (SIAM, 2008), Chapter 3.

Chapter 13

Performance and numerical hygiene

This chapter records the implementation choices that carry a performance or numerical-branch consequence: the ordering of the MO integral transform, the blocked CI solver, the automatic solver selection, the jit reference-array form, one repeated contraction in the CI matrix–vector product, and the memory-bounded integral-derivative backend. Each choice is an engineering decision with a measured anchor; none changes the definition of the energy. The measurements were taken on one 64-core node with JAX in double precision on CPU, with the BLAS thread count fixed at one; wall clock is quoted as a relative quantity only, while compilation counts and peak resident memory are load-independent.

13.1 MO integral transforms: four sequential contractions

The two-electron transform is the most expensive single contraction in the differentiable chain, and its algebraic ordering is part of the numerical contract. The implementation is `core.nevpt2.impl.stepwise_mo_ints`, which performs four sequential contractions,

$$h_{ij} = \sum_{ab} C_{ai}\, h_{ab}\, C_{bj}, \qquad (ij|kl) = \sum_{pqrs} C_{pi} C_{qj} C_{rk} C_{sl}\, (pq|rs), \tag{13.1}$$

as one two-index transform followed by three one-index transforms:

```
def stepwise_mo_ints(Cx, h_, e_):
    h1 = jnp.einsum('ai,ab,bj->ij', Cx, h_, Cx)
    z = jnp.einsum('pi,pqrs->iqrs', Cx, e_)
    z = jnp.einsum('qj,iqrs->ijrs', Cx, z)
    z = jnp.einsum('rk,ijrs->ijks', Cx, z)
    eri = jnp.einsum('sl,ijks->ijkl', Cx, z)
    return h1, eri
```

Every MO transform in the differentiable chain calls this function, including the three canonicalization transforms and the state-averaged driver. A second copy in the driver delegates to it, so the two call paths cannot drift apart.

13.1.1 Why the one-shot form is not used

The one-shot form contracts four orbital coefficients against the four-index integral in a single expression, `jnp.einsum('pi,qj,rk,sl,pqrs->ijkl', C, C, C, C, eri)`. It is algebraically identical and was the original implementation. Two measurements removed it.

First, on a 96^4 integral array the one-shot form carried an absolute error around 10^{-6} against the sequential transform (the recorded value is 1.28×10^{-6}, with the active block agreeing to 8.9×10^{-16}). The error is harmless for the energy, which is insensitive to it, but a linear molecule's generalized Fock matrix contains exact π and δ degeneracies whose gaps are at the 10^{-15} level.

A perturbation of 10^{-6} against a gap of 10^{-15} rotates the eigenvector basis inside the degenerate subspace completely; the ratio is about 10^9. The theta gradient differed by eight percent between the two contraction orders, and the Hessian-vector product was NaN for the one-shot form and finite for the stepwise one.

Second, the sequential form is faster at production size, not slower. On the physical ThO/ECP array with 96 orbitals ($\max|eri| = 4.78$), the one-shot evaluation took 5.08–5.79 s and the stepwise evaluation 0.417–0.467 s, a factor of eleven to thirteen; on a random array of the same shape both forms finished in about 50 μs. The difference is the contraction path XLA selects for the physical memory layout, and it is a runtime effect, not a reproducibility precaution.

A release-tree cross-check qualifies the first measurement. On the same physical array the current one-shot form agrees with the stepwise form to 3.33×10^{-16} (rms 5.73×10^{-18}), so the 10^{-6} defect was not reproduced in this JAX version and on this array. The ordering is retained for both reasons above: the branch sensitivity of a degenerate subspace is a property of the contraction order rather than a fixed numerical offset, and the sequential form is the faster route at the size where the transform dominates.

13.1.2 Accuracy on the release tree

The two orders were compared at fixed geometry and fixed theta, before and after the substitution. The differences are at or below 5.7×10^{-14} Ha for the energy and 2.7×10^{-14} for the gradient; the non-degenerate H2O case is bit-identical.

System and quantity	ΔE (Ha)	$\Delta g_{\max}$
H2O/6-31G CAS(2,2), CASSCF energy and theta gradient	0	0
CO/6-31G CAS(4,4), linear, CASSCF energy and theta gradient	5.7×10^{-14}	2.7×10^{-14}
H2O/6-31G CAS(2,2), NEVPT2 energy (eager and jit)	0	—
H2O/6-31G CAS(2,2), NEVPT2 theta gradient	—	6.7×10^{-15}

The acceptance band for the substitution was 10^{-12} Ha on non-degenerate systems; the largest observed energy difference is 5.7×10^{-14}.

13.2 The S_z-blocked CI solver

`core.casci.cs_casci_blocked` diagonalizes the CI Hamiltonian one spin-projection block at a time. The partition is exact: the Hamiltonian is strictly zero between different S_z blocks (the largest measured off-block element is 0.000×10^0), so the global lowest eigenvalue is the lowest eigenvalue over the block minima. For H2O/6-31G CAS(2,2) the block structure is one determinant at $S_z = +6$, four at $S_z = +8$, and one at $S_z = +10$; the global minimum lies in the four-determinant block, whose smallest in-block gap is 2.545×10^{-2}, and the block energies are separated by 0.34 Ha.

The solver exists because the unblocked CI matrix carries a three-fold degeneracy at -84.8416418 Ha for H2O/6-31G CAS(2,2). The reverse rule of a dense `eigh` forms $1/(w_j - w_i)$ for every pair in the matrix and returns NaN at that degeneracy; inverse iteration defers the NaN from the first derivative to the second but does not remove it. Block separation is necessary but not sufficient: the reverse pass forms the same factor for every pair inside a block whether or not the pair carries cotangent, so one bit-exact pair anywhere in any block turns the untouched entries into $0 \times \infty$. The block eigenproblem therefore goes through `core.eigh.eigh_safe`, whose softened rule freezes the rotation inside a degenerate subspace.

13.2.1 Memory and time at two determinant counts

The two anchor spaces are CAS(6,6) with 924 determinants and CAS(8,8) with 12870 determinants, both on H2O/6-31G. The 12870 case used ncore = 1. Peak resident memory was measured under a cgroup limit; the plain 12870 run required a 48 GB limit and reached 37.83 GB.

Route	E (Ha)	Time	Peak RSS
plain, eager	−85.21730226017026	27.65 s	37.83 GB
blocked, eager	−85.21730226017024	84.1–88.7 s	6.78 GB
blocked, jit	same	41.70 s (1.97 compile + 39.73 evaluate)	7.78 GB
plain, jit	—	not run	—

The memory ratio is 5.58 in eager execution and 4.86 in the jitted form. The time direction is the opposite: blocked eager is 3.2 times slower than plain eager, and the blocked jit evaluation (39.73 s, excluding its 1.97 s compile) is still 1.44 times the plain eager time, although it is 2.1 times faster than its own eager form. The plain jitted form was not run because the same unblocked CI graph already exceeded 2400 s of compilation at 924 determinants; the 12870-determinant graph is larger. The blocked solver's benefit is therefore memory and compilation feasibility, not single-call time: it builds one block at a time, so the largest boolean tensor drops from $(12870, 12870, 18)$ to $(4900, 4900, 18)$ at the cost of per-block dispatch. The energy difference between the two routes is 2×10^{-14} Ha.

At 924 determinants the picture reverses: plain takes 14.7–16.9 s at 1.20–1.23 GB and blocked takes 27.8–29.8 s at 1.26–1.33 GB. There is no memory benefit and the eager solver is 1.8 times slower.

13.3 Automatic solver selection

`core.nevpt2.impl.resolve_ci_solver` maps `ci_solver='auto'` to one of the two solvers from the determinant count,

$$N_{\mathrm{det}} = \binom{2\,n_{\mathrm{act}}}{n_{\mathrm{ele,act}}}, \tag{13.2}$$

with the threshold `BLOCKED_CI_MIN_DET` = 2000: at or above it the blocked solver is selected, below it the plain solver. An explicit `'plain'` or `'blocked'` always wins, and any other value raises `ValueError`. The threshold is an engineering choice between the 924 and 12870 anchors, not a measured optimum; the crossover between the two anchors was not measured.

```
from diffmrc.core.nevpt2.impl import (canonicalize_jax,
                                      resolve_ci_solver)

# Inspect the decision without running a calculation:
solver = resolve_ci_solver('auto', nact, nele_act)   # 924 -> 'plain'

# canonicalize_jax accepts 'auto', 'plain', or 'blocked':
C_can, ci_can, eps = canonicalize_jax(
    C, h1e, eri, ci, ncore=ncore, nact=nact, nele_act=nele_act,
    ci_solver='auto')
```

The flag applies to the two internal CI re-solves of the canonicalization routines (`canonicalize_jax` and `canonicalize_sa_jax`). The main CI solve of the production chain remains on the plain solver deliberately: the blocked solver returns the exact eigenvector, while the plain solver returns an inverse-iteration vector with a residual of order 10^{-8}, and substituting the vector would change the NEVPT2 correlation energy by more than the 10^{-12} acceptance band. In the state-averaged routine the flag also has no effect when more than one root is requested, because there is no multi-root blocked solver and the state-average branch of `cs_casci` is required.

13.3.1 What the switch leaves invariant

Orbitals and orbital energies are bit-identical between the two routes; the CI vector differs by a global phase and by the plain solver's residual. The values below are the phase-aligned differences.

System	N_{det}	$\max \lvert\Delta C\rvert$	$\max \lvert\Delta\epsilon\rvert$	CI difference
H2O/6-31G CAS(2,2)	6	0	0	4.71×10^{-10}
CO/6-31G CAS(4,4)	70	0	0	5.39×10^{-8}

The identity is the intended semantics: the internal re-solve enters no returned quantity except the CI vector, while the natural-orbital rotation and the sub-block Fock diagonalization are built from the input vector.

13.3.2 The small-system penalty the default avoids

Forcing the blocked solver on a small space is a net cost. On the H2O/6-31G CAS(2,2) NEVPT2 chain, the eager correction takes 23.50 s with the plain solver and 31.41 s with the blocked solver, a 36 percent increase, and the end-to-end wall clock moves from 60.3 s to 74.2 s. With `'auto'` the same chain resolves to plain and returns to 22.26 s eager and 62.8 s wall clock, within the load band of the pre-change measurement. The measured anchors behind the threshold are collected below.

N_{det}	System	plain	blocked	Decision
6	H2O CAS(2,2)	23.50 s eager, 60.3 s wall	31.41 s eager, 74.2 s wall	plain
70	CO CAS(4,4)	9.5–16.4 s	17.2–24.6 s	plain
924	H2O/CO/N_2 CAS(6,6)	14.7–16.9 s, 1.20–1.23 GB	27.8–29.8 s, 1.26–1.33 GB	plain
12870	H2O CAS(8,8)	27.65 s, 37.83 GB	84.1–88.7 s, 6.78 GB	blocked

13.4 Reference arrays as jit arguments

The theta-only factories compute the reference SCF and the AO integrals once, at construction, and return a function of the rotation parameters. The reference arrays can enter that function in one of two ways: closed over, or passed as traced arguments. In the closure form the arrays become constants of the traced graph, so a jitted wrapper compiles one executable per geometry. In the array form the arrays are runtime arguments, and one executable serves every geometry whose arrays have the same shape and dtype.

The API exposes both members on the returned callable: `E_theta.component_args` is the array-argument function and `E_theta.arrays` is the tuple to pass, in order. The state-averaged energy factory `make_sa_energy_fn` carries the same pair, and the MC-PDFT table passes them through `converge_orbital_point(..., arrays=E_sa.arrays)` so that the orbital search does not compile the AO integrals into its graph.

```
import jax

E_theta = make_nevpt2_theta_fn(mol, ncore, nact, nele, nmo, pairs,
                               scf_builder)
E_jit = jax.jit(E_theta.component_args)   # one executable
E_ref = E_jit(theta, *E_theta.arrays)     # for every same-shaped geometry

# The orbital-point search takes the same pair:
converge_orbital_point(E_theta.component_args, len(pairs),
                       arrays=E_theta.arrays)
```

The measured difference on three geometries (the reference scaled by −1 percent, 0, and +1 percent about the center of mass) is one compile against three, and 2.67 s against 8.60 s for the three calls, measured as interleaved medians.

Form	jit objects / compilations	Total for three geometries
closure constants	3 / 3	8.60 s
array arguments	1 / 1	2.67 s

The compile count is the load-independent quantity; the wall clock matters at production scale, where a single f-block window compile is of order fifteen minutes. Three closures then cost about forty-five minutes against fifteen minutes plus two evaluations in the array form.

One restriction belongs to this section. The geometry-dependent joint entry `make_nevpt2_joint_fn` cannot be traced under `jax.jit` in this pyscfad build, because the SCF it runs inside the call raises a tracer-conversion error. The array-argument form is therefore a fixed-geometry device and belongs to the theta-only factories. The correlation contraction itself is inside the jitted theta-only function: the NEVPT2 energy in eager and jitted execution agrees bit for bit, so the untraceable part is the SCF, not the second-order correction.

13.5 Repeated core folding in the CI matrix–vector product

`core.ci_matvec` builds the effective one-electron operator once per matrix–vector product. The original code recomputed that fold separately inside the apply and diagonal paths, so every product paid it twice. The apply path now receives the folded quantities $(h_{\mathrm{eff}}, g, e_{\mathrm{core}})$ from a shared helper, and the diagonal path keeps its own call. The output is unchanged bit for bit: the norm of the test vector is 3977.4197449984595 in every interleaved run before and after the change, and the six leading components agree as well.

The timing consequence is small. In eager execution the apply routine's cumulative time moved from 5.49 s to 5.36 s, about 2.5 percent and inside the load noise; after jitting, one product takes 10–30 ms and the before/after distributions are indistinguishable. This is a code-hygiene change, not a performance promise.

13.6 Memory-bounded integral derivatives

The nuclear gradient of every energy in this package is a reverse-mode derivative through pyscfad's integral objects. The upstream default represents the four-centre two-electron integrals as a `custom_jvp`; its reverse transpose replays one basis tangent per Cartesian direction and materialises `nctr * nao`^{4} doubles, with `nctr` $= 3\,n_{\mathrm{atm}}$. `core.adint` selects pyscfad's `custom_vjp` kernel (`moleintor_opt = True`) for the four-centre integrals; its backward kernel contracts the `nao`^{4} cotangent in C and returns only an $(n_{\mathrm{atm}}, 3)$ array. The working set is the cotangent itself, not a factor `nctr` times it.

The switch is process-wide and off by default. Set

```
export DIFFMRC_VJP_INTEGRALS=1
```

before the process starts; `core.adint` reads the variable at import time, so a queued run selects the backend without an edit to its script. The values `1`, `true`, `yes` and `on` (case-insensitive) enable it; anything else, and an unset variable, leave it off. The same selection is available in code as `use_vjp_integrals(True)`.

Turning the switch on installs a guard on `pyscfad.gto.Mole.build`: the basis exponents and the contraction coefficients are left untraced by default, because the vjp kernel cannot differentiate them and raises rather than dropping them silently. A caller that passes `trace_exp=True` or `trace_ctr_coeff=True` explicitly is refused with a message that names the argument. New code

should build the molecule with `adint.build_mole`, which sets the two flags openly and leaves the nuclear coordinates as the only traced leaf.

The tensor layout is the visible difference. The vjp kernel knows only the `s4` packing, so `int2e_s1` requests `s4` and restores `s1` with two differentiable gathers on the `(nao, nao)` pair index; no `nao`^{4} integer index map is built. Neither switch changes a value: with the switch off, `int2e_s1` is exactly `mol.intor('int2e', aosym='s1')`, and the two backends agree on the ERI coordinate derivative to 10^{-15} relative in the unit tests and on the nuclear gradient to 10^{-12} relative.

13.6.1 Measured peak memory

Quantity	`nao`	default backend	vjp backend
bare integral derivative, 8 atoms	124	18 999 MB	7 797 MB
bare integral derivative, 4 atoms	104	9 565 MB	4 151 MB
ThO CAS(8,8), SCF rung	96	8 606 MB	2 537 MB
ThO CAS(8,8), SCF + MO transform	96	14 398 MB	6 125 MB
ThO CAS(8,8), SCF + MO + CI rung	96	oom-killed	19 267 MB
ThO CAS(8,8), analytic gradient	96	—	19 278.5 MB

On ThO (`Th=stuttgart_rsc` ECP, `O=6-31G`, CAS(8,8)) the vjp backend moves the stage ladder from an out-of-memory kill at the CI rung to a completed ladder. The analytic gradient at that rung finishes at 19 278.5 MB, against a central difference of the same energy: 1.22×10^{-5} Eh/bohr absolute, 2.04×10^{-7} relative. The canonicalization and NEVPT2 rungs remain above 20 GB on both backends; their cost is the perturbation layer, not the integral derivative (ThO/CAS(6,6) forward alone: 8 709 MB, 1 152 s).

The two-centre integrals stay on the `custom_jvp` backend even with the switch on, because the `custom_vjp` two-centre kernel segfaults on `ECPscalar` (measured on HI/lanl2dz, for both the bare integral gradient and the SCF gradient; `int1e_kin`, `int1e_nuc` and `int1e_ovlp` are fine). A two-centre tensor is `nao`^{2}, so its `custom_jvp` transpose costs `nctr * nao`^{2} — six orders of magnitude below the four-centre term at `nao` = 100 — and is not worth the risk.

13.7 Choosing among the options

Option	Default	When to change it
`ci_solver`	`'auto'`	Keep the default. Force `'plain'` only when the exact pre-change CI vector must be reproduced; force `'blocked'` only on a large determinant space where the plain tensors do not fit, and expect the single-call time to grow.
MO transform	stepwise	Always use `stepwise_mo_ints`; the one-shot form is not exposed as an option.
jit reference arrays	array form	Use `component_args` and `arrays` for every repeated geometry. Do not jit the joint geometry-dependent entry.
`ci_matvec` folding	single fold	No user action; the change is internal and numerically neutral.
`DIFFMRC_VJP_INTEGRALS`	off	Set to `1` on a memory-bound gradient chain. Build molecules with `adint.build_mole` or accept the build guard; the gradient agrees with the default backend to 10^{-12} relative.

Two properties of the measurements above belong to their reading. Wall-clock numbers are relative and were taken on one shared node with the load recorded; the compile counts, the peak resident memory, and the energy and gradient differences are the reproducible quantities. The 2000-determinant threshold is a choice between two measured anchors, and a user whose determinant count lies between them should measure the two routes on that space rather than assume the default.

Part V

New in version 0.3

Chapter 14

New modules in version 0.3

Version 0.3 adds the layers of this chapter to the public surface: a relaxed-path NEVPT2 nuclear gradient, a coupled CASPT2 contraction, a valence-level spin-orbit mean field for effective-core potentials, a RIJONX self-consistent-field driver, and a scalar density-fitted X2C CASSCF layer. Each is additive: no existing signature changed, and the release core described in Chapter 5 is untouched. This chapter states, for each layer, what the entry point computes, what the caller must supply, and where the implementation stops. The RI factor conventions reused by two of the layers are those of Chapter 12.

Two subjects receive dedicated chapters because they carry a class table or a second wavefunction representation rather than a single entry point. The eight CASPT2 classes, the three-active-index treatment, the coupled path, and the three-center contraction are in Chapter 15. The scalar and two-component X2C layers, including the two-component Hartree–Fock layer that is not part of this chapter's scalar summary, are in Chapter 16.

Module	Main entry point	Layer
`driver.nevpt2_zvector`	`nevpt2_relaxed_gradient_zvector`	response on top of the differentiated chain
`core.caspt2`	`caspt2_contracted_e2_coupled`	value (NumPy)
`core.caspt2_three_center`	`caspt2_contracted_e2_three_center`	value (NumPy, three-center coupling)
`core.somf_ecp`	`valence_somf_matrix`	value (NumPy)
`driver.ri_scf`, `driver.ri_scf_driver`	`rijonx_scf`, `rijonx_scf_gradient`	SCF (pyscfad)
`core.x2c_casscf_df`	`df_x2c_casscf`	value and analytic gradient
`core.x2c_df`	`df_x2c_hf`, `df_x2c_hf_gradient`	two-component HF value and analytic gradient

14.1 Relaxed NEVPT2 nuclear gradients

`driver.nevpt2.nevpt2_nuclear_gradient` differentiates the frozen-theta energy: the SCF reference and the CI vector are rebuilt at every displaced geometry, but the orbital-rotation parameters θ stay at their stationary values. Along the physical path θ follows the CASSCF stationarity condition, and NEVPT2 is not variational in the orbitals, so the frozen partial is not the relaxed derivative. The correction is the Z-vector response

$$g_{\mathrm{relaxed}} = g_{\mathrm{frozen}} - M^T H_{\theta\theta}^{-1} s, \qquad H_{\theta\theta} = \frac{\partial^2 E_{\mathrm{CASSCF}}}{\partial \theta^2}, \quad M = \frac{\partial^2 E_{\mathrm{CASSCF}}}{\partial \theta\, \partial R}, \quad s = \frac{\partial e_2}{\partial \theta}, \tag{14.1}$$

with $e_2 = E_{\mathrm{NEVPT2}} - E_{\mathrm{CASSCF}}$. The vector $z = H_{\theta\theta}^{-1} s$ is the orbital Z-vector: one linear solve per state serves every Cartesian direction. The CI coordinates need no separate multiplier in this

reduced space, because both energy functions solve the CI eigenvalue problem internally; the CI block of the full Hessian is replaced by its Schur complement, which is $H_{\theta\theta}$ as written.

Entry point	Returns and purpose
`make_nevpt2_joint_fn_si_exact(...)`	$E(\mathrm{mol}, \theta)$ with the exact near-zero `Si` channels; same contract as `driver.nevpt2.make_nevpt2_joint_fn`. Takes the space definition, the orbital pairs, the SCF builder, `e2_scale`, and optionally `align_ref` and `align_bounds`.
`align_reference_blocks(...)`	Differentiable blockwise alignment of the SCF coefficients onto a fixed reference; arguments `(C, S, C_ref, bounds)`, where `bounds` is a list of `(lo, hi)` orbital ranges.
`orbital_zvector(...)`	Solves $H_{\theta\theta} z = s$ by truncated pseudo-inverse; arguments `(H_tt, source, rcond=None, symmetrize=True)`; returns `(z, info)` with the spectrum, condition number and residual.
`lagrangian_relaxed_gradient(...)`	The contraction $g_{\mathrm{frozen}} - M^T z$; arguments `(g_frozen, z, M)`.
`nevpt2_relaxed_gradient_zvector(...)`	The assembled relaxed gradient; returns `(gradient, info)`. Required arguments: `mol`, `theta`, `E_casscf`, `E_nevpt2`; optional `rcond`, `H_tt`, `M`, `source`, `g_frozen`.
`si_energy_from_invariants(...)`	The `Si` channel sum over `(norm, ener, diff)`; `mode='masked'` reproduces the release value mask for diagnostics only.

Three settings carry the accuracy of the response.

- *Reference convergence.* The reference SCF is converged with `conv_tol` of 10^{-13} and `conv_tol_grad` of 10^{-10} at every geometry. The response operator is a second derivative of the same chain, so its accuracy is bounded by the orbital-gradient convergence of the reference, not by the energy convergence alone.
- *Blockwise alignment.* The SCF solution is defined only up to orthogonal mixing inside the core, active and virtual blocks, and two neighbouring geometries can return different representatives (a phase flip, or an ordering change inside a near-degenerate pair). Because θ is defined relative to that representative, an unaligned reference makes the same θ mean different orbitals: on H_2O/6-31G the measured fixed-theta energy slope was -5.8 Eh/bohr against -0.0528 Eh/bohr in the aligned gauge. Pass the base-geometry coefficients as `align_ref` and the block bounds as `align_bounds`; the map is the identity at the base geometry.
- *Exact near-zero `Si` channels.* The release core evaluates the `Si` class as $-\sum_i n_i/(d_i + h_i/n_i)$ with a value mask $|n_i| > 10^{-12}$, where n_i is the channel norm, d_i its zeroth-order denominator and h_i the core-coupling term. Symmetry can drive an individual n_i to 10^{-13}: dropping such a channel changes the energy by about 10^{-13} Eh but its derivative by order 0.1–1, because the mask is a discontinuity in the energy, not a removable singularity. The module evaluates the same algebra as $-\sum_i n_i^2/(d_i n_i + h_i)$ with the guard moved to the denominator. Only the `Si` class is treated this way; the other four contraction classes keep the release masks.

The validation covers closed-shell, frozen-core H_2O/6-31G and LiH/6-31G, CAS(2,2), against finite differences of the relaxed path. Open-shell, ECP, f-block and large-basis systems are outside the validated range. A minimal call is

```
import jax.numpy as jnp
from pyscfad import scf
```

```
from diffmrc.core.orblib import orbital_pairs
from diffmrc.driver.nevpt2_zvector import (
    make_nevpt2_joint_fn_si_exact, nevpt2_relaxed_gradient_zvector)

ncore, nact, nele = 4, 2, 2
nmo = mol.nao_nr()
pairs = orbital_pairs(ncore, nact, nmo, include_cv=True)

def builder(m):
    mf = scf.RHF(m)
    mf.conv_tol = 1e-13
    mf.conv_tol_grad = 1e-10
    return mf

mf = builder(mol)
mf.kernel()
bounds = [(0, ncore), (ncore, ncore + nact), (ncore + nact, nmo)]
ref = jnp.asarray(mf.mo_coeff)

# theta: CASSCF stationary parameters at this geometry
E_casscf = make_nevpt2_joint_fn_si_exact(
    ncore, nact, nele, nmo, pairs, builder, e2_scale=0.0,
    align_ref=ref, align_bounds=bounds)
E_nevpt2 = make_nevpt2_joint_fn_si_exact(
    ncore, nact, nele, nmo, pairs, builder, e2_scale=1.0,
    align_ref=ref, align_bounds=bounds)

grad, info = nevpt2_relaxed_gradient_zvector(
    mol, theta, E_casscf, E_nevpt2, rcond=1e-7)
```

The cutoff `rcond` is a relative cutoff on the pseudo-inverse. The H_2O CASSCF Hessian carries one soft mode (eigenvalue 3.8×10^{-6}, condition number 2.3×10^{7}) whose response is not part of the correlation source; 10^{-7} cuts it, while a looser cutoff keeps it and amplifies roundoff.

The response is validated on the closed-shell frozen-core systems H_2O/6-31G and LiH/6-31G, both CAS(2,2), against finite differences of the relaxed path. The chain-level checks that the exact `Si` assembly does not disturb the release total are in `tests/driver/test_nevpt2_zvector.py`: with a core present the corrected and release totals agree to 10^{-11} on H_2O/STO-3G, the LiH/6-31G chain reproduces the release total to 10^{-10} with a masked channel, and the no-core case reduces identically. The same file pins the algebra of `si_energy_from_invariants`, the blockwise alignment map, the adjoint solve, the Lagrangian identity, and the soft-mode cutoff. The scope of the response operator is narrower than the scope of the energy: open-shell, ECP, f-block, and large-basis relaxed gradients are outside the validated range, and the exact near-zero treatment covers the `Si` class only. The other four contraction classes retain the release value mask; no validation system with a masked channel outside `Si` has been found.

14.2 Coupled CASPT2

`core.caspt2` has two contraction schemes of the same eight-class internally contracted basis; the class table and the three-active-index treatment are documented in Chapter 15, and the summary here is limited to the coupled path. The diagonal path (`caspt2_contracted_e2`) evaluates each contracted function with its own denominator,

$$e_2 = -\frac{W^2}{A - E_0 S}, \tag{14.2}$$

which is the CASPT2-D approximation. The coupled path replaces the diagonal zeroth-order Hamiltonian by the state-specific generalized Fock operator $H_0 = \sum_{pq} f_{pq} E_{pq}$. Its off-diagonal blocks (inactive–active, active–virtual, inactive–virtual) connect determinants of different excitation classes, and the amplitudes solve the projected linear system

$$(A - E_0 S)\, x = -W, \qquad A_{pq} = \langle K_p | H_0 | K_q \rangle, \quad S_{pq} = \langle K_p | K_q \rangle, \quad W_p = \langle K_p | H | 0 \rangle, \tag{14.3}$$

with $e_2 = \sum_p W_p x_p$ and the class decomposition $e_{2,\mathrm{class}} = \sum_{p \in \mathrm{class}} W_p x_p$. The overlap matrix S is regularly linearly dependent; it is diagonalized and eigenvectors with eigenvalue below `rtol` $\times$ $\max(1, \lambda_{\max})$ are dropped.

Entry point	Returns and purpose
`caspt2_contracted_e2_coupled(...)`	`(e2_total, breakdown)`, plus the assembled `S`, `A`, `W`, `E0`, `x` and the class labels when `return_system=True`. Arguments: the MO integrals, the CI vector and determinant bitmaps, `ncore`, `nact`, `nele`; optional `fock` and `rtol`. With `fock` omitted, the state-specific generalized Fock is built from the CI vector and the integrals.
`caspt2_contracted_e2`, `caspt2_contracted_e2_jax`	The diagonal-Fock path and its differentiable twin; the latter is the jax chain that can be traced in the CI coefficients.

The relative weight of the `VM` class follows the reference implementation: with the two orderings written cross and direct, the Molcas normalisation is $V_P = \mathrm{cross} + \mathrm{direct}$ and $V_M = \sqrt{3}\,(\mathrm{cross} - \mathrm{direct})$. The factor $\sqrt{3}$ multiplies the whole `VM` function and cancels in $W^2/(A - E_0 S)$, so the operator content carried here is cross $-$ direct, that is, a relative weight of -1 between the two orderings. Both components enter the energy independently for off-diagonal pairs and coincide for diagonal pairs. Version 0.3 corrects the previous convention for this component.

The coupled total against the reference CASPT2 class table is 1.0081 on H_2O/6-31G and 1.0100 on CO/6-31G, both CAS(2,2). The path is a NumPy value layer: it has no jax twin, and the differentiable CASPT2 route remains the diagonal-Fock contraction `caspt2_contracted_e2_jax` at the CASPT2-D level. The cost is n_{fn}^2 function-pair contractions plus one H_0 application per function, intended for small and medium active spaces.

```python
from diffmrc.core.caspt2 import caspt2_contracted_e2_coupled

# h1e_mo, eri_mo: MO integrals in chemist notation, from the CASSCF
# orbitals; ci: reference CI vector; dets: determinant bitmaps.
e2, breakdown = caspt2_contracted_e2_coupled(
    h1e_mo, eri_mo, ci, dets, ncore=ncore, nact=nact, nele=nele)

# With an explicit state-specific generalized Fock matrix, and the
# assembled system for diagnostics:
e2, breakdown, system = caspt2_contracted_e2_coupled(
    h1e_mo, eri_mo, ci, dets, ncore=ncore, nact=nact, nele=nele,
    fock=fock, return_system=True)
```

14.3 Valence-level SOMF for effective-core potentials

The all-electron SOMF operator of `core.somf` uses the bare nuclear charges and the full electron density. With an effective core potential the core electrons are absent, and the spin-orbit part of the ECP replaces their contribution; adding the all-electron formula unchanged would use

the ECP effective charge in `int1e_pnucxp` and a valence-only density. The module defines the valence-level operator

$$h_k^{\text{SOMF-ECP}} = \frac{\alpha^2}{2}\left[h_k^{\text{nuc}} + h_k^{2e}[P]\right] - V_k^{\text{ECP}}, \tag{14.4}$$

with h_k^{nuc} the nuclear one-electron term on centers without a spin-orbit ECP, $h_k^{2e}[P]$ the valence $J - \frac{3}{2}K$ mean field of the spin-summed density P ($\mathrm{tr}(PS) =$ `mol.nelectron`), and V_k^{ECP} the three-component `ECPso` integral. On a center carrying a spin-orbit ECP the ECP term replaces the nuclear term: the ECP is fitted to reproduce the all-electron spin-orbit splitting of the valence configuration and already contains the nuclear one-electron term and the core-valence screening for that center. The Breit–Pauli terms carry the factor $\alpha^2/2$; `ECPso` is already an effective operator in energy units and is subtracted.

Entry point	Returns and purpose
`valence_somf_matrix(...)`	The operator, shape `(3, nao, nao)`. Arguments: `(mol, P)`, then `nuclear='auto'`, `ecp_term`, `contract`, `memory_limit`, `convention='i'`, `check`. `nuclear` selects which centers keep the nuclear term (`'auto'` excludes spin-orbit ECP centers); `convention='i'` returns $H_{\text{SO}} = i\sum_k h_k s_k$.
`ecp_soc(mol)`	The three-component `ECPso` integral, shape `(3, nao, nao)`, with no $\alpha^2/2$ factor.
`nuclear_soc(mol, atoms=None)`	The nuclear one-electron SOC in the `int1e_pnucxp` convention, optionally restricted to a list of centers.
`so_ecp_centers(mol)`, `nuclear_centers(mol)`	Atom indices that carry a spin-orbit ECP, and those that do not.
`somf_density_response(...)`	The density-response matrix $M_{rs} = \sum_{pq} D_{pq}[g_{pqrs} - \frac{3}{2}g_{prsq} - \frac{3}{2}g_{sqpr}]$ for `(mol, D)`; only `contract='explicit'` is implemented.
`somf_definition(mol)`	A dictionary recording which centers contribute which term and the approximations that are not fixed by the ECP.

The two-electron kernel is not symmetric under the pair exchange $(pq) \leftrightarrow (rs)$. Consequently the derivative of $\mathrm{tr}(D\,h^{2e}[P])$ with respect to P is not $h^{2e}[D]$; a gradient of an SOC matrix element must use `somf_density_response`, or the equivalent bilinear form. The value layer is NumPy; orbital and CI response are the caller's responsibility, and only the explicit contraction is implemented in the response kernel.

There is no unique first-principles decomposition of an ECP into one- and two-electron spin-orbit parts: the ECP is fitted to atomic reference states, so part of the valence–valence mean field is already folded into its parameters. Adding the explicit valence $J - \frac{3}{2}K$ term is therefore an approximation with a double-counting error that cannot be removed without the ECP generation data. The operator is a prototype for relative comparisons and is not a production absolute-SOC method. The pieces are exposed independently (`nuclear=`, `ecp_term=`) so that variants can be compared and the error bounded.

```
from pyscf import gto

from diffmrc.core import somf_ecp

mol = gto.M(atom='Yb 0 0 0; F 0 0 3.6',
            basis={'Yb': 'stuttgart_rsc', 'F': '6-31g'},
            ecp={'Yb': 'ecpds28mwbso'}, unit='Bohr', verbose=0)

# P: spin-summed density of the electrons present in the calculation,
```

```
# trace(P S) = mol.nelectron.  D: the same density, or the transition
# 1-RDM for a matrix element between two scalar states.
h_soc = somf_ecp.valence_somf_matrix(mol, P, convention='i')
m_resp = somf_ecp.somf_density_response(mol, D)
```

On the YbF test molecule the `ECPso` norm on Yb is 17.04 per component, while the raw nuclear operator norm on the same center is of order 10^4; adding both would be dominated by an operator that the ECP was built to replace.

14.3.1 Element range and validation

The operator applies exactly to the centers whose effective core potential carries a spin–orbit component. A molecule is eligible when `mol.has_ecp_soc()` is true; otherwise `ecp_soc` raises and the caller must use the all-electron `core.somf` module instead. A scalar-only ECP does not supply `ECPso` and cannot be promoted to a spin–orbit operator by this module. Within the validated set, the parameterizations exercised are the shape-consistent and energy-consistent spin–orbit sets exposed by pyscf: Ge with `ecpds28mdfso` and Yb with `ecpds28mwbso`, in both cases paired with a matching orbital basis and a scalar ECP on the ligand centers. The module makes no statement about elements outside that set; eligibility is a property of the ECP data, not of the atomic number.

On GeF with a spin–orbit ECP the valence-level SOMF splitting is 28.37 cm^{-1}, against 43.72 cm^{-1} for the nuclear-only operator on the same density. The difference is the valence mean field and is the quantity the ECP decomposition does not fix uniquely. The gradient prototype on the same system follows a finite difference of the SOC matrix element to 2.4×10^{-5} relative; the separate density-response term is 4.4×10^{-7} in that direction, which is the size the non-symmetric kernel contributes and the reason `somf_density_response` exists. The YbF case exercises the opposite regime, where the ECP term dominates the nuclear operator.

14.4 RIJONX SCF

The reference SCF of the multireference chain still formed the four-center Coulomb tensor through the dense integral path. The RIJONX route fits the Coulomb term and keeps the exchange exact:

$$E = E_{\mathrm{nuc}} + \mathrm{Tr}(Dh) + \tfrac{1}{2}\mathrm{Tr}(DJ_{\mathrm{RI}}) - \tfrac{1}{4}\mathrm{Tr}(DK_{\mathrm{exact}}), \qquad J_{\mathrm{RI}} = WDW^T, \tag{14.5}$$

with $W = BS^{-1/2}$ the metric-folded three-center tensor of Chapter 12. The exchange branch is the dense four-center integral, so the method is RIJONX: not RI-JK, which has no analytic gradient here, and not RIJCOSX, whose fixed-grid prototype is outside the production surface.

Entry point	Returns and purpose
`rijonx_scf(...)`	A converged pyscfad mean-field object; arguments `(mol, auxbasis=None, rcond=1e-12, uhf=False, **kernel_kwargs)`. `RIJONX_ROHF` is selected when `mol.spin` is non-zero, and `RIJONX_UHF` when `uhf=True`; a non-converged run raises.
`rijonx_scf_gradient(...)`	Full nuclear gradient of the converged RIJONX energy (RHF, ROHF, or UHF layout), shape `(natm, 3)` in Eh/bohr; arguments `(mf, auxbasis, rcond)`.
`rijonx_energy(mol, C, ...)`	The energy of Eq. (14.5) at fixed coefficients.
`rijonx_gradient(mol, C, ...)`	The fixed-coefficient (skeleton) gradient: RI-J analytic, exchange dense, plus core and nuclear terms.
`ri_j_*`, `exact_k_*`, `core_hamiltonian_*`, `nuclear_repulsion_gradient`	The individual energy and gradient pieces.
`make_cosx_grid`, `cosx_exchange_matrix`, `cosx_exchange_energy`	A fixed-grid COSX prototype for exchange; not a production route.

The gradient convention is the point that matters. The functions in `driver.ri_scf` return a fixed-coefficient gradient: the coefficients are held at their reference values and only the integrals and the nuclear repulsion move. That is not the SCF gradient even at a converged density; the missing orbital-response term reduces to the overlap-derivative (energy-weighted density) contribution, $-2\sum_{\mu\in A} Y_{\mu\nu}\,\partial S_{\mu\nu}/\partial R$, with $Y = 2C_{\mathrm{occ}}\epsilon C_{\mathrm{occ}}^{T}$. Without it the individual terms cancel strongly and the skeleton sum differs from the SCF gradient by order 1 Eh/bohr. `rijonx_scf_gradient` supplies the completion and is the entry point to use for a converged SCF.

The auxiliary basis defaults to `def2-universal-jkfit` and is resolved by `make_auxmol` in `core.ri`, which falls back to the orbital basis when a fitting set does not cover every element. Build a fresh molecule and mean-field object per geometry: the RI factor cache and the pyscfad integral caches are keyed on the built molecule and are not reliably refreshed by an in-place geometry edit.

The exact exchange branch still forms and caches the dense `nao`^{4} tensor and, under geometry differentiation, its dense derivative. RI-J removes the Coulomb part of that work, not the exchange part. Range-separated Coulomb is refused because the factors are built for the full operator. ECP molecules work when the molecule is built with an explicit `ecp` argument.

The explicit gradient covers closed-shell and open-shell references. `rijonx_scf_gradient` dispatches on the occupation layout. For a closed-shell reference it uses the overlap completion with $W = 2C_{\mathrm{occ}}\epsilon C_{\mathrm{occ}}^{T}$. For an ROHF or UHF reference it calls the open-shell branch, which assembles the skeleton gradient from the spin-summed energy

$$E = E_{\mathrm{nuc}} + \mathrm{Tr}(D_{\mathrm{tot}}h) + \tfrac{1}{2}\mathrm{Tr}(D_{\mathrm{tot}}J_{\mathrm{RI}}[D_{\mathrm{tot}}]) - \tfrac{1}{2}\sum_{\sigma}\mathrm{Tr}(D_{\sigma}K[D_{\sigma}]), \tag{14.6}$$

so the RI-J term and the core Hamiltonian see the total density while the exact exchange sees each spin density separately. The orbital response is the overlap term $-2\sum_{\mu\in A} W_{\mu\nu}\,\partial S_{\mu\nu}/\partial R$, with the spin-resolved energy-weighted density that pyscf's ROHF and UHF gradient modules use: $P_a f_a P_a + P_b f_b P_b$ for ROHF, and the occupation-weighted orbital-energy densities for UHF. The Fock matrices come from `mf.get_veff`, that is, from the same RI-J plus exact-K assembly the SCF converged on. A restricted open-shell reference is the default for a non-zero `mol.spin`; pass `rijonx_scf(mol, uhf=True)` to select `RIJONX_UHF`. Fractional occupations are still refused.

```
from diffmrc.driver.ri_scf_driver import rijonx_scf, rijonx_scf_gradient

mf = rijonx_scf(mol, auxbasis='def2-universal-jkfit')
e_tot = mf.e_tot
```

```
# Closed-shell full gradient (skeleton plus overlap completion):
grad = rijonx_scf_gradient(mf)

# ROHF reference (mol.spin != 0), same entry point:
# grad_open = rijonx_scf_gradient(mf_rohf)

# UHF reference:
# mf_uhf = rijonx_scf(mol, uhf=True)
# grad_uhf = rijonx_scf_gradient(mf_uhf)
```

The energy and gradient are validated on H_2O and CO (closed shell), the OH radical (ROHF and UHF, gradients against implicit differentiation and central differences), OF (a second ROHF system), and HI/def2-svp with an ECP (gradient against central differences). The explicit and implicit closed-shell gradients agree to 10^{-6} Eh/bohr; the ROHF explicit gradient agrees with implicit differentiation to 10^{-6} Eh/bohr and with central differences to 10^{-5} Eh/bohr; the UHF explicit gradient agrees with implicit differentiation to 10^{-5} Eh/bohr; and the ECP gradient deviation is 3.0×10^{-7} Eh/bohr. The measured RIJONX-minus-dense energy difference on HI is -2.6×10^{-5} Eh, which is the J fit, not an implementation error.

14.5 Scalar DF-X2C CASSCF

`core.x2c_casscf_df` adds a state-specific and state-averaged CASSCF layer on the scalar (spin-free) X2C one-electron Hamiltonian with a density-fitted two-electron part. The full relativistic family, including the two-component DF-X2C Hartree–Fock layer of `core.x2c_df` and the common input conventions, is in Chapter 16; this section states the scalar layer's energy and entry points:

$$E = \sum_{pq} \gamma_{pq} h_{pq} + \tfrac{1}{2} \sum_{pqrs} \Gamma_{pqrs} (pq|rs) + E_{\mathrm{nuc}}, \qquad (pq|rs) \approx \sum_{P} w_{pqP} w_{rsP}, \tag{14.7}$$

with h the spin-free Foldy–Wouthuysen one-electron Hamiltonian, $w = BS^{-1/2}$ and no picture-change correction on the Coulomb operator. The wavefunction is a real spin-adapted determinant expansion; the orbitals carry no spin-orbit coupling.

Entry point	Returns and purpose
`df_x2c_casscf(...)`	A `DFX2CCASSCFResult` with the converged orbitals, the CI vectors, the state energies, the state-averaged energy and the orbital-gradient norm. Required arguments `(mol, ncore, nact, nele)`; optional auxiliary basis, `nroots`, `weights`, `target`, convergence controls, `theta0` and `rhf`. `nroots=1` is the single-state calculation; `weights` default to equal weights.
`df_x2c_casscf_gradient(...)`	`(grad, parts)` for one CI root, shape `(natm, 3)` in Eh/bohr, with the one-electron, two-electron, Pulay and nuclear contributions separated. Arguments `(mol, result, state=None)`; pass the result object, which carries the CAS space definition and the factor derivatives.
`df_x2c_energies_at(...)`	The state energies at a new geometry with the rotation parameters held fixed; arguments `(mol, result, theta=None)`. The reference orbitals follow the geometry through a displaced SCF and are aligned to `result.c_ref`.
`df_x2c_sa_state_gradient(...)`	The state-specific gradient of a state-averaged calculation; requires `result.nroots >= 2`. Arguments `(mol, result, state=None)`, with finite-difference steps `fd_step` and `theta_step`, a pseudo-inverse cutoff `rcond`, and `return_info`. Returns `(grad, info)` when `return_info` is true.
`df_x2c_rhf(...)`	The closed-shell reference SCF on the same Hamiltonian; arguments `(mol, factors, auxmol, auxbasis, conv_tol, max_cycle, dm0)`.
`sfx2c_hcore(mol)`, `spatial_df_factors(mol, ...)`	The Hamiltonian and the spatial RI factors.

At a converged CASSCF point the orbital and CI responses vanish, so the single-state gradient is the direct derivative at fixed orbitals and CI vector: the one-electron term uses the spin-free X2C hcore derivative, the two-electron term uses the three-center and metric derivatives of Chapter 12, and the Pulay term uses $-\sum_{pq} F_{pq}\,\partial S_{pq}/\partial R$.

A state-averaged calculation is different: the orbitals minimize E_{SA}, so the target-state orbital gradient g_t does not vanish. The state-specific gradient is

$$\frac{dE_t}{dR} = \left.\frac{dE_t}{dR}\right|_{\theta} + z^T M, \qquad Hz = -g_t, \qquad H = \frac{\partial^2 E_{\mathrm{SA}}}{\partial \theta^2}, \tag{14.8}$$

with $M = \partial g_{\mathrm{SA}}/\partial R$. Both H and M are evaluated by finite differences of the analytic state-averaged orbital gradient: H over θ (the CI vectors are re-diagonalized at every step, so the CI response is included) and M over displaced geometries, rebuilding the integrals and the geometry-dependent reference orbitals. The direct term is a central difference of the target-state energy along the same envelope path; the closed-form single-state formula does not apply because the target state is not stationary at the SA orbitals.

The single-state energy and gradient are validated on H_2O/6-31G (6-31G orbital basis, `def2-universal-jkfit`, CAS(2,2), four core spatial orbitals) against a density-fitted sfx2c1e CASSCF in pyscf: the energy agrees within 10^{-5} Eh, the gradient within 10^{-4} relative, the envelope finite difference within 5×10^{-6} Eh/bohr, and the state-averaged state gradients satisfy the weighted sum rule to 10^{-5} Eh/bohr. The layer is scalar (spin-free): full-electron basis sets only, because the spin-free X2C helper raises `NotImplementedError` for ECP molecules. Complex two-component CI, Kramers-restricted response and ECP support are not implemented. The CI, RDM and Fock contractions are dense in the spin-orbital space (`nso`^{4} for the 2-RDM),

so the prototype is sized for small validation systems; production f-block active spaces need a factorized 2-RDM route.

```
from diffmrc.core import x2c_casscf_df as xc

res = xc.df_x2c_casscf(mol, ncore=4, nact=2, nele=2,
                       auxbasis='def2-universal-jkfit')
grad, parts = xc.df_x2c_casscf_gradient(mol, res)
e_states = xc.df_x2c_energies_at(mol, res)

res_sa = xc.df_x2c_casscf(mol, ncore=4, nact=2, nele=2, nroots=2,
                          weights=[0.5, 0.5],
                          auxbasis='def2-universal-jkfit')
g_state, info = xc.df_x2c_sa_state_gradient(
    mol, res_sa, state=0, return_info=True)
```

14.6 Validation scope and boundaries

The table collects the validated systems and the boundaries of the five layers. A system absent from the second column is not validated, and the absence is a statement about coverage, not about expected accuracy.

Module	Validated on	Not covered
`nevpt2_zvector`	H_2O/6-31G, LiH/6-31G, CAS(2,2), closed shell, frozen core	open shell, ECP, f block, large basis; the exact treatment covers the `Si` class only
`caspt2` coupled	H_2O/6-31G and CO/6-31G, CAS(2,2), against the reference class table	differentiable coupled path; large active spaces
`somf_ecp`	GeF with a spin-orbit ECP (splitting and gradient); YbF `ECPso` norms; operator and response identities	production absolute SOC; orbital and CI response; elements outside the fitted spin-orbit ECP sets
`ri_scf`, `ri_scf_driver`	H_2O, CO, OH ROHF and UHF (energies and explicit gradients), OF ROHF, HI/def2-svp with an ECP (gradient)	fractional occupations; RI-JK; production COSX
`x2c_casscf_df`	H_2O/6-31G, CAS(2,2) and SA(2), against density-fitted sfx2c1e CASSCF	ECP; complex two-component CI; Kramers response; large active spaces (see Chapter 16)
`x2c_df`	H_2O/6-31G two-component HF, against pyscf `SpinorX2CHelper` and exact-ERI X2C UHF	multireference response; Kramers-restricted CASSCF; ECP (see Chapter 16)

Chapter 15

CASPT2 class structure and contraction paths

15.1 Purpose and scope

The perturbation layer of the release is a contracted second-order treatment of dynamic correlation on a CASSCF reference. Its energy is assembled from eight excitation classes, and the class table is part of the result rather than a diagnostic: the total is meaningful only when the classes that carry the reference's correlation are present, and the comparison with an external program is meaningful only when the two programs use the same class definitions.

The chapter applies to a CASSCF reference whose orbitals are semicanonical, whose CI vector is state-specific, and whose active space is small enough for the determinant expansion. It does not cover a state-averaged reference, a reference built from a CASCI calculation alone, or a production-size active space. The coupled and three-center paths are value layers; only the diagonal path has a JAX twin.

15.2 Theory and derivation

15.2.1 Reference, excitation operators, and the first-order wavefunction

Let $|0\rangle$ be the state-specific CASSCF reference,

$$|0\rangle = \sum_I c_I \, |I\rangle, \tag{15.1}$$

and let

$$E_{pq} = \sum_{\sigma \in \{\uparrow,\downarrow\}} a^\dagger_{p\sigma} a_{q\sigma} \tag{15.2}$$

be the spin-summed excitation operator. A contracted class function is a product of two such operators acting on the reference,

$$|K_p\rangle = O_p|0\rangle, \qquad O_p = E_{p_1 q_1} E_{p_2 q_2}, \tag{15.3}$$

with the index pairs drawn from the core, active, and virtual pools. The first-order wavefunction is the internally contracted expansion

$$|1\rangle = \sum_p x_p \, |K_p\rangle, \tag{15.4}$$

and the second-order energy is

$$e_2 = \langle 0|H|1\rangle = \sum_p W_p \, x_p, \qquad W_p = \langle 0|H|K_p\rangle. \tag{15.5}$$

The functions are orthogonal to the reference for every excitation class, $\langle 0|K_p\rangle = 0$, so no overlap with $|0\rangle$ enters the energy expression.

15.2.2 The projected amplitude equation

Projecting the Schrödinger equation $(H - E_0)|0 + 1\rangle = 0$ onto the class space gives

$$\sum_q \langle K_p|(H_0 - E_0)|K_q\rangle\, x_q = -\langle K_p|H|0\rangle, \tag{15.6}$$

or, in matrix form,

$$(A - E_0 S)\, x = -W, \qquad A_{pq} = \langle K_p|H_0|K_q\rangle, \qquad S_{pq} = \langle K_p|K_q\rangle. \tag{15.7}$$

The energy is $e_2 = W^T x$, and the class decomposition is $e_{2,\mathrm{class}} = \sum_{p\in\mathrm{class}} W_p x_p$.

Two choices of H_0 are implemented. The diagonal-Fock choice is

$$H_0^{\mathrm{diag}} = \sum_p \epsilon_p E_{pp}, \qquad E_0 = \sum_p n_p^0 \epsilon_p, \tag{15.8}$$

where ϵ_p is the semicanonical Fock diagonal and n_p^0 is the reference occupation. The state-specific generalized-Fock choice is

$$H_0 = \sum_{pq} f_{pq} E_{pq}, \qquad f_{pq} = h_{pq} + \sum_{rs} D_{rs}\left[(pq|rs) - \tfrac{1}{2}(pr|qs)\right], \tag{15.9}$$

with D the state 1-RDM and $E_0 = \sum_{pq} f_{pq} D_{pq}$ (the state 1-RDM is symmetric). The diagonal of f is the semicanonical orbital-energy array when the reference orbitals are semicanonical; its off-diagonal blocks connect determinants of different excitation classes. The diagonal choice drops those blocks; the coupled choice keeps them.

15.2.3 The diagonal approximation

If both A and S are approximated by their diagonal elements, the projected system decouples:

$$x_p = -\frac{W_p}{A_{pp} - E_0 S_{pp}}, \qquad e_2 = -\sum_p \frac{W_p^2}{A_{pp} - E_0 S_{pp}}. \tag{15.10}$$

For a single contracted function K the quantities are

$$A_{KK} = \langle K|H_0^{\mathrm{diag}}|K\rangle = \sum_p n_p(K)\, \epsilon_p, \qquad S_{KK} = \langle K|K\rangle, \tag{15.11}$$

where $n_p(K)$ is the diagonal occupation of orbital p in the contracted function. This is the CASPT2-D approximation: the off-diagonal Fock coupling between functions is neglected.

The diagonal reading is not unique when the class overlap has degenerate eigenvalues. The implementation builds the plain-product basis $E(p_1q_1)E(p_2q_2)|0\rangle$, diagonalizes

$$S = U\,\mathrm{diag}(\lambda)\,U^T, \tag{15.12}$$

keeps eigenvectors above $\lambda > \texttt{nthresh}\cdot\lambda_{\max}$, and evaluates

$$e_2 = -\sum_k \frac{(T^T W)_k^2}{(T^T A T)_{kk}}, \qquad T = U\,\mathrm{diag}(\lambda^{-1/2}), \tag{15.13}$$

in the orthonormalized basis. The option `class_diagonal='block'` instead solves Eq. (15.7) inside the retained subspace,

$$(A - E_0 S)\, x = -W, \qquad e_2 = W^T x, \tag{15.14}$$

which includes the intra-class coupling and is invariant under a change of basis inside the class. The option `'function'` is the per-function Molcas VP/VM reading of Eq. (15.10).

15.2.4 The VP/VM pairing

For a double excitation with both pairs non-diagonal, the two orderings of the excitation operators define two components

$$V_P = \text{cross} + \text{direct}, \qquad V_M = \sqrt{3}\,(\text{cross} - \text{direct})\,, \tag{15.15}$$

which is the OpenMolcas `mkrhsh` normalization. The $\sqrt{3}$ multiplies the whole V_M function and cancels in $W^2/(A - E_0 S)$ and in the projected linear solve, so the operator content carried by the code is the relative weight -1 between the two orderings. Both components enter the energy independently for off-diagonal pairs and coincide for diagonal pairs. The V_M component exists only when both pairs are non-diagonal.

15.2.5 The eight classes

The orbital space is partitioned into core (C, `ncore` spatial orbitals), active (A, `nact`), and virtual (V, `nmo-ncore-nact`). A class is labelled by the two orbital pools it excites out of and the two it excites into.

Class	Pool labels	Operator content	Comment
`V0_ijab`	$(C,C) \to (V,V)$	$E(a,i)E(b,j)$	two core electrons to two virtuals
`Vm1_iab`	$(A,C) \to (V,V)$	$E(a,t)E(b,i)$	one active, one core to two virtuals
`Vm2_ab`	$(A,A) \to (V,V)$	$E(a,t)E(b,u)$	two active electrons to two virtuals
`V1_ija`	$(C,C) \to (A,V)$	$E(t,i)E(a,j)$	two core electrons to one active and one virtual
`V2_ij`	$(C,C) \to (A,A)$	$E(t,i)E(u,j)$	two core electrons to two active orbitals
`V0_ia`	$(C) \to (V)$, active pair	cross $E(v,u)E(a,j)$, direct $E(v,j)E(a,u)$	one inactive hole and one virtual particle dressed by an active-pair reorganization; the two components coincide for $u = v$
`Vm1_a`	TUVA	$E(a,t)E(u,v)$	three active indices, a virtual, t,u,v active
`V1_i`	ITUV	$E(t,i)E(u,v)$	three active indices, i core, t,u,v active

The two three-active-index classes are not single excitations. They were single-excitation functions before version 0.3.2; the release treats them as the plain products shown in the table.

15.2.6 The three-active-index classes

The products $E(a,t)E(u,v)|0\rangle$ and $E(t,i)E(u,v)|0\rangle$ span a strongly linearly dependent set. At `nthresh`= 10^{-6} the TUVA basis of N_2/6-31G CAS(6,6) has rank 1264 of 1728 functions and the ITUV basis rank 632 of 864, that is, about 73% of the function count. A per-function sum $\sum_i W_i^2/(A_{ii} - E_0 S_{ii})$ therefore overcounts the same direction by roughly a factor of three. Both classes are evaluated by the orthonormalized diagonal of Eq. (15.13) with the deterministic gauge described above.

On the CAS(2,2) anchors the two classes are below 10^{-9} Eh. On N_2 they contribute -0.003648 Eh (`Vm1_a`) and -0.001625 Eh (`V1_i`), and the default-path total moves from 0.916 to 0.989 of the Wave 12 frozen ORCA CASPT2-D reference. Against the release reference measured with a single BLAS thread the default-path ratio is 0.983; the difference between the two reference sets is a property of the reference calculation, not of the class treatment.

A residual remains, and it is a class-space difference rather than a numerical one. `V1_ija` stands at 1.553 of the ORCA CASPT2-D value in every basis, because the diagonal approximation of that class is gauge dependent (random gauges inside the degenerate overlap blocks span 0.88 to

1.29 of the reference) while its exact class-block value is gauge invariant at 1.553. ORCA prints 192 functions for the class against the 768 of the plain-product basis, so the two programs are contracting different class spaces.

15.2.7 The coupled full-Fock system

The coupled path keeps the off-diagonal blocks of Eq. (15.9). The amplitudes solve Eq. (15.7) with the state-specific H_0, and the class decomposition is the same $e_{2,\mathrm{class}} = \sum W_p x_p$.

The overlap matrix is regularly linearly dependent. It is diagonalized and eigenvectors with eigenvalue below `rtol` $\times \max(1, \lambda_{\max})$ are dropped; the default `rtol` is 10^{-8}. The assembly costs n_{fn}^2 function-pair contractions plus one H_0 application per function. It is a NumPy pilot intended for small and medium active spaces, and it has no JAX twin. The differentiable production route remains the diagonal-Fock `caspt2_contracted_e2_jax` at the CASPT2-D level.

15.2.8 The three-center contraction

`core.caspt2_three_center` replaces the four-center MO tensor by a three-center factor

$$(ab|cd) \approx \sum_P u_{abP}\, u_{cdP}, \tag{15.16}$$

with the metric factor already folded into u, as returned by `ri_eri_mo` in `core.ri` with `full=False`. To derive the coupling assembly, expand

$$g(D) = \sum_J c_J \langle J|H|D\rangle = g_h(D) + \sum_{abcd} \eta_{abcd}(D)\,(ab|cd), \tag{15.17}$$

where $\eta(D)$ collects the Slater–Condon coefficients of the ERI terms. Substituting Eq. (15.16),

$$g(D) = g_h(D) + \sum_{abP} \left(\sum_{cd} \eta_{abcd}(D)\, u_{cdP} \right) u_{abP}. \tag{15.18}$$

The inner sum is the pair-density tensor $Z_{abP}(D) = \sum_{cd} \eta_{abcd}(D) u_{cdP}$, and the coupling of a contracted function is one contraction $W = \sum_{abP} Z_{abP} u_{abP}$ plus the one-electron terms. Neither the four-center MO tensor nor the spin-orbital tensor is formed: the working set is $n_{\mathrm{mo}}^2 n_{\mathrm{aux}}$ per function, against n_{mo}^4 for the materialized array and $16 n_{\mathrm{mo}}^4$ for the spin-orbital array of the reference path.

The three-center path is a value layer. It is not wired into the JAX chain, and it does not change the approximation level: it computes the same CASPT2-D energy with the RI factorization of the integrals.

15.3 Input and operation

All contraction functions consume the same five objects:

- `h1e_mo`: the one-electron Hamiltonian in the MO basis, shape `(nmo, nmo)`.
- `eri_mo`: the two-electron integrals in chemist notation, shape `(nmo, nmo, nmo, nmo)`; the three-center path replaces this object by `u_mo`.
- `ci`: the state-specific reference CI vector over the determinant list.
- `dets`: determinant bitmaps with the core spin orbitals first, as returned by `cas_dets_for2` in `core.nevpt2.impl` for the arguments `(ncore,nact,nele)`.

- `eps`: the semicanonical Fock diagonal over the same MOs, from `canonicalize_jax`. The SCF orbital energies are not a substitute: both the CASPT2 and the NEVPT2 denominators are defined only in the semicanonical basis.

The reference chain is RHF, CASSCF, state-specific CASCI, then canonicalization with `canonicalize_jax`. The four entry points are

Entry point	Role and defaults
`caspt2_contracted_e2`	diagonal-Fock (CASPT2-D level), NumPy; defaults `nthresh`= 10^{-6}, `active_row='replica'`, `class_diagonal='function'`, `v0_ia='block'`
`caspt2_contracted_e2_jax`	differentiable twin of the diagonal path; same options for `active_row`, traceable in the CI coefficients
`caspt2_contracted_e2_coupled`	state-specific generalized-Fock zeroth order, NumPy; default `rtol`= 10^{-8}, optional `fock` and `return_system`
`caspt2_contracted_e2_three_center`	same eight classes and diagonal denominator, with the coupling from a three-center factor; `u_mo` shape `(nmo,nmo,naux)`

The options change how the class space is represented, not what the class is. `active_row='replica'` drops every (u, u) active row beyond the first active orbital in `Vm2_ab` and `V2_ij`; it is exact only for a two-orbital active space and is ignored by the `ortho` and `block` readings, which enumerate every index tuple. `'keep'` retains all rows. `class_diagonal='function'` is the per-function Molcas pairing, `'ortho'` the orthonormalized diagonal of Eq. (15.13), and `'block'` the projected class solve of Eq. (15.14). `v0_ia='block'` solves one two-component system per (u, v, j, a); `'class'` solves the whole class as one projected system. The three-active-index classes keep the orthonormalized diagonal in every setting.

The coupled call is

```
caspt2_contracted_e2_coupled(h1e_mo, eri_mo, ci, dets, *, ncore, nact,
                             nele, fock=None, rtol=1e-8,
                             return_system=False)
```

With `return_system=True` the function also returns a dictionary with `S`, `A`, `W`, `E0`, `x`, `T`, `labels`, `fock`, `dm1`, and `functions`. The system is the object to inspect when a class value is questioned: the smallest eigenvalues of `S`, the condition number of the reduced matrix, and the projection of `W` on the dropped directions are available without re-running the assembly.

15.4 Worked example

15.4.1 N_2/6-31G CAS(6,6): three paths

The recorded N_2/6-31G CAS(6,6) comparison against the ORCA class table is below. The three rows are the default per-function reading (`class_diagonal='function'`), the class-block reading (`class_diagonal='block'` with `v0_ia='class'`), and the coupled full-Fock path.

Path	Total ratio	Class evidence
default (`function`)	0.989 (Wave 12 frozen), 0.983 (release, one BLAS thread)	apparent agreement is a cancellation: `V1_ija` 1.553 (+55%), `V2_ij` 0.24 (−76%), `V0_ia` +10%, `Vm2_ab` +12%
class-block	1.0232	`Vm2_ab` 1.0000, `V0_ia` 0.9970; the class projections are gauge invariant
coupled full-Fock	1.0232	full-FIC level; diagonal/full coupling ratio 0.9795831 against the ORCA 0.9795829

The ratios are the release record in `README.md` and `CHANGELOG.md`; the runnable driver is `scripts/caspt2_n2_validation.py`, run in three stages:

```
python scripts/caspt2_n2_validation.py --stage ref --outdir out
python scripts/caspt2_n2_validation.py --stage default --outdir out
python scripts/caspt2_n2_validation.py --stage coupled --outdir out
```

The default path's total is close to the reference for the wrong reason: the class residuals have opposite signs and cancel. The block reading and the coupled path agree on the total because both include the intra-class coupling; they differ in what they keep beyond that, and the agreement is the diagnostic that the class-space projection is the missing piece of the default path.

15.4.2 H_2O/6-31G and CO/6-31G: the CAS(2,2) class table

On the two-orbital active space the class-resolved ratios against the ORCA CASPT2 class table are

System	V0_ijab	Vm1_iab	Vm2_ab	V1_ija	V2_ij
H_2O/6-31G	1.017	1.011	0.999	1.009	0.955
CO/6-31G	1.009	1.011	0.963	0.999	0.987

The eight-class totals are 1.0103 (H_2O) and 1.0026 (CO). The H_2O case uses `ncore`= 4 and the CO case `ncore`= 6; both follow the reference chain of the runnable example below. The `V0_ia` target on H_2O is -0.011135 Eh (ORCA). These are the anchors that establish the class definitions before the three-active-index treatment is exercised on N_2.

15.4.3 Running the three contraction routes

```
import numpy as np
import jax
jax.config.update('jax_enable_x64', True)
import jax.numpy as jnp

from pyscf import gto, scf, mcscf
from diffmrc.core.casci import cs_casci
from diffmrc.core.nevpt2.impl import canonicalize_jax, cas_dets_for2
from diffmrc.core.caspt2 import (caspt2_contracted_e2,
                                 caspt2_contracted_e2_coupled)
from diffmrc.core.caspt2_three_center import (
    caspt2_contracted_e2_three_center)
from diffmrc.core import ri

ncore, nact, nele = 4, 2, 2
nocc = ncore + nact

mol = gto.M(atom='O 0 0 0; H 0.7557 0 0.5880; H -0.7557 0 0.5880',
            basis='6-31g', unit='Angstrom', verbose=0)
mf = scf.RHF(mol).run(verbose=0)
mc = mcscf.CASSCF(mf, nact, nele)
mc.kernel()

C = jnp.asarray(mc.mo_coeff)
h1e = jnp.asarray(mol.intor('int1e_kin') + mol.intor('int1e_nuc'))
eri = jnp.asarray(mol.intor('int2e', aosym='s1'))

def mo_ints(C, h1e, eri):
    h1 = jnp.einsum('ai,ab,bj->ij', C, h1e, C)
```

```
    z = jnp.einsum('pi,pqrs->iqrs', C, eri)
    z = jnp.einsum('qj,iqrs->ijrs', C, z)
    z = jnp.einsum('rk,ijrs->ijks', C, z)
    return h1, jnp.einsum('sl,ijks->ijkl', C, z)

nmo = mol.nao_nr()
h1_mo, eri_mo = mo_ints(C, h1e, eri)
_, ci = cs_casci(h1_mo[:nocc, :nocc],
                 eri_mo[:nocc, :nocc, :nocc, :nocc],
                 ncore=ncore, nact=nact, nele_act=nele,
                 nroots=1, return_ci=True)
C2, ci2, eps = canonicalize_jax(C, h1e, eri, ci, ncore=ncore,
                                nact=nact, nele_act=nele)
h1_mo2, eri_mo2 = mo_ints(C2, h1e, eri)
dets = cas_dets_for2(ncore, nact, nele)

e2, breakdown = caspt2_contracted_e2(
    np.asarray(h1_mo2), np.asarray(eri_mo2), np.asarray(ci2), dets,
    ncore=ncore, nact=nact, nele=nele, eps=np.asarray(eps))
for name, value in breakdown.items():
    print(f'{name:10s} {value: .9f}')
print('default', e2)

e2_block, bd_block = caspt2_contracted_e2(
    np.asarray(h1_mo2), np.asarray(eri_mo2), np.asarray(ci2), dets,
    ncore=ncore, nact=nact, nele=nele, eps=np.asarray(eps),
    class_diagonal='block', v0_ia='class')
print('class-block', e2_block)

e2_cpl, bd_cpl, system = caspt2_contracted_e2_coupled(
    np.asarray(h1_mo2), np.asarray(eri_mo2), np.asarray(ci2), dets,
    ncore=ncore, nact=nact, nele=nele, return_system=True)
print('coupled', e2_cpl,
      'min |eig(S)|', np.min(np.abs(np.linalg.eigvalsh(system['S']))))

auxmol, label = ri.make_auxmol(mol)
factors = ri.ri_factors(mol, auxmol)
u_mo, _ = ri.ri_eri_mo(np.asarray(C2), factors.b, factors.jhalf,
                       full=False)
e2_3c, bd_3c = caspt2_contracted_e2_three_center(
    np.asarray(h1_mo2), u_mo, np.asarray(ci2), dets,
    ncore=ncore, nact=nact, nele=nele, eps=np.asarray(eps))
print('three-center', e2_3c,
      'max class difference',
      max(abs(bd_3c[k] - breakdown[k]) for k in breakdown))
```

Three points about the call are easy to get wrong. First, `ci` must be the state-specific CI vector of the same determinant list, not a state-averaged vector and not the natural-orbital CI of a different active-space ordering. Second, `eps` must come from `canonicalize_jax`; passing the RHF orbital energies leaves the denominators undefined. Third, `u_mo` must be the metric-folded factor; the raw `ri.ri_three_center` tensor without `factors.jhalf` changes the energy by the metric factor.

15.5 Pitfalls

- *Reading only the total.* The default N_2 total agrees with the reference through a cancellation between classes with opposite errors. Read the class table and quote the class ratios when

the total is used for a comparison.

- *Comparing a gauge-dependent class.* When the class overlap has degenerate eigenvalues, the diagonal approximation depends on the basis chosen inside the block. Random rotations inside the blocks move a class energy by up to 25% (`V2_ij`, measured). Use `class_diagonal='block'` when the class energy enters a decision.
- *The `active_row` replica assumption.* The default `'replica'` drops (u, u) rows beyond the first active orbital. It is exact only for a two-orbital active space; on larger spaces the rows are not replicas unless the block projection is used.
- *The `V1_ija` class-space residual.* It stands at 1.553 of the ORCA value in every basis and does not shrink with the basis set. This is a definition difference (ORCA prints 192 functions for the class against 768 in the plain-product basis), not a convergence artifact.
- *Using the three-center path as a different method.* It is the same CASPT2-D energy with RI integrals; the RI fitting error enters through `u_mo`.
- *Scaling the coupled path.* The assembly is n_{fn}^2 and is intended for small and medium active spaces; it is not a production-scale implementation.
- *Forgetting the canonical basis.* The denominators are defined in the semicanonical basis. A CASCI reference with SCF orbital energies is not the same calculation.

15.6 Validation and references

Check	System	Reference and result
Diagonal class table	H_2O/6-31G CAS(2,2)	ORCA CASPT2 class table; class ratios 1.017, 1.011, 0.999, 1.009, 0.955; eight-class total 1.0103
Diagonal class table	CO/6-31G CAS(2,2)	class ratios 1.009, 1.011, 0.963, 0.999, 0.987; total 1.0026
Three-active-index classes	N_2/6-31G CAS(6,6)	default-path ratio 0.989 (Wave 12 frozen), 0.983 (release, one BLAS thread); class contributions -0.003648 Eh and -0.001625 Eh
Three paths	N_2/6-31G CAS(6,6)	default 0.989, class-block 1.0232, coupled 1.0232; `Vm2_ab` 1.0000, `V0_ia` 0.9970, `V1_ija` 1.553, `V2_ij` 0.24
Coupled path	H_2O, CO CAS(2,2)	totals 1.0081, 1.0100; N_2 diagonal/full ratio 0.9795831 against 0.9795829
Three-center parity	deterministic exact-factor tensor	class-by-class agreement with the materialized path at 10^{-10}; dense-coupling elementwise check

The files behind the table are `tests/core/test_caspt2.py` (class definitions, JAX twin, option behaviour), `tests/core/test_caspt2_three_center.py` (three-center parity and coupling scatter), `scripts/caspt2_n2_validation.py` (the N_2 reference chain and the ORCA comparison; stages `ref`, `default`, `coupled`), and `scripts/caspt2_sparse_assembly.py` (sparse class assembly).

The class structure and the internally contracted amplitude equations follow the OpenMolcas and ORCA conventions documented in K. Andersson, P.-A. Malmqvist, B. O. Roos, A. J. Sadlej and K. Wolinski, *J. Phys. Chem.* **94**, 5483 (1990); K. Andersson, P.-A. Malmqvist and B. O. Roos, *J. Chem. Phys.* **96**, 1218 (1992); C. Angeli, R. Cimiraglia, S. Evangelisti, T. Leininger and J.-P. Malrieu, *J. Chem. Phys.* **114**, 10252 (2001) and **117**, 9138 (2002); and the class table of D. Kollmar, K. Sivalingam and F. Neese, *J. Chem. Theory Comput.* (2020),

and D. Kollmar, Y. Guo and F. Neese, *J. Chem. Theory Comput.* (2021). The OpenMolcas implementation records are the `mkrhsh` and `caspt2` sources referred to in the module docstrings.

Chapter 16

Relativistic one-electron layers: scalar and two-component X2C

16.1 Purpose and scope

The release carries two density-fitted X2C layers that differ in what they transform and in what wavefunction they support. The scalar layer, `core.x2c_casscf_df`, applies the spin-free Foldy–Wouthuysen one-electron Hamiltonian and runs a real spin-adapted CASSCF on top of it. The two-component layer, `core.x2c_df`, keeps the spinor structure of the X2C transform and runs a complex generalized Hartree–Fock calculation in the contracted spinor basis.

The two are not interchangeable. The scalar layer has orbital and CI response but no spin–orbit coupling; the two-component layer has spin–orbit coupling at the one-electron level and no multireference response. Both use the non-relativistic Coulomb operator, density fitted, with no picture-change correction on the two-electron part. This is one-electron X2C (1eX2C); the separate `core.x2c_sap` module implements the atomic-potential approximation to the missing two-electron picture-change term and is outside the validation record here.

The scope is full-electron basis sets. Both X2C helpers raise `NotImplementedError` for an effective core potential, and the layers inherit that limit. The scalar layer is sized for small and medium active spaces; the two-component layer is a Hartree–Fock layer.

16.2 Theory and derivation

16.2.1 The four-component problem in block form

The Dirac–Coulomb Hamiltonian in a molecular basis, after removing the rest energy, has the block form

$$H_{4c} = \begin{pmatrix} V & T \\ T & \dfrac{W}{4c^2} - T \end{pmatrix}, \qquad S_{4c} = \begin{pmatrix} S & 0 \\ 0 & \dfrac{T}{2c^2} \end{pmatrix}, \tag{16.1}$$

where, in the decontracted spinor basis used by the transform,

$$S = \langle \chi_i | \chi_j \rangle, \quad T = \langle \chi_i | T_{\mathrm{kin}} | \chi_j \rangle, \quad V = \langle \chi_i | V_{\mathrm{nuc}} | \chi_j \rangle, \quad W = \langle \chi_i | \boldsymbol{\sigma} \cdot \mathbf{p}\, V_{\mathrm{nuc}}\, \boldsymbol{\sigma} \cdot \mathbf{p} | \chi_j \rangle. \tag{16.2}$$

The generalized eigenproblem

$$H_{4c}\, C = S_{4c}\, C\, E \tag{16.3}$$

has positive- and negative-energy solutions. Partition the positive-energy eigenvectors into large and small components,

$$C = \begin{pmatrix} C_L \\ C_S \end{pmatrix}, \qquad X = C_S C_L^{-1}. \tag{16.4}$$

The matrix X is the small-to-large ratio of the positive-energy branch. The implementation obtains it from the eigenproblem rather than solving the quadratic equation for X: it builds Eq. (16.1) with the spinor integrals, calls `scipy.linalg.eigh`, selects the positive-energy block, and forms X by Eq. (16.4). This is the construction in `core.x2c_df.X2CTransform`.

16.2.2 Block decoupling and the Foldy–Wouthuysen Hamiltonian

Eliminating the small component with X and retaining terms through $\mathcal{O}(c^{-2})$ gives the effective two-component Hamiltonian

$$S_+ = S + \frac{1}{2c^2} X^\dagger T X, \tag{16.5}$$

$$H_+ = V + TX + (TX)^\dagger - X^\dagger T X + \frac{1}{4c^2} X^\dagger W X. \tag{16.6}$$

The large-component metric is not the original overlap, so a renormalization matrix R is chosen such that

$$R^\dagger S_+ R = S, \qquad H_{2c} = R^\dagger H_+ R. \tag{16.7}$$

The implementation computes R from the symmetric eigendecomposition of S and the transformed metric (the `_metric_renormalization` helper). The result is a complex Hermitian Hamiltonian in the contracted spinor basis. For a closed-shell determinant its spin–orbit content is quenched at first order, but the operator still differs from the spin-free one through the spin-dependent renormalization and the spinor structure of X.

The scalar (spin-free) layer performs the same construction with spin-averaged integrals: the transform is applied to the spatial S, T, V, W blocks, the resulting Hamiltonian is real and acts on spatial orbitals, and the spin–orbit terms are omitted. This is the `SpinFreeX2CHelper` limit used by `core.x2c_casscf_df`.

16.2.3 Why the scalar and two-component energies differ

The two shipped layers are not a controlled one-parameter comparison: the scalar layer is CASSCF and the two-component layer is HF. The X2C-specific difference can be isolated only at the same wavefunction level. At that level there are three sources:

- the spin-free transform drops the spin-dependent part of the renormalization and of the picture-change matrix;
- the two-component transform retains the spin–orbit operator, whose expectation value vanishes for a non-degenerate closed-shell determinant but whose second-order and response contributions do not;
- the two-electron treatment differs when the scalar layer uses spatial DF factors and the two-component layer uses spinor RI-JK factors, even before correlation is added.

For a closed-shell singlet the first source usually dominates; for an open-shell or degenerate system the second can dominate. A comparison that does not hold the wavefunction method and the two-electron operator fixed measures all three together.

16.2.4 The two-electron part and picture change

Both layers use the non-relativistic Coulomb interaction,

$$(pq|rs) = \iint \frac{\phi_p^*(\mathbf{r})\phi_q(\mathbf{r})\,\phi_r^*(\mathbf{r}')\phi_s(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|}\, d\mathbf{r}\, d\mathbf{r}', \tag{16.8}$$

density fitted as in Chapter 12. The X2C transformation of the one-electron operator induces a picture-change correction on the two-electron operator; it is not applied here. The missing term is the dominant residual of 1eX2C for heavy elements, and the SAP-X2C module is the separate approximation to it.

16.2.5 The SOMF operator

The Breit–Pauli spin–orbit operator for electron i is

$$H_{\mathrm{SO}} = \frac{\alpha^2}{2} \sum_i \left[\sum_A \frac{Z_A}{r_{iA}^3} \left(\mathbf{r}_{iA} \times \mathbf{p}_i\right) \cdot \mathbf{s}_i - \sum_{j \neq i} \frac{1}{r_{ij}^3} \left(\mathbf{r}_{ij} \times \mathbf{p}_i\right) \cdot \left(\mathbf{s}_i + 2\mathbf{s}_j\right) \right], \tag{16.9}$$

with $\mathbf{r}_{iA} = \mathbf{r}_i - \mathbf{R}_A$, $\mathbf{r}_{ij} = \mathbf{r}_i - \mathbf{r}_j$, and α the fine-structure constant. The first term is the nuclear one-electron SOC; the second contains spin-same-orbit and spin-other-orbit contributions.

The spin-orbit mean-field (SOMF) approximation replaces the two-electron term by its contraction with the reference density P,

$$h_k^{2e}[pq] = \sum_{rs} P_{rs} \left[(pq|g_k|rs) - \frac{3}{2}(pr|g_k|sq) - \frac{3}{2}(sq|g_k|pr) \right], \tag{16.10}$$

with the geometric kernel

$$g_k(\mathbf{r}_i, \mathbf{r}_j) = \frac{(\mathbf{r}_{ij} \times \mathbf{p}_i)_k}{r_{ij}^3}, \tag{16.11}$$

in the raw `int2e_p1vxp1` convention. The global sign of the raw integral is the same as that of `int1e_pnucxp`; it is a convention and cancels in the ratio tests. The coefficients 1 and $-3/2$ are the spin-summed spin-same-orbit and spin-other-orbit combinations. The factor $\alpha^2/2$ is applied once, in Eq. (16.13): the `int2e_p1vxp1` integral carries the geometric kernel without it, and the raw one-electron integral `int1e_pnucxp` likewise carries no prefactor. The `ECPso` integral is already an effective operator in energy units and receives no $\alpha^2/2$. The one-electron nuclear term is

$$h_k^{\mathrm{nuc}}[pq] = \sum_A Z_A \left\langle p \left| \frac{(\mathbf{r}_{iA} \times \mathbf{p}_i)_k}{r_{iA}^3} \right| q \right\rangle, \tag{16.12}$$

where the sum runs over all centers for the all-electron operator; in the ECP variant of Eq. (16.14) it runs only over centers without a spin–orbit ECP. The SOMF operator is

$$h_k^{\mathrm{SOMF}} = \frac{\alpha^2}{2} \left[h_k^{\mathrm{nuc}} + h_k^{2e}[P] \right]. \tag{16.13}$$

The two-electron kernel is not symmetric under the pair exchange $(pq) \leftrightarrow (rs)$, so the derivative of $\mathrm{tr}(D\, h^{2e}[P])$ with respect to P is not $h^{2e}[D]$; it is the separate contraction in `somf_density_response`. A gradient of an SOC matrix element must use that response, or the equivalent bilinear form.

16.2.6 The ECP spin–orbit term

With an effective core potential, the core electrons are absent from the calculation and the spin–orbit part of the ECP replaces their contribution. On a center carrying a spin–orbit ECP the valence-level operator is

$$h_k^{\text{SOMF-ECP}} = \frac{\alpha^2}{2} \left[h_k^{\mathrm{nuc}} + h_k^{2e}[P] \right] - V_k^{\mathrm{ECP}}, \tag{16.14}$$

where h_k^{nuc} is kept only on centers without a spin–orbit ECP, $h_k^{2e}[P]$ is the valence $J - \frac{3}{2}K$ mean field of the spin-summed density P with $\mathrm{tr}(PS) = N_{\mathrm{elec}}$, and V_k^{ECP} is the three-component `ECPso` integral. On a center carrying a spin–orbit ECP the ECP term replaces the nuclear term: the ECP is fitted to reproduce the all-electron spin–orbit splitting of the valence configuration and already contains the nuclear one-electron term and the core–valence screening for that center. The Breit–Pauli terms carry $\alpha^2/2$; `ECPso` is already an effective operator in energy units and is subtracted.

There is no unique first-principles decomposition of an ECP into one- and two-electron spin–orbit parts. The ECP is fitted to atomic reference states, so part of the valence–valence mean field is already folded into its parameters. Adding the explicit valence $J - \frac{3}{2}K$ term is therefore an approximation with a double-counting error that cannot be removed without the ECP generation data. The pieces are exposed independently (`nuclear=`, `ecp_term=`) so variants can be compared.

16.3 Input and operation

16.3.1 Common conventions

- `mol` is a built `pyscf.gto.Mole`. The scalar layer expects the spatial orbital basis; the two-component layer builds its own decontracted spinor molecule internally through `SpinorX2CHelper` and never asks the caller for a spinor basis.
- Energies are in Eh. Nuclear gradients are in Eh/Bohr. Atom coordinates may be supplied in Angstrom or Bohr through the usual pyscf `unit` argument; the gradient is always Eh/Bohr.
- Only full-electron basis sets are accepted. Both X2C helpers raise `NotImplementedError` for a molecule that carries an effective core potential.
- The auxiliary basis defaults to `def2-universal-jkfit`, resolved by `core.ri.make_auxmol`, which falls back to the orbital basis when a fitting set does not cover every element. The label actually used is `result.auxbasis` for the scalar result and `result.factors.auxbasis` for the two-component result; it should be quoted with any number.
- The scalar layer's `ncore`, `nact`, and `nele` are spatial-orbital and electron counts, not spinor counts. The two-component layer has no active-space interface.

A minimal skeleton for both layers is

```
import numpy as np
from pyscf import gto
from diffmrc.core import x2c_casscf_df as xc
from diffmrc.core import x2c_df

mol = gto.M(atom='O 0 0 0; H 0 -0.757 0.587; H 0 0.757 0.587',
            basis='6-31g', unit='Angstrom', verbose=0)

# Scalar (spin-free) CASSCF, CAS(2,2), four core spatial orbitals.
res = xc.df_x2c_casscf(mol, ncore=4, nact=2, nele=2,
                       auxbasis='def2-universal-jkfit')
grad, parts = xc.df_x2c_casscf_gradient(mol, res)

# Two-component Hartree-Fock on the same molecule.
h = x2c_df.df_x2c_hcore(mol)
hf = x2c_df.df_x2c_hf(mol, auxbasis='def2-universal-jkfit')
g_hf, parts_hf = x2c_df.df_x2c_hf_gradient(mol, result=hf)
```

16.3.2 Scalar DF-X2C CASSCF

The energy is

$$E = \sum_{pq} \gamma_{pq} h_{pq} + \tfrac{1}{2} \sum_{pqrs} \Gamma_{pqrs} (pq|rs) + E_{\mathrm{nuc}}, \qquad h = \texttt{SpinFreeX2CHelper}(mol).\texttt{get_hcore}(mol), \tag{16.15}$$

with the density-fitted Coulomb operator of Eq. (16.8). The wavefunction is a real spin-adapted determinant expansion over the active spin orbitals.

Entry point	Returns and purpose
`sfx2c_hcore(mol)`	Spin-free X2C one-electron Hamiltonian in the spatial AO basis.
`sfx2c_hcore_derivatives(mol)`	Its nuclear derivatives, shape `(natm, 3, nao, nao)`.
`spatial_df_factors(mol, auxmol=None, auxbasis=None, rcond=1e-12)`	`SpatialDFFactors` with `b`, `s`, `jhalf`, the metric eigendecomposition, the auxiliary label, and the factor derivatives.
`df_x2c_rhf(mol, factors=None, auxmol=None, auxbasis=None, conv_tol=1e-12, max_cycle=200, dm0=None)`	Closed-shell reference SCF on the same Hamiltonian.
`df_x2c_casscf(mol, ncore, nact, nele, auxmol=None, auxbasis=None, nroots=1, weights=None, target=0, max_cycle=300, g_conv=1e-6, rcond=1e-12, theta0=None, rhf=None, verbose=False)`	`DFX2CCASSCFResult` with orbitals, CI vectors, state energies, state-averaged energy, and orbital-gradient norm.
`df_x2c_casscf_gradient(mol, result=None, state=None, auxmol=None, auxbasis=None, rcond=1e-12)`	`(grad, parts)` for one CI root, shape `(natm, 3)` in Eh/Bohr; parts are one-electron, two-electron, Pulay, nuclear.
`df_x2c_energies_at(mol, result, theta=None)`	State energies at a new geometry with the rotation parameters fixed; the reference orbitals follow the geometry through a displaced SCF aligned to `result.c_ref`.
`df_x2c_sa_state_gradient(mol, result=None, state=None, fd_step=1e-3, theta_step=1e-5, rcond=1e-8, auxmol=None, auxbasis=None, return_info=False)`	State-specific gradient of a state-averaged calculation; requires `result.nroots >= 2`.

`nroots=1` is the single-state calculation. With `nroots>1`, `weights` default to equal weights and `target` selects the state reported in `result.e_tot`. The state-averaged energy is `result.e_sa`; the individual energies are `result.state_energies`. The `converged` flag is `gnorm < 1e-5`; the energy and the nuclear gradient are insensitive to the residual along flat orbital directions through the envelope theorem.

The single-state gradient is

$$\frac{dE}{dR} = \sum_{pq} \gamma_{pq} \frac{dh_{pq}}{dR} + \sum_{P} \sum_{pq} \frac{dw_{pqP}}{dR} X_{pqP} - \sum_{pq} F_{pq} \frac{dS_{pq}}{dR} + \frac{dE_{\mathrm{nuc}}}{dR}, \qquad X_{pqP} = \sum_{rs} w_{rsP} \Gamma_{pqrs}. \tag{16.16}$$

The one-electron derivative comes from pyscf's spin-free X2C hcore gradient generator; the three-center and metric derivatives reuse Chapter 12. The gradient entry point requires the converged `result`.

A state-averaged calculation is different: the orbitals minimize E_{SA}, so the target-state orbital gradient g_t does not vanish, and

$$\frac{dE_t}{dR} = \left.\frac{dE_t}{dR}\right|_{\theta} + z^T M, \qquad Hz = -g_t, \qquad H = \frac{\partial^2 E_{\mathrm{SA}}}{\partial \theta^2}, \tag{16.17}$$

with $M = \partial g_{\mathrm{SA}} / \partial R$. Both H and M are finite-difference constructions around the analytic state-averaged orbital gradient: H over θ (the CI vectors are re-diagonalized at every step, so the CI response is included) and M over displaced geometries, rebuilding the integrals and the geometry-dependent reference orbitals. The direct term is a central difference of the target-state energy along the envelope path; the closed-form single-state formula does not apply because the target state is not stationary at the SA orbitals. The Hessian is pseudo-inverted with a relative cutoff; the retained directions are reported in `info`.

After the quasi-Newton stage the scalar DF-X2C CASSCF refines the point with a Newton polish. The orbital Hessian is built by central differences of the analytic orbital gradient (two gradient evaluations per parameter, so the stage is skipped above 256 parameters), pseudo-inverted with a relative cutoff of 10^{-10}, and the step is capped at 0.3 rad; a step is accepted only if the gradient strictly decreases and the energy does not rise. On H_2O/6-31G the final orbital gradient falls from 2.1×10^{-6} to 4.3×10^{-12} in two steps, the envelope finite-difference relative error from 1.4×10^{-4} to 1.6×10^{-5}, and the rotation invariant from 1.7×10^{-6} to 5.6×10^{-10}; the energy sits 1.1×10^{-11} Ha from pyscf. No test threshold was relaxed, and `polish=False` keeps the previous behaviour.

16.3.3 Two-component DF-X2C Hartree–Fock

The two-component layer solves

$$E = \mathrm{tr}(DH_{\mathrm{X2C}}) + \tfrac{1}{2}\mathrm{tr}(DJ) - \tfrac{1}{2}\mathrm{tr}(DK) + E_{\mathrm{nuc}}, \tag{16.18}$$

with the RI-JK factorization

$$(ij|kl) \approx \sum_P w_{ijP} w_{klP}, \qquad w = BS^{-1/2}, \qquad B_{ijP} = (ij|P), \tag{16.19}$$

built directly with `int3c2e_spinor`. The one-electron operator is Eq. (16.7) from the spinor integrals. The values agree with pyscf's `SpinorX2CHelper` to 4.5×10^{-11} on H_2O/6-31G (test bound 10^{-9} absolute and 10^{-10} relative).

The spinor ERI convention is the complex-GHF one,

$$J_{ij} = \sum_{kl} E_{ijkl} D_{lk}, \qquad K_{ij} = \sum_{kl} E_{ikjl} D_{kl}, \tag{16.20}$$

which for $w = BS^{-1/2}$ becomes

$$\rho_P = \sum_{ij} w_{ijP} D_{ji}, \qquad J = \sum_P w_P \rho_P, \qquad K = \sum_P w_P D w_P. \tag{16.21}$$

The three-center tensor is Hermitian in the AO pair, $B_{ijP} = \overline{B_{jiP}}$, so the derivative with respect to the second AO center carries a complex conjugate. The conjugate was verified by finite differences of B at 2.5×10^{-9} relative on H_2O/6-31G.

Entry point	Returns and purpose
`df_x2c_hcore(mol, C=None, c=LIGHT_SPEED, return_deriv=False, step=1e-4)`	Complex Hermitian two-component Hamiltonian, optionally in a supplied basis and optionally with `dH/dR`, shape `(natm, 3, n, n)`.
`X2CTransform(mol, c=LIGHT_SPEED)`	Transform at one geometry: decontracted spinor molecule, spinor contraction matrix, and the four input integrals.
`spinor_df_factors(mol, auxmol=None, auxbasis=None, rcond=1e-12)`	`SpinorDFFactors` with `b`, `s`, `jhalf`, metric eigendecomposition, and factor derivatives.
`df_x2c_hf(mol, auxmol=None, auxbasis=None, dm0=None, conv_tol=1e-11, max_cycle=200, rcond=1e-12, verbose=False)`	`DFX2CHFResult` with spinor density, orbitals, orbital energies, occupations, Fock matrix, energy parts, and gradient norm.
`df_x2c_hf_gradient(mol, result=None, auxmol=None, auxbasis=None, dm0=None, transform_step=1e-4, rcond=1e-12, verbose=False)`	`(grad, parts)` for the converged DF-X2C HF energy.
`energy_weighted_density(result)`	$W = C_{\mathrm{occ}}\mathrm{diag}(\epsilon_i n_i) C_{\mathrm{occ}}^{\dagger}$.
`spinor_overlap_derivatives(mol)`	dS/dR of the contracted spinor overlap.

At a converged SCF the density response collapses to the Pulay term:

$$\frac{dE}{dR} = \sum_{ij} D_{ij} \frac{dH_{ji}}{dR} + \left.\frac{dE_{2e}}{dR}\right|_D - \sum_{ij} W_{ij} \frac{dS_{ji}}{dR} + \frac{dE_{\mathrm{nuc}}}{dR}. \tag{16.22}$$

With $G_P = \rho_P D^T - (D w_P D)^T$,

$$dE_{2e} = \sum_{ijP} G_{ijP}\, dw_{ijP}, \qquad dw = dB S^{-1/2} + B\, dS^{-1/2}, \tag{16.23}$$

and the metric inverse-square-root derivative uses the Daleckii–Krein form of Chapter 12. The X2C transform derivative combines analytic nuclear derivatives of the four input integrals, the implicit eigenbasis derivative of the picture-change matrix (a complement-space sum safe at the exact Kramers degeneracy), and a transform-layer central difference at 10^{-4} Bohr for the polynomial and matrix-square-root chain; the development probes measure its error at about 10^{-7} relative on H_2O.

16.4 Worked example

16.4.1 OH and OF: one-electron versus SOMF spin–orbit splitting

The SOMF validation on OH and OF uses two state-averaged roots, the same geometry and active space for the stack and ORCA 6.1.1, and the SI convention of this stack. The recorded numbers are

System	one-electron (ours)	SOMF (ours)	ORCA one-electron	ORCA SOMF
OH	206.52	129.23	205.88	128.83
OF	273.06	176.77	272.06	176.12

in cm^{-1}. The screening ratio SOMF/one-electron removes the overall one-electron convention offset between the codes. The recorded ratios are 0.625738 (OH) and 0.647358 (OF), against ORCA 0.625753 and 0.647357; the scale-corrected SOMF values are 128.827 and 176.120 cm^{-1}, with deviations -0.0030 cm^{-1} (-2.35×10^{-5} relative) and $+0.00024$ cm^{-1} (1.34×10^{-6} relative). The source file is `experiments/somf/results_somf.json`; the driver is `experiments/somf/validate_oh_of.py`.

The one-electron column is not the physical SOC; it omits the two-electron screening. The SOMF column is the mean-field approximation to that screening. The near-perfect ratio agreement is the evidence that the two-electron contraction has the right shape, not that the absolute operator is exact.

16.4.2 GeF with an ECP: the ECP spin–orbit operator

The ECP validation on GeF uses `lanl2dz` on Ge and F, the spin–orbit ECP `ecpds28mdfso` on Ge, and a two-state state-averaged calculation. The recorded valence-level SOMF splitting is 28.37 cm^{-1}, against 43.72 cm^{-1} for the nuclear-only operator on the same density. The difference is the valence mean field and is the quantity the ECP decomposition does not fix uniquely. On YbF the `ECPso` norm on Yb is 17.04 per component, while the raw nuclear operator norm on the same center is of order 10^4; adding both would be dominated by an operator that the ECP was built to replace. The gradient prototype on GeF follows a finite difference of the SOC matrix element to 2.4×10^{-5} relative, with a density-response term of 4.4×10^{-7} in that direction. The source files are `experiments/somf_ecp/results_compare_orca_gef.json` and `experiments/somf_ecp/results_somf_grad_f.json`.

16.4.3 H_2O/6-31G: two-component X2C acceptance bounds

The two-component DF-X2C HF layer on H_2O/6-31G with `def2-universal-jkfit` is checked against pyscf's `SpinorX2CHelper` and exact-ERI X2C UHF. The recorded acceptance bounds are

Check	Bound
hcore against `SpinorX2CHelper`	max $< 10^{-9}$, relative $< 10^{-10}$
DF SCF against exact X2C UHF	$< 10^{-4}$ Eh (the fit)
gradient against 3-point differences	relative $< 2 \times 10^{-5}$
gradient against 5-point differences	relative $< 2 \times 10^{-6}$
translation and rotation invariance	relative $< 10^{-6}$
factor derivatives against geometry FD	relative $< 10^{-6}$

These are the release's recorded criteria; the raw gradient components are not stored in the release tree. A requested XeH^+ three-direction gradient record is also not part of this tree; the XeH^+ file that is present (`tests/core/test_x2c_sap_heavy.py`) checks only that the SAP-X2C energy moves toward the four-component DHF reference and is gated behind `DIFFMRC_X2C_SAP_HEAVY=1`. The source files for the table are `tests/core/test_x2c_df.py` and `experiments/df_x2c_gradient_validation.py`.

16.5 Pitfalls

16.5.1 Scalar layer

- Effective core potentials are not supported. The spin-free X2C helper raises `NotImplementedError` for an ECP molecule, and the layer inherits that limit.
- The CI, RDM, and Fock contractions are dense in the spin-orbital space; the 2-RDM is an n_{so}^4 object. The prototype is sized for the validation systems. Production f-block active spaces need a factorized 2-RDM route.
- The complex two-component CI, complex orbital rotations, and the Kramers-restricted response are designed but not implemented.
- The state-averaged response is a finite-difference construction of H and M around the analytic orbital gradient. Its cost grows with the number of rotation parameters and displaced geometries.
- The displaced-geometry reference has a second orbital minimum on H_2O in the same θ space. `df_x2c_energies_at` follows the envelope path; re-optimizing at the displaced geometry is a different, branch-sensitive path and is not the reference for the analytic gradient.

16.5.2 Two-component layer

- The layer is Hartree–Fock. A state-specific or state-averaged two-component CASSCF is not implemented.
- Full-electron basis sets only; the ECP limit is the same as in the scalar layer.
- The physical X2C multireference path remains open. The pyscf `int2e_spinor` array does not obey the chemist pair symmetry (measured violation 0.62 on H_2O/6-31G), and the CI Hamiltonian built from it with the determinant-energy ordering does not commute with many-body time reversal on OH/6-31G. Kramers-restricted response is therefore not available on this stack.
- The X2C transform derivative carries a transform-layer central difference at 10^{-4} Bohr, with a measured error about 10^{-7} relative on H_2O. It is not an analytic derivative of the transform chain.

- A scalar-versus-two-component energy difference is not a pure X2C effect when the two layers use different wavefunction methods. Hold the method and the two-electron operator fixed before assigning the difference to relativity.

16.5.3 SOMF and ECP

- The SOMF operator is density dependent. A gradient of an SOC matrix element must include the density response; the Hellmann–Feynman expression alone is incomplete.
- The two-electron kernel is not symmetric under $(pq) \leftrightarrow (rs)$; use `somf_density_response`, not the value contraction, for a derivative.
- The ECP decomposition is not unique. The split between the ECP spin–orbit operator and the explicit valence mean field is a fitting freedom; report the variant bracket.
- Eligibility is a property of the ECP data, not of the atomic number. A scalar-only ECP does not supply `ECPso`.

16.6 Validation and references

The files behind this chapter are `tests/core/test_x2c_casscf_df.py`, `tests/core/test_x2c_df.py`, `tests/core/test_somf.py`, `tests/core/test_somf_ecp.py`, `experiments/df_x2c_casscf_validation.py`, `experiments/df_x2c_gradient_validation.py`, `experiments/somf/validate_oh_of.py`, `experiments/somf_ecp/validate_ecp_somf.py`, and `experiments/somf_ecp/validate_somf_grad.py`. The measured values are the OH/OF splittings, the GeF ECP splitting and gradient, the YbF ECP norms, and the acceptance bounds of Section 16.4.

The X2C construction follows W. Kutzelnigg and W. Liu, *J. Chem. Phys.* **123**, 241102 (2005), and the review in M. Reiher and A. Wolf, *Relativistic Quantum Chemistry* (Wiley, 2009). The SOMF operator follows the ORCA 6.1 manual (Section 5.26), F. Hess, C. M. Marian, U. Wahlgren and O. Gropen, *Chem. Phys. Lett.* **251**, 365 (1996), and F. Neese, *J. Chem. Phys.* **122**, 034107 (2005). The effective-core-potential construction and its spin–orbit integrals are documented in the pyscf `gto` and `ECPso` documentation and in the Stuttgart–Köln ECP papers cited by the `stuttgart_rsc` basis data.

Chapter 17

MPS/DMRG support in the release

17.1 Purpose and scope

The release tree does not contain a tensor-network DMRG solver, and it does not contain the material-diffusion tools of the development branch. What it contains is one graduated DMRG-line utility, `core.mps.pin_degenerate_columns`, which resolves a reproducibility problem that appears before any DMRG calculation can be run: pyscf RHF orbitals are unique only up to arbitrary orthogonal rotations inside exactly degenerate irreps, so two processes can return different representatives of the same degenerate pair. A column-referenced active-space window, such as "columns 6 and 7", then means different orbitals in different runs.

The utility applies to any column-referenced orbital window on a canonical SCF or CASSCF orbital set. It does not select an active space, rank windows, or run a DMRG calculation; it only fixes the representative of a degenerate group. It is NumPy-only and has no JAX or pyscf dependency.

17.2 Theory and derivation

17.2.1 Degenerate subspaces and gauge freedom

Let F and S be the Fock and overlap matrices in the AO basis, and let C be a set of orthonormal molecular orbitals,

$$C^T S C = I, \qquad F C = S C\, \epsilon, \tag{17.1}$$

with $\epsilon = \mathrm{diag}(\epsilon_1, \ldots, \epsilon_{n_{\mathrm{mo}}})$. Suppose a group of columns $G = \{g_1, \ldots, g_m\}$ has

$$|\epsilon_i - \epsilon_j| < \tau_{\mathrm{deg}}, \qquad i, j \in G, \tag{17.2}$$

with τ_{deg} the `deg_tol` parameter. For any orthogonal matrix $U \in O(m)$ the rotated set

$$C'_G = C_G\, U \tag{17.3}$$

is an equally valid orthonormal eigenbasis of the same degenerate subspace: the span is unchanged, the Fock eigenvalues are unchanged, and the energy of any wavefunction built from the complete group is unchanged. Different SCF runs, different linear-algebra backends, or different starting guesses can return different U.

If a column-referenced window cuts a degenerate group, the selected columns are not a well-defined subspace. On N_2/6-31G the RHF spectrum carries the π_u pair at columns $(5, 6)$, the π_g pair at $(7, 8)$, and two higher pairs. A CAS(2,2) on the recorded start window $\{6, 7\}$ scattered over 0.02 Ha across fresh builds although the RHF energy matched to 3×10^{-13}, because the occupied π_u mixture decides which combination sits in the frozen core.

17.2.2 The pinning map

The utility picks a deterministic basis inside each degenerate group by diagonalizing a caller-supplied Hermitian operator X^2 in the group's span:

$$\left(C_G^T X^2 C_G\right) V = V\,\Lambda, \qquad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_m), \tag{17.4}$$

and replaces

$$C_G \leftarrow C_G\, V_{[:,\pi]}, \tag{17.5}$$

where π orders the eigenvalues in descending order. The expectation values λ_i are the diagonal X^2 values of the new columns, and the columns are ordered by that value. The phase of each resulting column is fixed by making its largest-magnitude coefficient positive:

$$c_i \leftarrow \mathrm{sign}\left(c_{i,\mu(i)}\right) c_i, \qquad \mu(i) = \arg\max_{\mu} |c_{i\mu}|. \tag{17.6}$$

Non-degenerate columns pass through unchanged except for this phase convention.

The map has three properties that make it safe to insert into an existing workflow:

- *Span invariance.* A window that contains a degenerate group whole is invariant under Eq. (17.5): the span is preserved, so its CI energies do not move.
- *Determinism.* A split group becomes deterministic. Any single representative of a degenerate pair is symmetry-equivalent, but the process-to-process arbitrariness is removed.
- *Idempotence and gauge invariance.* Pinning is idempotent and invariant to an input rotation applied inside a degenerate group: if C_G is replaced by $C_G U$, the projected operator $U^T(C_G^T X^2 C_G)U$ has the same eigenvectors transformed by U, and the final columns agree.

17.2.3 Conditions and failure modes

The construction is unique only when the restriction of X^2 to the group has a non-degenerate spectrum. If two eigenvalues of Λ coincide, the eigenvectors inside that sub-block are again arbitrary, and the pinning cannot resolve the remaining gauge. The energy of a complete window is still invariant, but a column-split window is not. On a linear molecule the x^2 operator distinguishes the two components of a π pair, which is why it is the operator used in the recorded calibration; a spherical operator would not.

The grouping is an energy window, not a symmetry classification. A `deg_tol` that is too small splits a physical near-degeneracy and pins only part of the group; one that is too large merges distinct states. The recorded test checks both sides of the tolerance ($2.0 + 0.5 \times 10^{-7}$ inside, $2.0 + 5.0 \times 10^{-7}$ outside the default 10^{-7} window).

17.3 Input and operation

Entry point	Returns and purpose
`pin_degenerate_columns`	Arguments `(mo_coeff, mo_energy, x2_ao, deg_tol=1e-7)`. Returns `(C_pinned, n_rotated)`. Every group of columns whose energies agree within `deg_tol` is rotated into the eigenbasis of `x2_ao`, ordered by descending expectation value; each column's phase is then fixed by making its largest-magnitude entry positive.

`mo_coeff` has shape `(nao, nmo)` and is column-ordered; `mo_energy` has length `nmo`; `x2_ao` is the `(nao, nao)` matrix of the chosen operator in the same AO basis, and `deg_tol` is in the energy unit of `mo_energy` (Eh for SCF orbital energies). `n_rotated` counts the columns inside

degenerate groups that were re-rotated; the phase convention is applied to all columns but is not counted. The function returns a new array and does not modify the input.

A minimal call is

```
import numpy as np
from pyscf import gto, scf
from diffmrc.core.mps import pin_degenerate_columns

mol = gto.M(atom='N 0 0 0; N 0 0 1.10', basis='6-31g',
            charge=0, spin=0, verbose=0)
mf = scf.RHF(mol).run(verbose=0)

rx = mol.intor('int1e_r')[0]
C, n_rot = pin_degenerate_columns(np.asarray(mf.mo_coeff),
                                  np.asarray(mf.mo_energy), rx @ rx)
print('re-rotated columns', n_rot)
```

17.4 Worked example

17.4.1 N_2/6-31G: the recorded start window {6,7}

On N_2/6-31G at $R = 1.10$ Å the raw RHF energy is -108.867618373058 Eh. The call above returns `n_rot`=8: four degenerate pairs are re-rotated. A CAS(2,2) on the pinned $\{6, 7\}$ window gives

$$E_{\mathrm{CAS}(2,2)} = -108.8679136653 \text{ Eh}, \tag{17.7}$$

stable across fresh builds to the printed digits. Without pinning the same window scattered over 0.02 Ha. The regression is the `test_n2_6_31g_start_window_reference` case in `tests/core/test_mps.py`.

17.4.2 N_2/6-31G: the exhaustive-window best

The same pinned basis reproduces the exhaustive-window best of the calibration:

$$E_{\mathrm{elec}} = -132.4777614802 \text{ Eh}, \qquad E_{\mathrm{tot}} = -108.9053220847 \text{ Eh}, \tag{17.8}$$

to 8×10^{-11}. The two examples differ in the window, not in the pinning: the first is the recorded start window, the second is the best window found by the exhaustive scan. The module docstring records the calibration and the reproduction margins; the test pins the first window, and the second value is the reference quoted in the same graduation record.

17.5 Pitfalls

- *A split group.* Pinning a subset of a degenerate group preserves that subset's span but not the physical subspace. The window must contain the group whole, or the calculation must use a window defined after pinning.
- *A degenerate `x2_ao` restriction.* If the chosen operator has equal expectation values inside the group, the eigenbasis is again arbitrary. The x^2 operator works for the π pairs of a linear molecule; a spherical operator does not.
- *The tolerance.* `deg_tol` is an energy window, not a symmetry test. A value below the numerical splitting of a physical degeneracy leaves the group split; a value above a physical gap merges distinct states.

- *Non-Hermitian or non-Hermitianized* `x2_ao`*.* The eigenproblem in Eq. (17.4) assumes a Hermitian restriction; a non-symmetric input can return complex or unordered eigenvectors.
- *Wrong basis.* `x2_ao` must be in the same AO basis as `mo_coeff`. A Cartesian component from a different basis or a different unit system defines a different deterministic basis; the span invariance still holds, but the recorded reference values do not transfer.
- *It is not a DMRG solver.* The function fixes orbital representatives for column-referenced windows; the DMRG candidate space and the material-diffusion tools mentioned in the development notes are not part of this release tree.

17.6 Validation and references

`tests/core/test_mps.py` pins the synthetic contracts (pair axes, span, phases, idempotence, invariance to an input rotation, tolerance behaviour, and the no-group case) and the N_2/6-31G molecule regression: RHF -108.867618373058 Eh, `n_rot`$= 8$, pinned CAS(2,2) -108.8679136653 Eh. The test needs pyscf for the molecular case and NumPy only for the synthetic cases. The graduation record is the module docstring of `core.mps`; the original measurements were made in the T5 prototype driver (`code/t416_t5_paper_systems.py` in the development tree) and are summarized in the T5 report, neither of which is part of this release tree.

The construction is the standard resolution of an orbital gauge freedom inside a degenerate subspace; the pyscf orbital conventions and the Fock eigenproblem are documented in the pyscf `scf` and `mcscf` documentation. The x^2-based pinning was chosen for the linear-molecule π pairs because the x^2 restriction has a real eigenvalue gap inside the pair; the calibration and the reproduction margins are the record cited above.

Chapter 18

Manifold-merged entanglement entropy

18.1 Purpose and scope

The active-space criterion of this package ranks orbitals by the four-state entanglement entropy of their one-orbital reduced density matrix. The entropy is defined for one state, and the release computed it for one state. A spin multiplet is not one state: it is a manifold of $2S+1$ degenerate components related by spin projection, and the component that a single-reference calculation returns is a poor representative of the manifold for the purpose of this criterion. This chapter defines the merge that fixes the representative, gives the derivation, documents the API, and describes the verification gate at the robust level that the merge now feeds.

18.2 Why one state is not enough

18.2.1 The four occupation weights

For one orbital, the one-orbital reduced density matrix in the occupation basis $\{|0\rangle, |\uparrow\rangle, |\downarrow\rangle, |\uparrow\downarrow\rangle\}$ is diagonal, and its four weights are

$$w = \big(1 - n_\alpha - n_\beta + p_2,\;\; n_\alpha - p_2,\;\; n_\beta - p_2,\;\; p_2\big), \tag{18.1}$$

where n_α, n_β are the spin-resolved occupations of the orbital and p_2 is the probability of finding it doubly occupied. The entropy is

$$s_1 = -\sum_{k=1}^{4} w_k \ln w_k, \qquad 0 \le s_1 \le \ln 4. \tag{18.2}$$

An orbital that is always empty, always singly occupied with a fixed spin, or always doubly occupied has $s_1 = 0$; an orbital whose occupation fluctuates strongly approaches $\ln 4$.

18.2.2 The $M_S = +S$ component of a high-spin determinant

Consider the component of largest spin projection of a pure high-spin state. It is a single Slater determinant, and every open shell carries exactly one electron of the majority spin. For such an orbital the triple is

$$n_\alpha = 1, \qquad n_\beta = 0, \qquad p_2 = 0, \tag{18.3}$$

so the weights are $w = (0, 1, 0, 0)$ and

$$s_1 = 0 \tag{18.4}$$

for every open shell. The nitrogen atom in its ^4S ground state is the smallest example: three singly occupied $2p$ orbitals, and the single-state criterion reports zero entropy for all three, although a correct description of the atom requires all three in the active space.

The zero is not a numerical accident. In that determinant the orbital is in a definite Fock state, so there is no entanglement between it and the rest with respect to the spin degree of freedom. What the determinant does not show is the degeneracy: the same electronic structure appears in the $M_S = -S, \ldots, +S$ components, and the physical state is the manifold, not one projection of it.

18.2.3 Merging the manifold

Let $\{c^{(k)}\}_{k=1}^{R}$ be the CI vectors of the R components, with normalized columns $\sum_I |c_I^{(k)}|^2 = 1$, and let ω_k be their ensemble weights. The mixed one-orbital reduced density matrix is the weighted mean of the per-state matrices. The map from the triple (n_α, n_β, p_2) to the weight vector in Eq. (18.1) is affine, so the weights of the mixture are the weighted mean of the per-state weights:

$$\bar{n}_\alpha = \sum_k \omega_k n_\alpha^{(k)}, \qquad \bar{n}_\beta = \sum_k \omega_k n_\beta^{(k)}, \qquad \bar{p}_2 = \sum_k \omega_k p_2^{(k)}. \tag{18.5}$$

No density matrix is assembled, and the cost is one matrix product per root: the per-state occupations follow from the determinant bitmaps already used by the single-state route.

For a complete spin manifold the mean is spin-isotropic, $\bar{n}_\alpha = \bar{n}_\beta$, and a singly occupied shell obtains

$$\bar{n}_\alpha = \bar{n}_\beta = \tfrac{1}{2}, \qquad \bar{p}_2 = 0, \qquad w = \big(0, \tfrac{1}{2}, \tfrac{1}{2}, 0\big), \qquad s_1 = \ln 2. \tag{18.6}$$

The value $\ln 2$ is the entropy of one electron of unknown spin in one orbital, which is exactly what the criterion should read for a degenerate open shell.

18.3 Which roots form the manifold

For a pure spin state of spin S the manifold has $2S + 1$ components, and in a calculation that returns the lowest eigenvalues in ascending order they are the lowest $2S + 1$ roots, because the components are degenerate. The gate therefore takes

$$R = 2S + 1, \tag{18.7}$$

which the driver evaluates from the spin multiplicity of the request. The assumption fails when a state of different spin and lower energy lies inside the same window; the merged occupations then contain a foreign state. The worked example in Sec. 18.6 is the case where the assumption holds and can be checked.

18.4 API

The analysis layer exposes the occupation triple and the entropy of a manifold:

```
from diffmrc.analysis.ci import (manifold_occupations,
                                 manifold_entropy_from_ci)

# vectors: shape (ndet,) or (ndet, nroots), in the determinant order
# produced by core.nevpt2.impl.cas_dets_for2
s1 = manifold_entropy_from_ci(vectors, dets, ncore=ncore, nact=nact)
n_a, n_b, p2 = manifold_occupations(vectors, dets, ncore=ncore,
                                    nact=nact)
```

Each column is normalized before use; the mixture weights carry the ensemble, not the arbitrary scale of a vector. The default is equal weights. A weight vector shorter than the root count is padded with zeros, and negative weights, a zero weight sum, more weights than states, and an all-zero column are refused with a message.

The differentiable sibling, used when the entropy is itself a function of the orbital parameters, is `driver.entropy.manifold_entropy_from_ci_masks` (masks precomputed once per determinant list) and `driver.entropy.manifold_single_orbital_entropy_from_ci` (determinant list given directly). Both are bound to the NumPy route by a test at 10^{-12}.

The CI vectors of a converged calculation are obtained without re-solving in a different basis:

```
from diffmrc.driver.kernel import casci_vectors_at

vectors = casci_vectors_at(mol, C, space, theta, nroots=2 * S + 1)
```

`casci_vectors_at` shares the integral block and the orbital rotation with the energy route, so the vectors belong to the reported energy.

18.5 The robust-level gate

At the robust level the driver evaluates the merged spectrum after the CASSCF has converged and records two entries of `algorithm_data`:

```
{'active_space_verified': True | False,
 'active_space_entropy': {'s1': [...], 'nroots': R,
                          'verdict': '...', 'plateau_heights': [...],
                          'evidence': [...]}}
```

The check passes when the merged spectrum is not single-reference, that is, when its largest single-orbital entropy reaches the multireference criterion of 0.14. A verdict of `single_reference` fails the check, and the robust level refuses the run: there is no active space to verify. A verdict that the spectrum is larger than the selected space is recorded but does not veto the run, because the element rules add ligand orbitals whose entropy lies below the line by construction; vetoing them would contradict the selection rules themselves.

Two properties of the gate should be stated plainly.

Scope. The spectrum is the one the selected space can show. An orbital outside the space has zero entropy by construction, so the gate can certify that the space is multireference and that its orbitals are not noise; it cannot certify that the space is not truncated at the frontier. Completeness at the frontier remains the responsibility of the selection rules.

Failure to evaluate. If the spectrum cannot be evaluated the gate reports unverified with the reason, rather than raising. At the robust level an unverified space is a refusal, which is the same outcome the exception would have produced and a more useful record.

18.6 Worked example: the nitrogen quartet

The ^4S ground state of the nitrogen atom, CAS(3,3) over the three $2p$ orbitals with the 6-31G basis, at the ROHF reference orbitals. The calculation returns four roots; Table 18.1 lists the single-orbital entropy of each root and of the merged manifold.

Root 2 is the determinant of Eq. (18.4) and reports zero; roots 0, 1 and 3 are the other projections and report values that depend on the projection. The merged value equals $\ln 2 = 0.693147$ for all three orbitals to the printed precision, which is the spin-isotropic result of Eq. (18.6) and confirms that the four roots form the complete manifold.

For comparison, two closed-shell cases run through the same gate report 4.787×10^{-3} (H_2O/6-31G, CAS(2,2) over the two frontier orbitals) and 6.792×10^{-2} (H_2/STO-3G, CAS(2,2)). Both

Table 18.1: Single-orbital entropy of the N ^4S manifold, CAS(3,3)/6-31G. Each root is a component of the quartet; read individually the roots disagree by three orders of magnitude, and the merged value is $\ln 2$.

root	$s_1(2p_x)$	$s_1(2p_y)$	$s_1(2p_z)$
0	6.60849×10^{-1}	6.60849×10^{-1}	6.60849×10^{-1}
1	6.36514×10^{-1}	6.36514×10^{-1}	6.36514×10^{-1}
2	2.2×10^{-25}	1.5×10^{-24}	5.1×10^{-25}
3	1.68823×10^{-1}	1.68823×10^{-1}	1.68823×10^{-1}
merged	6.93147×10^{-1}	6.93147×10^{-1}	6.93147×10^{-1}

lie below the 0.14 criterion and are reported as single-reference, which is the intended outcome: an active space is not justified for them.

18.7 Limitations

Root selection. Equation (18.7) assumes that the lowest $2S+1$ roots are the components of the target manifold. A lower state of another spin inside the same window contaminates the mixture. The assumption can be checked by printing the root energies and confirming degeneracy, which the driver records in the report but does not enforce.

Equal weights. The default weights are equal, which is the state-averaged convention for a degenerate manifold. A manifold split by spin-orbit coupling is not degenerate; the API accepts explicit weights, and the caller is responsible for choosing them.

Within-space scope. As stated in Sec. 18.5, the spectrum cannot see orbitals that the selection left out.

A pre-existing failure on the quartet. The end-to-end driver evaluates the orbital Hessian for its curvature check, and on the N ^4S case that Hessian is not finite: at the converged point, which sits within 2×10^{-7} of the symmetric $\theta = 0$, all 676 entries are NaN, while a random displacement of 0.01 gives a finite Hessian. The exact degeneracy of the high-spin multiplet makes the second derivative of the blocked eigensolver undefined. The driver now recovers the curvature from a symmetric difference of the analytic gradient, which is finite at the same point, using a step of 10^{-5} in the rotation parameters. The route costs two gradient evaluations per parameter and is capped at 300 parameters; a rotation space with more than 300 parameters is refused with the diagnosis rather than approximated by the difference route. On N_2 the spectra at steps 10^{-5}, 10^{-4} and 10^{-3} agree (minimum 0.0648, maximum 66.96, no negative eigenvalues), and `algorithm_data['curvature_method']` records which route ran. The numbers in Table 18.1 were produced by calling `casci_vectors_at` directly at the SCF orbitals. The defect is recorded here so that it is not mistaken for a property of the merge.